

`/* Code For Your Life */`

“Wise, compassionate, and inspiring.”

—Judith Siegel



`{ John Arundel; }`

Contents

Praise for ‘Code For Your Life’	11
Introduction	12
1. Directions	13
A career ending mistake	13
The end	13
Out of control	14
Choose your destination	14
Three kinds of careers	15
Seniority	15
What does a senior IC do?	15
What they don’t do	16
How to get there	16
Seniority means mastery	16
Management	17
What does a manager do?	17
Being a great manager	17
Getting started	18
Independence	18
The pros and cons of independent working	18
When it’s time to leave	19
Testing the waters	19
Making the choice	19
Figuring out who you are	20
Making small course corrections	20
It’s about the planning, not the plan	20
2. Jobs	21
A typology of tech jobs	21
Dev versus ops: the caste system	22
“Have you tried turning it off and on again?”	22
What in the world is “DevOps”?	22
QA	23
Making yourself an easy hire	23
Personal recommendation	23
Making connections	24
Networking groups	24
Be interesting, and interested	25

Becoming a brand	26
Being a big name	26
How to be internet famous	27
You need a website	28
This is your time to suck	28
Building a portfolio of projects	29
When to start looking for new jobs	29
Keep cashing the checks	30
Leveraging an offer	30
Don't be a dinosaur	31
Avoiding the expertise trap	32
Where to look for jobs	33
Apply to employers, not jobs	33
Recruiters	33
Choosing jobs to apply for	34
Apply to jobs you're unqualified for	34
The experience paradox	34
Mastering the application	35
Knowing the keywords you need	35
Tailoring the application	36
What not to say	36
References	37
3. Interviews	38
Preparation	38
Plan for the long haul	39
Interviews are more demanding than you think	39
Keep the job listing for reference	39
Strike while the job is hot	40
The initial phone screen	40
Be on time and ready	40
Double-check your setup	40
The conversational interview	41
Talking about yourself	41
Telling your success stories	41
Stick to the point	42
The tricky questions	43
"Why do you want to change jobs?"	43
"What's your greatest weakness?"	43
"What salary are you looking for?"	44
Turning the salary question around	44
The technical interview	44
Non-technical interviewers	44
The dreaded whiteboard task	45
It's not about your program	45
Show you know how to collaborate	45

Learn some CS basics	46
But practice beats theory	46
The take-home task	47
Scope and time management	47
It's about the programming, not the program	47
Documenting your process	48
Aim to impress	48
The in-depth interview(s)	48
Be prepared for a multi-stage interview process	49
But don't persist if it doesn't feel right	49
It's about who you are, not what you know	49
Show that you're a smart person who gets things done	50
"Do you have any questions for me?"	50
Some example questions	50
After the interview	51
Thank your interviewer	51
When you get ghosted	51
Surviving a "no"	51
Getting feedback	52
4. Negotiation	53
The offer	53
You don't have to accept right away	53
Negotiation is not a dirty word	54
The negotiation	54
Knowing the numbers	54
Making your counter-offer	55
What if they won't negotiate?	55
Be prepared to (politely) walk	56
Keep negotiating as long as necessary	56
Let them win a little too	56
Equity as compensation	56
You're getting a stake in the company	57
What are stock options worth?	57
You may not get to exercise options anyway	57
Valuing options in startups	58
Beyond cash and equity	58
Creative negotiation	58
Can it ever hurt you to negotiate?	59
Only bad employers will view you negatively	59
Negotiation is a key skill for any job	59
Negotiating your contract	59
Check the fine print	59
Get the whole agreement in writing	60
You'll have no power to change the agreement later	60
Take your time and don't give in to pressure	60

The anatomy of a contract	61
What doesn't it say?	61
Compare with similar contracts	61
The contract doesn't matter, until it does	61
Moonlighting and spare-time work	62
Intellectual property	62
Making the final decision	62
Expect pushback, but don't yield to it	62
Remember you can still walk away	63
Assume this is the best the deal will ever be	63
5. Work	64
Starting out	64
Can I even do this?	64
Impostor syndrome	65
You can do this	65
Don't show off	65
Listen before you speak	66
How to make suggestions	66
Settling in	66
Coping with "Day 1 Panic"	66
The trial period	67
Expect to be slow at first	67
This is what learning feels like	68
Being effective	68
Social capital	69
Managing your commitments	69
Saying "no" is a superpower	69
Finding your feet	70
Don't rearrange the furniture	71
Making the right first impression	71
Collaboration is your biggest challenge	71
People skills	72
Human relationships	72
Overcoming shyness	72
Social bonding	73
Listening	73
Reflection	74
Don't make it about you	74
Being a good co-worker	75
Keep your opinions to yourself	75
Don't be an advocate	75
Exercising empathy	75
Meetings	76
Less noise, more signal	76
The stakes can be high	76

No one likes a doom-monger	77
We just want to be visible	77
Validating and being supportive	78
“Say my name”	78
Being a good co-worker is your job now	79
Courtesy is a sign of strength	79
6. Time	81
Being a Time Lord	81
Control your schedule	82
Make a plan	82
Make time for what really matters	82
Things to do: make a list	83
Get everything out of your head	83
Deep work	84
Schedule deep work deliberately	84
Remove all distractions	84
Don’t overdo it	85
Not all work is deep work	85
Make yourself uninterruptable	87
Attention is a muscle: flex it	87
Protect your time	88
Sometimes helping means not helping	88
Skip useless meetings	89
Decline most invitations	89
Saying “no” nicely	90
“Not yet” is almost always fine	90
Respect other people’s time, too	91
Don’t say “hi”, just ask the question	91
Be punctual to be polite	92
Don’t renege, renegotiate	92
7. Mastery	94
Choosing your path	94
Should you stay or should you go?	95
You’ll need to supply your own challenges	95
Broadening your mind	96
Being T-shaped	96
Try as many languages as you can	96
Finding your love language	97
Walking the path	97
Be prepared to train hard	97
Write every kind of program imaginable	98
Learning how to learn	98
Work on solo projects	99
Where does mastery come from?	99

Do you need a college degree?	99
Should you buy software engineering books and courses?	100
Save your money for professional coaching	101
A coach can hugely accelerate your learning	101
Potential mentors are all around you	102
You can mentor others without being an expert	103
Pair programming	103
Good pairing is about trust	103
How pairing works in practice	104
Pairing problems, and how to fix them	104
Pairing distributes knowledge around the team	105
Don't try to pair all the time	106
Code review	106
The life and times of a changeset	106
Pair reviewing	107
The gentle art of code review comments	108
Accentuate the positive	109
When your vanity is wounded	109
"I see what you tried to do here"	110
Stylistic nitpicks	110
The end of the rainbow	111
8. Seniority	112
Beginnings	112
Juniority	112
Doing your One Job	113
When do you become senior?	113
Going further: management or staff-plus engineer?	114
Recording, thinking, and planning	114
Visibility	115
Learning the ropes	116
Every organisation is also a game	116
Patterns for success... or failure	116
Don't be an office politician	117
Dirty tricks, and how to avoid them	117
Winning by not playing	118
Moving on up	118
Mentoring: from mid-level to senior	118
Supporting and encouraging	119
Absorb blame, distribute credit	120
Be a product engineer, not a software engineer	120
9. Leadership	122
Tech leading	122
What is a tech lead?	122
What does a tech lead do?	123

Conflict and consensus	123
“Is there anyone who can’t live with that?”	124
Negotiating from values, not positions	124
Are you a potential manager?	124
The staff engineer track	125
What does a staff engineer do?	125
Your job is not to fix everything	126
Bridging the gap between engineering and management	126
Staff-plus-plus	127
Staying technical	127
Managing up	128
What is a manager’s job?	128
How to be managed well	129
The manager README	129
Keeping your manager informed	130
Planning promotion	130
Get your manager on your side	130
Don’t wait for the annual review	131
Advancement doesn’t just happen	131
Doing good work isn’t enough	132
Quality <i>and</i> visibility	132
What’s the goal?	132
10. Money	133
Your financial position	133
Making a budget	133
Your net worth	134
Changing the curve	135
Money management	135
Building up an emergency reserve	135
Don’t wait for something to turn up	136
Systems are fun	136
Putting your money to work	137
Investment plus time equals wealth	137
How much cash do you need?	137
Real estate: buy, don’t rent	138
The stock market	138
Pensions	139
Money buys freedom	139
11. Agency	140
Preparing for the jump	140
Finding your niche	141
Resigning in your head	141
Consulting	142
Emotional distancing	142

The secret consultant	142
Do a stint in consulting	143
Don't solve the problem too quickly	143
You can't "win" this game	144
Let them solve their own problem	144
Time to leave?	145
Avoiding the comfort trap	145
Why is everyone around me getting dumber?	145
The good engineers are evaporating	146
Stranded by the tide	146
The company won't love you back	147
When it's time to quit	147
Say no to exit interviews	148
12. Independence	149
Working to live	149
Being rich is not the goal	150
Build a life, not a fortune	150
Running a business is work	150
Sources of revenue	151
Selling your time	151
Making money	152
How much should you charge?	152
Don't take fixed-price contracts	152
Estimates and change budgets	153
Ratcheting up your rates	153
Don't drop your price to get the deal	154
The vendor-client relationship	154
When the client doesn't pay	154
How to fire a client	155
Trust your first impressions	155
How to be a great vendor	155
"Policy" just means "we don't want to"	156
Building products	156
Running out of time	156
Selling software is hard	156
Expect tough competition	157
Knowledge products	157
You can be your own publisher	158
Don't over-think the marketing	158
Your true fans are your marketing department	159
Control	159
Independence, or co-founders? Pick one	159
Keep ultimate control	160
A business does not need to grow indefinitely	160
Hiring: whether, when, and how	161

An army of one	161
Making it	162
What does “making it” mean for you?	162
Pre-tirement	163
The beginning	163
About this book	164
Who wrote this?	164
Feedback	164
Free updates to future editions	165
Join my Go Club	165
For the Love of Go	165
The Power of Go: Tools	166
The Power of Go: Tests	167
Video course	169
Further reading	169
Credits	170
Acknowledgements	171

Praise for ‘Code For Your Life’

This book contains a lot of very well-written lessons that I’ve learned on a harder, longer path. I wish I’d had this book twenty years ago.

—Ivan Fetch

All John’s books are an insta-buy.

—Eldon Alameda

John’s books are phenomenal. Highly recommended.

—Patrick Picard

Full of wise, compassionate, and inspiring life advice for software professionals.

—Judith Siegel

Introduction

This is my attempt to distil the lessons I've learned from many decades in the tech industry into a practical, readable how-to manual. If you're about to embark on your career, or you've only just got started, this book is here to encourage you, to advise you, and to help you avoid some common mistakes and pitfalls. It will help you plan where you're going, and give you some solid tips on how to get there.

But it's not just for those new to tech. If you're already well established in your career, but feeling a bit stuck or frustrated about your progress, you'll find advice here on how to break through to the next stage. Should you move into leadership and management, or stay in a technical role? I'll help you scope out the options and where they might lead.

If you've risen as high as you can in your current company, but aren't sure you're ready to run your own firm, or what that would look like, this book will help you decide. And if, like me, you've already declared independence, I hope you'll find some valuable tips on running a successful business. Maybe you'll be looking to hire engineers yourself one day. If so, give me a call: I know some great people.

Your career is a journey, and people who don't know where they're going tend to end up walking uselessly in circles. Or, even worse, ending up somewhere they didn't *want* to be. But choosing the right destination isn't enough by itself, if you can't then figure out how to *get* there from here.

So let's start at the end.

1. Directions

If you don't change direction, you may end up where you're heading.
—Saying



A career ending mistake

This isn't about the time I inadvertently shut down one of Britain's nuclear power stations, an entirely true story for which the world is nevertheless not yet prepared. Nor is it about the poor junior developer who [accidentally destroyed the production database](#) on their first day (they'll be fine, bless them).

Instead, I want to talk about another kind of career ending mistake, one that affects more than just the unlucky few. Indeed, it's a mistake we're probably all making right now. And that's *not planning the end of our careers*.

The end

By “the end”, I don't necessarily mean picking your retirement date. This is really about the aim or goal of your career. Where will you be when you realise that this is where you've always wanted to be?

If you love what you're doing now and don't ever want to change jobs, great: you've reached the end of your career, even if it plays out over many decades. If you don't love

it, though, and that's much more likely, then it's worth asking what job you *would* love, and how you're going to get it.

So, where do you want to end up? And is that where you're currently heading? If not, what should you do about it?

Out of control

The word “career” means, among other things, “to rush about wildly”, or, of a vehicle, “to move fast and in a way that is out of control”. Isn't that apt? And doesn't it describe the career experience of most of us pretty well?

The indispensable first step to getting what you want is this: decide what you want.

—Saying

Most of us, in fact, don't really know what we want to do with our working lives until we're more or less doing it. By the time we've made our minds up where we want to go, we may already be a long way down the wrong track.

It's not surprising, then, that many of us find ourselves in less than fully satisfying jobs, with doubtful or non-existent prospects for advancement. In all honesty, if we *are* happy in our jobs, it's probably more often a matter of luck than of design.

Maybe rather than rushing around wildly, we should give a little thought to what we want to achieve in our careers. Rather than just sit back and hope we get the career we want, maybe we should actively plan and take steps to bring it about.

As software engineers, we're constantly making detailed, elaborate plans for computers to execute; that's what software is, after all. Isn't it kind of weird that we rarely give a moment's thought to the program we want to write for ourselves?

Choose your destination

The best time to start a pension is always twenty years ago, and career planning is a bit like that, too. By the time you realise you need it, you usually wish you'd started a lot earlier. On the other hand, it would be foolish to let that prevent you from starting at all. Even a late, sketchy, and uncertain plan is way better than no plan.

So what would a career plan look like? And when should we start making it?

The first phase of your career is probably too early to make serious plans, and any decisions you make at this stage are rarely critical: there's plenty of room to experiment and make mistakes.

In the final phase, by contrast, you have fewer options, and there's not much time to pull off a significant change of direction. The autopilot tends to lock in and steer you further down the glide path you're already on.

You should be planning, in other words, to enter the final phase of your career in the right position, at the right level, and at the right time to land where you want to. So where is that, exactly?

Three kinds of careers

Your career is likely to encompass lots of different things. Over the years, you'll do all sorts of different jobs at different companies, and the perfect career destination for you may end up being something you can't even imagine right now.

To help us think about it, though, let's divide the possibilities, very broadly, into three groups:

1. Seniority
2. Management
3. Independence

In the rest of this chapter, we'll take a closer look at each of these destination airports, check out the local weather conditions, and ask what kind of flight plan would get us there. Throughout the rest of the book, we'll explore the strategy and tactics you'll need along the way.

So, what kind of career do you want to have?

Seniority

The first kind of career destination we'll consider is *senior individual contributor* (IC), which usually means something like "high-level engineer". A senior IC doesn't run their own firm, or work independently. They're at the top of their game technically, and may be a technical leader for others, but they spend the majority of their time *doing* the work, not managing other people who do.

What does a senior IC do?

The exact ranks and job titles for senior engineers vary from one company to the next, and there may be many levels for you to achieve within a given company. But in terms of your *eventual* career goal, let's say we're talking about the top level you can reach while still being primarily an engineer.

In a small company, that might simply be something like "senior developer", or even team leader. In larger companies, the pinnacle of engineering pay and responsibilities might be called something like "principal engineer" or even "distinguished engineer" (sounds good, doesn't it?).

This isn't an executive position: you won't have your own washroom, or a seat on the board. On the other hand, you'll have plenty of money, status, and authority, and unlike your colleagues in management, you won't spend all day in meetings. You'll be at the highest level of your profession, and getting well paid to excel at the work you love.

Your work is going to fill a large part of your life, and the only way to be truly satisfied is to do what you believe is great work. And the only way to do great work is to love what you do.

—Steve Jobs, [Stanford commencement address](#)

What they don't do

There are a few limitations, though. You probably won't get to choose *what* to work on, and you may not agree with all the decisions of the powers that be. In fact, it's practically certain you won't. After all, you know more about the subject matter than they do. If, in the end, you can't live with those decisions, you can go work for another company, but of course you'll find the same dynamic there.

A senior IC role appeals to those who want to stay technical and keep their hands on the keyboard, or at least the mouse. You don't have the time-consuming, cross-disciplinary responsibilities of a business owner, or the political challenges of a manager. You get to do all the fun stuff: the building and making. And you can do it until you choose to retire—hopefully in financial comfort.

How to get there

Find out what the situation is where you work. If senior ICs are a thing in your company, talk to them. Ask them for advice. How did they get where they are? What were the steps along the way? And is it what they thought it would be?

Talk to your boss. Do they know this is what you want? What are they prepared to do to help you get it? And what will they expect from you in return? Establishing this dialogue is important, but it's not enough: you need to keep it open throughout your employment. Check in with them often on how you're doing, and what more you can do to help them help *you*.

Don't wait for annual reviews. A year is too long to wait to find out that you've made no progress towards your career goal. Some people blithely assume that if they keep doing what they're doing for long enough and don't screw up, they'll eventually make senior IC.

Well, maybe. But a more reliable strategy is to take personal control of your skills development, and start taking it seriously.

Seniority means mastery

The most senior engineers in any company are the most accomplished, the most experienced, and those who have the most to teach others. If you want to reach this level, you'll need to become a master of your chosen craft.

Don't wait for the company to train you. They prefer to promote people who don't need training. Instead, once you've found the craft you love, you'll need to dedicate yourself to mastering it.

Start seeing your current work not as a simple exchange of your time for their money, but rather as an opportunity to find out what you're good at, and to get better at it. A professional coach can help you figure this out, support your learning, and keep your eye on the ball.

Manhood is patience. Mastery is nine times patience.

—Ursula K. Le Guin, “[A Wizard of Earthsea](#)”

Management

While very senior engineers can be well-compensated, their roles are usually not the most senior in the *company*. If you want to rise even higher in pay and responsibility while staying within the firm, you may need to think about leaving your technical work behind and switching to *management*.

Management is perhaps the default destination for many tech careers. Provided you can stay in a company long enough, the natural path to further advancement may be for you to become a manager.

If you don't change direction, in other words, you may end up where you're heading. But *is* that where you want to go?

What does a manager do?

Engineering managers need a solid foundation of technical competence, to be sure, but the work itself is primarily about leading, supervising, hiring, and developing the skills of other technical people. It turns out those are all skills, too, and relatively rare ones.

Managing people is hard; much harder than programming. Computers just do what you tell them, whether that's right or wrong (usually wrong). Anyone can get good at programming, if they're willing to put in enough time and effort. I'm not sure anyone can get good at managing, and most don't. Most managers are *terrible*.

That's quite a sweeping statement, I know. (Prove me wrong, managers, prove me wrong.) But, really, would a car mechanic last long in the job if they couldn't fit a tyre, or change a spark plug? Would a doctor succeed if they regularly amputated the wrong leg? We would hope not. But many managers are just as incompetent, in their own field, and yet they seem to get away with it.

Being a great manager

Good managers, then, like good teachers, are rare, but all the more precious for it. If you've ever had a really good boss, you'll remember them all your life, and, if you're lucky, emulate them. (You'll remember the really bad ones, too.) And just because managers don't cut code or solder chips, it doesn't mean they don't have a big influence on the success of projects and companies.

Indeed, managers can have an outsize influence on events. I'm sure we can all cite examples of promising projects that sank without trace because of a disastrous manager. I'm less sure that there are many examples of inspirational managers rescuing doomed projects from the brink, though it does happen.

If you want to become a great manager, which I think is the only kind worth being, start practising now. Learn people skills, communication, collaboration, psychology. Work on understanding the things that make different kinds of people tick. Manage *yourself* excellently. If you can't organise yourself, how do you expect to be responsible for a team?

Getting started

Study your own manager. If they do the job well, figure out why (and talk to them about it). If they're a shambles, figure out what they're doing wrong, and decide how you'd do better.

A great manager understands what's happening with each person in their team, and can be there to eliminate problems and roadblocks almost before they happen. Why shouldn't you start doing this kind of thing right now, rather than waiting to be told?

Sometimes the de facto leader of a team is simply the person that everyone turns to when they have a problem they can't solve on their own. If that's you, you may already be on the road to becoming a memorable manager—for the right reasons.

Independence

Being independent means working for yourself, most likely in your own company, and maybe with others working for you, but also maybe not. Just being a one-person company doesn't necessarily make you truly independent, though. For example, are you a consultant or a contractor?

While the client tells a contractor what to do, a consultant tells the *client* what to do. The difference matters. A consultant is independent; a contractor is not.

The pros and cons of independent working

Running your own business, or otherwise being an independent worker, is great for those who like it. I do, and I was never really happy working for someone else. I couldn't wait to strike out on my own. I was probably a pretty mediocre employee for that reason, among others. And there was the little *nuclear* incident, of course.

On the other hand, not everyone wants the hassle of figuring out how to market their business and pitch clients, or the headache of handling accounts and taxes. And not everyone can manage on an irregular, unpredictable income, especially if they have a family to support. You don't get vacations, insurance, or sick pay. On the plus side, you own a business.

When it's time to leave

If you do want to swap your stable job and decent salary for the joys of career independence, the transition needs a little careful planning. It would be unwise, for example, to just quit one day in a fit of pique, *then* start wondering how you're going to make rent the first few months. Timing is important.

DENPOK: *Lao Tzu teaches: the best fighter is never angry. More important than the blow is knowing **when** to strike. Like, perhaps, after we experience the executive whitewater rafting trip in Coeur d'Alene.*

—“Silicon Valley”

And to make money on your own, you'll need to be *excellent* at what you do. There's no one else to pick up the slack. In a big company, you can learn on the job. When you run your own company, you'd better already know your trade.

Testing the waters

If you can make the time, it's a good idea to dip your toe into independent working by doing a few small, one-off side gigs. You'll gain experience and some satisfied clients, making it easier to go fully independent when you want to. And if it turns out that you don't enjoy the experience of working for yourself, it's better to find that out *before* you rage-quit your job, isn't it?

Of course, we don't always leave our jobs through choice. Layoffs are a fact of life in a volatile industry. You may not see the rocks coming; companies tend to fail gradually, then suddenly. But an unexpected transition to “funemployment” needn't be a disaster. If you've always dreamed of being independent but somehow never quite worked up the courage to jump ship voluntarily, then being made to walk the plank could be just the spur you need.

Making the choice

I hope I've encouraged you to think about where your career is going, where you *want* to go, and what you can do to get there. Of course, you may not yet know how you want to spend the bulk of your career. That's okay, and completely normal.

But you can *think* about it, even if it's too early to come to any firm decisions. And you're not limited to just *one* of these potential destinations: many successful careers combine seniority, management, and independence in some way.

For example, you could be a full-time or part-time manager, and also run your own business on the side. Or you could achieve independence by being a roving consultant within a large organisation, while still technically being a senior engineer. Or you could combine all three by being the chief executive *and* the director of engineering in your own firm. There are many possibilities.

Figuring out who you are

You can inquire of yourself what things you value, and how your working life could contribute to them. As your experience and knowledge of the world grows, ideas may start to slot into place for you about what you want to do and be.

*Tell me, what is it you plan to do
With your one wild and precious life?*

—Mary Oliver, “The Summer Day”

Your future may not, in fact, lie in the tech industry. That’s all right, too. I have more than one friend who, despite achieving considerable success as an engineer, has decided that this isn’t really what they want to do in the long term.

If you want to quit and be a doctor instead, or a schoolteacher, a [spaaaaaace](#) engineer, a woodworker, or simply wander the world like a [badass](#) righting wrongs, go to it. Don’t waste any more of your one wild and precious life careering down a blind alley.

Making small course corrections

Once you do have a sense of where you want to go, it can help guide your choices. Even if you don’t know exactly what your perfect job looks like, you may start to feel that you won’t be truly happy until you’re independent, or a senior IC, or a manager. You can steer away from things that would limit your options in those areas, and instead seek out companies, fields, or sectors where you’ll have the best chance of achieving the career you want.

That’s not to say you should have a detailed map of every step that you plan to take (“make junior VP by Q4 2035”). As engineers, we already know that a too-rigid plan rarely survives contact with reality. Instead, assume life will throw all kinds of crazy and unexpected things at you. Plan to be flexible, and to change your plans.

It’s about the planning, not the plan

You can’t stop the waves, as the saying goes, but you can [learn to surf](#). Chance favours the prepared mind. Never underestimate the role of serendipity. The perfect opportunity may show up just when you least expect it, but if you’ve never thought about what you want, how will you recognise it?

The time to start planning for the end of your career is now. It’s never too early, and it’s also never too late, provided, of course, that you don’t have your own little *incident*. Let’s be careful out there.

2. Jobs

The only productive way to answer “what should I do now?” is to first tackle the question of “who should I become?”

—Kevin Kelly, [“Excellent Advice for Living”](#)



In the previous chapter we looked at some possible career destinations: seniority as an engineer, the management track, or being independent (which combines the two). Maybe you already know the destination you want, or maybe you’re still thinking about it. Either way, let’s look at some of the possible routes by which you could get there.

As we’ve seen, it’s unlikely that anyone will *start* their career as an independent or a senior IC, and the same applies to management. So your route to the career destination of your choice is likely to be through one or more *jobs*. How do you find the right job, then, and what should you do to maximise your chances of getting it?

A typology of tech jobs

First, what kind of job should you be looking for? To keep things simple, let’s focus on two main fields of employment in the tech industry: *development*, and *operations*. They aren’t really different fields, but that’s how the industry sees them, so it’s important to understand why, and what difference that makes.

Dev versus ops: the caste system

Development jobs are primarily about writing software. Operations jobs are primarily about *running* software. In general, development jobs are perceived as relatively high status, and difficult to get. Operations jobs are, broadly speaking, low status, and difficult to get *out* of. Let's unpack that a little.

It's a sort of caste system. Software development is perceived to be difficult and mysterious, and those who can do it are seen as a superior *kind* of person: a member of the priestly caste. Not just anyone can do it, in other words: you almost have to be born into it.

“Have you tried turning it off and on again?”

By contrast, operations (often referred to broadly as “IT”) workers are members of the low caste. The perception is that operations work is boring and trivial, and anyone can do it. There's no real distinction in many people's minds between, say, fixing a Kubernetes cluster, and fixing a printer. It certainly doesn't have the same prestige as software development.

Actual operations work nowadays, though, it's important to say, is bound to involve coding in various ways. And, since you'll be writing and maintaining the software that provides the infrastructure for everyone *else's* software to run on, this can be some of the most interesting work you'll ever do as a programmer.

The caste system is dumb, insulting, and just plain wrong. In reality, we're all in basically the *same* job: trying to make computers work properly. But the choice of door you walk in through makes a major difference to your career path, compensation, and promotion opportunities.

So it's wise to start by going through the door labelled “development”, because you'll always have the chance to branch out into operations work later. By contrast, being perceived as a lowly IT worker will limit your future options.

If operations is your thing, though, don't let me stop you. Depending on the company, this can be a great route into tech, and for those without any industry experience at all, it can be a way to get a foot in the door without going through the intensely competitive developer interview process.

What in the world is “DevOps”?

You got me. I don't know. But nor does anyone else: while most people agree that “DevOps” is some kind of crossover between development and operations (hence the name), there's no real consensus on what it means as a *job* title.

Personally, I think “DevOps” is best used to describe a mindset or way of working: one in which development and operations are not *seen* as distinct kinds of work, done by different groups of people. After all, it's no good just *writing* software: you need to deploy it, run it, monitor it, maintain it, and manage the infrastructure estate it depends on.

And the best people to do that are the same people who wrote the software in the first place: they know what it's supposed to do.

So in a company where DevOps is a thing, there's no distinction between dev and ops: you build it, you run it. On the other hand, many companies think DevOps is a job title describing someone with operations skills who can also write code. So if you see a job advertised for a "DevOps engineer", it would be wise to find out what *they* think that means.

QA

There's also *quality assurance* (QA), which can be perceived as part of either development or operations, depending on the company and the team. You'll be involved in testing software, of course, but how you do it matters. If that testing is purely manual (for example, following scripted procedures to use the software and note down any bugs), then this is seen as pretty low-status, and you'll find it hard to break into development later if that's really what you want to do.

On the other hand, if you're involved in test *automation* (for example, writing code that tests other people's code), then some companies consider this essentially a development role, and refer to you as a Software Development Engineer in Test (SDET). This is more like being a developer with an additional expertise in testing, and can be a very high-status job in some firms.

Making yourself an easy hire

We've said that development jobs are hard to get, and it should be clear by now why that is: everyone wants to be a developer. That means there are dozens or hundreds of candidates for every vacancy, and the application and interview process are gruelling—in many cases, brutal. That's the bad news.

The good news is that most of these candidates are of very low quality. Not that the *people* are low-quality, of course. It's just that they probably don't have anything like the level of experience or skill that's appropriate for the job. If *you* do, though, you won't have as much realistic competition as you might think.

We'll talk more about the interview process and how to survive it later on, but before you can get the interview, you've got to look for the *vacancy*. How do you do that?

Personal recommendation

Actually, I think the most efficient way to find a job is *before* there's a vacancy for it, for the reasons we've discussed. How is that possible? By knowing people.

If opportunity doesn't knock, build a door.

—Saying

A lot has to happen before a company posts a job advertisement online, and many vacancies are filled before they're even advertised. Why? Because someone knows someone. As a result, the company can skip most of the hiring process.

The power of personal recommendation is such that some cannier companies even pay a “finder's fee” to employees nominating someone who subsequently gets hired. They know this is far cheaper than going through the slow, expensive interview mill, and usually has better results.

After all, would *you* recommend someone you knew was no good? Of course not: it would reflect badly on you. Your boss's faith in your judgement would take a dive. Maybe they'd even start to wonder if *you* should be working here, never mind your friends.

So the first step to finding the perfect job is *networking*: knowing someone that knows someone.

Making connections

First, if you're already in a job, you know all your co-workers. Or, if you don't, it's time you did. Start there. Your colleagues are ready-made contacts: they work in the same industry as you, they probably have some skills and experience in common with you, and therefore they can recommend you to other employers. Whether they *will* recommend you or not is the question.

When you leave a company, don't lose the contacts you made there. LinkedIn is the obvious way to keep in touch with former colleagues, but everybody hates LinkedIn, and it's not the best way to cultivate meaningful human relationships. Instead, do it the human way: by emailing people, texting them, or even arranging to meet up in person once in a while.

Of course, you probably won't be re-applying for jobs at places where you've already worked before. But, just as you've moved on to other things, so will many of your colleagues. In a few months or years, people in your network will be working in places where *you'd* like to work, and that means they'll be in a position to help you—if they want to.

Networking groups

Expand your circle of connections beyond the people you already know. One way to do this is to go to local tech meetups and user group meetings. Wherever you live in the world, there's probably some kind of local interest group for your favourite programming language, operating system, or other technology. Sign up and go along.

If you're not a natural mingler, I feel for you: walking into a room full of strangers can be a scary prospect. But keep in mind most of them are also strangers to each *other*, and what's more they're all here specifically to meet new people. You may not be good at that, but neither are they, so in a sense you're all learning a new skill together.

Think of social skills as a new programming language. There are a few core concepts to this language that may be new to you, but luckily, the syntax is actually way simpler than you might realize. And sometimes the people who feel the least competent turn out to be the best at it.

—Sam Julien, “[The Painfully Shy Developer’s Guide to Networking for a Better Job \(Without Being Creepy\)](#)”

A common format for meetups is to have two or three brief talks, perhaps with a Q&A session, and then to socialise with food and drink. People will tend to form into little clusters; sidle over to the nearest one and look interested in the conversation. If you can join in without being too abrupt, do so. Or if there’s a natural gap in the chat, throw in a question: “So, how did you like that talk about X?”

If you’re *not* naturally shy, beware of dominating the conversation too much. You don’t want to be *that guy* who drones on about his favourite thing, never letting anyone else get a word in. Instead, ask your question or make your point, then keep quiet for a few minutes, or until at least two other people have spoken.

Be interesting, and interested

Don’t just chat, eat a couple of slices of pizza, and then go home: use this opportunity to find and make contact with interesting people. If you find this hard work, it may help to gamify the process. For example, set yourself the goal of exchanging contact details with three different people by the end of the evening.

A natural way to do this is to say “I was really interested in what you had to say about X—would you mind if I sent you a couple of follow-up questions by email?” Or you could say “Your project sounds terrific—I’d love to find out more and maybe get involved. Can I get in touch?”

Don’t pitch your product or company; that’s not what this is about. You’re here to get contacts, not customers. And don’t come across as too obviously goal-directed: no one likes to feel they’re being harvested for their potential connections.

Instead, aim just to be interesting, and interested. At this stage, all you need to do is plant the tiniest seed of a potential future relationship: who knows where it might go in the years to come?

Be a giver, not a taker. Look for people who could use your help, and offer it. Or find people who want to talk about something, and let them talk to you about it. If your company is hiring, mention that, and if you meet someone who’s looking for a job, offer to put them in touch with the right hiring manager. After all, just because *you* no longer want to work there, it doesn’t mean someone else doesn’t. You might even end up finding your own replacement.

Full-scale conferences can also be a useful way to make industry connections. Since they tend to be expensive to attend, this is only worth while if your employer is willing to pay for your ticket. If they are, make the most of the opportunity, and swap details with as many people as you can.

Don't forget to follow up on these new contacts, either. Set yourself reminders to follow up with each person within a month or so: a brief email to say hi and remind them who you are and where they met you will suffice.

Often, people won't respond, but that's okay. You've done the hard part, which is making the initial connection. It's fine to reach out again a few months later, and maybe you'll have bumped into them again by that time anyway. Not all of your connections will turn into fruitful, long-lasting relationships, but it's a numbers game. The more connections you make, the better the chances that something will eventually happen.

Becoming a brand

So networking is one way to get a job without going through the formal application process, but an even better one is to *have the employer come to you*. Why would they do that, you ask?

The answer is because of your incredible personal brand. We saw in the previous chapter that one of your possible career destinations is *independence*: being the head of your own firm. That's great, but most of us have to pass through a shorter or longer stage of working for someone else first, to build up our experience and skill set.

But in a broader sense, you're *always* the head of your own one-person firm. The business of that firm is the business of getting hired for the job you want. Your product is yourself, if you like. To sell that product, just like any product, requires *marketing*. What would that mean in this context?

Being a big name

Whatever technical field you're interested in, whether it be software development in Go or some other language, infrastructure engineering, and so on, there are people whose names are known. If I asked you to list four or five people who you associate with expertise in your field, you probably wouldn't have to think too hard.

It's worth taking a moment to ask yourself *how* you know these people's names. Perhaps you read one of their blog posts or books, or saw a conference talk they delivered. Maybe they're active on a mailing list or social media group you follow, or maybe they're an important contributor to one or more key software projects in the space. At any rate, they have a high profile, and their names are well known.

Now imagine you want to hire an engineer in this field: someone of proven experience and value who would be an immediate boost to your team's skills. Who would you reach out to? One or all of these big names, right? It does no harm to drop them an email and ask if they're currently looking for work.

Recruitment agents do the same thing, if they're smart. They almost invariably work on commission, so it's in their interest to find people they can successfully place in well-paid jobs without too much difficulty. Where would they start looking for people like that? The same place you would, naturally: online.

How to be internet famous

Now, you might not think of yourself as being in the same league as these luminaries, but the point is, *neither did they* when they started out. At some time in the past, they were a total beginner, just like you were; just like we all were. And they may, in fact, not even be the most knowledgeable or expert people in your field.

Undoubtedly they have *some* expertise, but what makes them a big name is that they also have a great *brand*. And that doesn't happen by accident.

It's tempting to think that if you just know a lot, you'll automatically become internet famous, but it doesn't work that way. You have to put yourself out there and *build* a brand. How do you do that? Just the ways we've talked about: by blogging, writing, giving talks, posting on social media, contributing to projects, and so on.

Most people, being commendably modest, resist this idea. "How can I possibly write a blog post about X until I'm the world's expert on X?" they ask. That's missing the point. Whatever you know about X, you know *something* that other people don't. You can write about it.

If you genuinely don't know anything at *all* about X, learn something. Then write about what you learned.

Whatever your thing is, make the thing you wish you had found when you were learning. Document what you did and the problems you solved. Organize what you know and then open source your knowledge.

—Shawn Wang, "[The Coding Career Handbook](#)"

After all, that's how the people whose names *you* know got started, probably. And there must be lots of other people who are tremendous experts on something or other, but no one's heard of them, because they don't *write* about it. You can, and you should.

We'll have more to say about *impostor syndrome*—the mistaken belief that you're the only one here who doesn't really know what they're doing—in a later chapter. For now, just adjust your expectations about what kind of person writes technical blog posts, or even books. It usually *isn't* the hugely experienced expert, but the intelligent and curious person who only just recently learned about the thing.

After all, that's the best qualification for explaining an idea to someone who doesn't know anything about it: you *were* that person very recently! Being a good explainer is not so much knowing the technical details, as being able to identify with the person who *doesn't* yet know them.

You can predict what questions they'll have, or the concepts they might stumble over, and anticipate them. Remembering what it's like not to know something gives you the ability to empathise with those who don't. You can structure your explanations so as to meet them where they are.

You need a website

Writing is also a great way for *you* to understand and retain what you’ve learned. But being able to communicate technical matters in writing, in a clear and readable way, is a key skill whatever your subject. It’ll help you build your brand to get you hired. Once you’re hired, writing skills will help you succeed in the job.

Just as no one was born a tech expert, no one was born a writer, either. It’s a craft you need to study and practice, and the more you do it, the better you’ll get. The easiest way to get started writing is to have a blog. While there are sites such as Medium that you can contribute blog posts to, I recommend you set up your *own* site instead.

After all, your website is the home of your brand. It’s your store display, your point of contact, and a potential source of work leads. You need to control it, otherwise your hard work is just driving eyeballs, clicks, and Google ranking to someone *else’s* online property.

Don’t think that posting on Twitter or Instagram or Facebook or LinkedIn is the same as having your own site. You’re just helping to build *their* brand, rather than your own.

Instead, get your own domain and site hosting, and set up your own site. You needn’t build a ton of infrastructure, though you can if you find that kind of thing enjoyable. There are plenty of sites where you can just select a blog template and get going.

Search engines favour well-established sites. It might take six months or a year for your articles to start showing up prominently in search results, which means you should set your site up well *before* you start actively looking for work.

The day you get laid off is a little too late to start panic-blogging (I don’t want to worry you, but such things do happen). It might be wiser to get a head start.

This is your time to suck

Once you have the site set up, start writing. Set yourself a schedule of regular posts: one a month, perhaps, or even more. They don’t have to be long: one to two thousand words is plenty. Keep a list of topics that interest you, or that you’d like to write about. If there’s something you’d like to *read* about, but you can’t find any good articles on it, write one.

This is your time to suck. When you have no following and no personal brand, you also have no expectations weighing you down. You can experiment with different formats, different domains. You can take your time to get good. Build the habit. Build your platform. Get comfortable with your writing/content creation process.

—Shawn Wang, “[The Coding Career Handbook](#)”

Keywords matter. Most of your traffic (hence most of your employment leads) will come through organic search: people looking for *information*, not for you specifically. If you find yourself searching for something online, write an article that would make a

good search result for those keywords. Then, when people search for that phrase in the future, they might land on your site.

Think about how much you use web search every day as an engineer. Provide the resources that other engineers need, targeting the keywords they're searching for. After a while, if you keep creating good and useful content, people will start to know who you are.

Building a portfolio of projects

It's always a good idea to have at least one or two really well-finished and polished software projects in your portfolio. Not only can you show these to interviewers and highlight them in your resume, but working on these projects will give you valuable practice for coding challenges and take-home tests.

If you're trying to get someone to hire you as a software developer, probably the most important thing you can show them is the software you've developed:

The first thing I look for in a resume is a GitHub URL. If I can find that, I stop reading the resume and start reading code. Doing open source is not a be-all and end-all by any means, but it's a good starting point.

After five minutes of reading through a candidate's code I have a good handle of their skill level and style, making the interview run much more smoothly.

— Alex McCaw, “[Resumes Are Dangerous](#)”

Some people neglect this at their peril. If you have reams of wonderful code that you've written, but it's all closed-source, proprietary, and confidential, that's too bad. You won't be able to get credit for it with potential employers. Make sure that you have at least a few high-quality programs you can actually *show*.

If you don't have a GitHub account with some finished, polished open source projects to show interviewers, start one. Build some projects. It's better to have one really good project than two or three average ones, but the more code you can show off, the better. This will take time, so don't wait until you're actually looking for jobs to start.

When to start looking for new jobs

The best time to look for a job, of course, is when you already have one. If you're unhappy at work, don't quit: instead, look for something better. If you're *happy* at work, ask yourself: will I be here forever? If not, it's worth thinking about where you'll go *next*.

The key question to ask, and I recommend you think about it once a year or so, is this: “Am I where I want to be, doing what I want to do?”

If the answer is no, then it's time to look for a new job.

Keep cashing the checks

If you hate your job with a passion, and you'd rather die than have another 8am Teams meeting, I sympathise. But still, don't quit—at least, not yet. Instead, look for a new job while you still have your old one. It might take a little longer than you expect, and in the meantime the paychecks will still be coming in, so you won't feel pressured by money worries into taking any old job that comes along.

And when things look bleak, it's easy to feel that any job would be better than this. But most likely it wouldn't. Be selective. Don't look for an escape capsule, look for a shuttle to the next stage of your career.

In other words, never go *from* a job, always go *to* a job. Think about how the change will look on your resume. Is this next job a natural, logical step towards the career you want? If not, what kind of job would be?

Leveraging an offer

Even if you don't plan to leave your current position any time soon, it's still worth knowing your options. Suppose you get an offer of a more senior job at another company, with better pay and more interesting work. That's a lovely thing to have in your back pocket, even if you don't plan to act on it.

You can use a job offer as leverage to improve your *current* job. If you've been angling for a promotion or a raise, a competitor's job offer is a powerful argument in your case. It's solid evidence of your value to the industry at large—if you use it the right way.

The *wrong* way to leverage a job offer would be to use it as a threat: “If you don't make me the team lead, I'll quit and go to work for Acme instead.” This is a mistake, especially if you don't really *want* to work for Acme. There's a danger that your boss might call your bluff.

Instead, you can frame the conversation as asking for advice. “I really enjoy working here, and I'd love to see my career develop further in this company, but things aren't moving as quickly as I'd hoped. Can you help me understand what I need to do to get the raise / promotion / extra responsibility I'm looking for? I'm getting attractive offers from other firms [*brandish offer letter*] but I'd prefer to see my long-term future here—if you can help me figure out how to make that happen.”

It may well be that this is just the evidence your boss has been looking for to advance their efforts to promote you within the company. After all, there has to be some actual reason to promote you: an offer letter could help show that you've become a more valuable employee.

On the other hand, your boss might say something like “I'd love to promote you, but there just isn't the budget this year. Maybe next year, or the year after.” Or even: “Just between us, I think you'd be well advised to take this offer. If anything, we're looking at ways of *reducing* headcount right now.”

Either way, a conversation like this with your boss will help you make up your mind

about what to do. And it's a no-lose situation for you. If you can parlay a job offer into a better position at your current firm, you win; if you can't, you can *take* the offer. It's all good. But none of this happens until you *have* the offer.

Don't be a dinosaur

Changing jobs is always a risk: it might not be as rewarding as you thought, or the company might not be as nice to work for as it looks. But there's also a potential risk in staying with one employer for too long.

The tech industry as a whole moves fast, but individual companies tend to be fairly conservative. Once they've found a set of technologies or workflows that brings success, they tend to stick with it even when that success starts to decline. This technological backwater can trap a lot of good engineers.

It's not always easy to spot when you're in the expertise trap. The company may be doing great business, dominating the market, making lots of revenue, and you personally may be pretty well paid. But the seeds of failure may already be sprouting. If you're not up to date on industry trends, you may not realise that the company's key tech is no longer the hottest thing. It could even be on the way out, and your career plans could go with it.

The biggest danger for you personally is becoming the company's senior expert on the dinosaur technology. You'll be praised, rewarded, promoted, and flattered by everyone, and you'll be feeling great. And then one day, probably without warning, your services will no longer be required.

All of a sudden, you'll find yourself in a job market where your much-vaunted skills are of no value. No one wants to hire experts on yesterday's tech. And kids fresh out of school will be beating you to jobs using the hot *new* tech, because they have experience with it and you don't.

At worst, you may find that it's now simply too late for you to catch up. If you have no proven experience with the new Tech X, you won't have much chance of landing a job where you'll *use* Tech X. And if you can't get a job using Tech X, you won't be able to get that experience. You're stuck.

This is something to be wary of, and while expertise is valuable, you should be careful to avoid becoming *too* much of a specialist in any one technology.

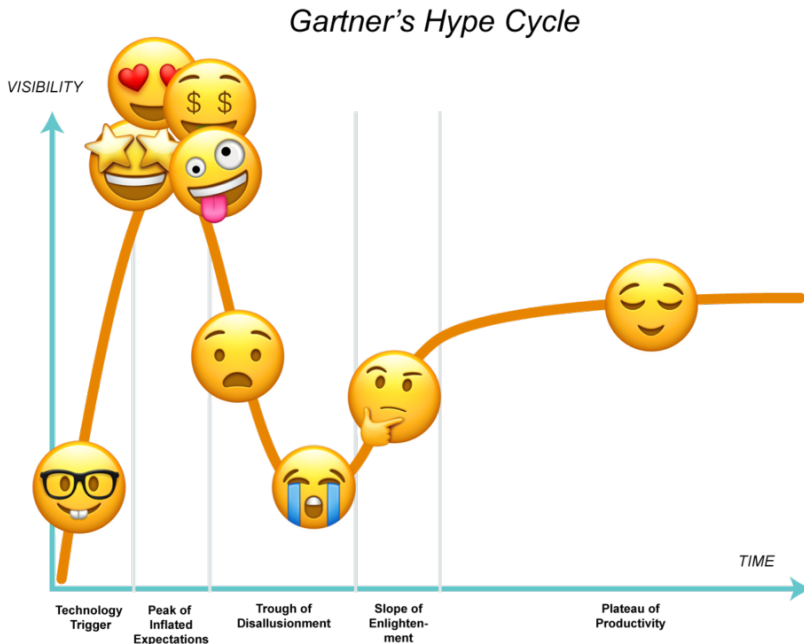
Specialists often have leverage when it comes to negotiating job offers and salary increases because their deep knowledge in a field may be considered a competitive advantage in certain industries.

But the market for those jobs is smaller overall. This presents an interesting problem: What if you specialize in the wrong thing?

—David Glassanos, “Junior to Senior”

Avoiding the expertise trap

When you're new to the industry, it's natural to think that whatever's hot *now* will be hot forever. It doesn't work that way. Instead, technologies tend to follow the well-known *hype cycle*:



And, since companies tend to stick with the tech that's worked for them in the past, it's easy to find yourself trapped in one of the last companies that still uses the old tech, with no career path forward. That trap is baited with lots of money and status, but there's no way out of it. Beware.

If you find the expertise trap closing on you, it's time to plan your escape route. Other jobs with other technologies may look less attractive, and probably less well paid, but that's how the trap *works*. It never seems to make sense to take a pay cut when changing jobs, but you need to be able to see the bigger picture.

If you focus only on the immediate tactical situation, you can make what seem to be sensible decisions at the time, but still wind up stuck in a dead end. Instead, think strategically. Consider jobs that may be outside your technical comfort zone. The broader the range of technologies you get to work with, the healthier your long-term career prospects.

If you find a job that would bring your tech knowledge up to date with the market, but that would mean you taking a pay cut, consider it as a potential investment. Alternatively, consider it as the premium you're paying to insure yourself against obsolescence. Less money now could mean more money—and more options—later.

Where to look for jobs

Online job sites make it easy for you to find and apply for lots of jobs, which is great. The trouble is, they also make it easy for other people to do the same. To tilt the odds in your favour, then, you might like to try applying for jobs through channels where there isn't so much competition.

Apply to employers, not jobs

Just because a company isn't currently advertising a vacancy doesn't mean they're not interested in great candidates. *Everyone* is interested in great candidates. If necessary, a job can be created for them. So try reaching out directly to companies you think would be neat places to work.

If possible, get an introduction to the appropriate hiring manager or engineering lead from one of your contacts. Email them and say "I hear that Initech is a really good place to work. I'm considering a job change soon, and I wondered if you could spare a few minutes to chat about any opportunities you might have coming up?"

If you can't get an introduction through a mutual contact, reach out anyway. Don't cold-email the company's general enquiries address. Instead, use a little detective work (or social engineering) to get the name and email address of the relevant hiring manager, and go straight to them. The bigger the company, the more gatekeepers there will be to ignore or delete your email. Bypass them and get your resume straight to the decision maker.

In bigger companies, you should try to find the right *team* to apply to. Even within a single company or department there may be many different teams with different work cultures and management styles. One of them may be a great fit for you. Having a friend on the inside will help you focus your efforts in the right spot.

Recruiters

Recruiters mostly work on contingency, meaning that if they successfully place you in a job, they will get a fee equivalent to perhaps 20% of your first year's salary.

In one way that's good, because it means the recruiter is strongly incentivised to help you *land* the job: that's how they get paid. And the bigger your salary, the bigger their commission.

The downside for you in this business model is that there's no downside for *them* if they put you forward for literally every job going, suitable or not. They're not in business

to find you a job you'll love, or to help companies find the perfect hire. Their payday comes from you signing a contract—any contract—as quickly as possible.

Choosing jobs to apply for

The most important thing to ask about a given job vacancy is “If I get this job, how will that help me advance towards my eventual career goal?” For example, if your ultimate destination is independence, you need to acquire a broad range of skills and experience across the industry.

So when you're applying for jobs, look for one that will fill in the gaps. If you've only worked in big companies, for example, then try to get a job in a small company or even a startup. This will put you much closer to the front line in terms of business, marketing, contact with customers, product management, and so on.

In larger companies such roles tend to be strongly demarcated, and you may never even visit the buildings where these activities go on, much less get to participate in them.

On the other hand, if your eventual goal is management, big companies are the place to look. They have more layers of management, and more people to manage, thus offering you more opportunities to learn the craft.

Apply to jobs you're unqualified for

There's an understandable tendency to look for jobs that your current capabilities would fit perfectly, but that's not ambitious enough if you want to take firm control of your career. Instead, you should really only be looking for jobs that you're *unqualified* for. If you don't get them, well, you've lost nothing. If you *do* get them, great! You've levelled up without all the tedious grinding.

You might be doubtful that it's worth your time applying for jobs you *know* you can't do yet. But you'd be surprised. Since employers are really looking for *people*, not skills, then if you're the right kind of person you may be able to walk into a job that's a little above your experience level and learn it from the inside. And if you interview but get rejected, then you'll get some valuable intelligence about what you still need to learn.

The experience paradox

Even though the best job for you is one where you can learn new skills, that's not what *companies* want. Instead, they want someone who can already do the job. When you hire a contractor to tidy up your garden, you don't expect to have to show them how to use a hedge trimmer.

But this poses a problem if you're trying to break into a sector or industry where you don't already have a lot of experience. How can you *get* that experience if no one will hire you without it?

The resolution to this paradox is for you to acquire that experience *yourself*, on your own account, without waiting for someone to hire you specifically to gain it. If you're currently in a job that's even vaguely related to the field you want to work in, leverage that for all it's worth.

Learn all you can on your current employer's time. Good employers will have a training budget; if so, use it. If not, spend your own money. It's the best investment you'll ever make: in yourself.

Buy books and other resources. Hire an expert coach or mentor to teach you what you need to know. Whatever it costs, it's money well spent if it lets you break through the experience barrier to the next stage of your career.

Mastering the application

There are always many more applicants than there are vacancies for a given job, so no company could interview them all. Even a simple pre-interview screening can hardly take less than 15-20 minutes, and a full, in-person interview could be more like half a day.

So the first thing a hiring manager needs to do is *filter* the applications, producing a shortlist of people who seem like good prospects for interview.

This means reading resumes—a lot of resumes—and it's a good bet that they won't be reading them in great depth. Rather, they'll be *reading to reject*. Poor presentation is probably the quickest way your resume will get rejected. To avoid this, use a resume template or copy an attractive-looking online sample. Visual elegance alone won't get you the job, but it will at least get you through the first set of filters.

Knowing the keywords you need

Another important thing the hiring manager may be looking for is a bunch of *keywords*. What these are depends on the specific job vacancy. For example, if the job title is “Golang backend developer”, then they'll be looking for an early mention of “Golang” somewhere in your resume. Ideally, in the first paragraph. The more of the document they have to read to find the keyword, the less likely you are to make it through this filter.

Words matter, and if you've written about your experience with “Go”, the reader may not realise that that's the same thing as “Golang”. Remember, they probably won't be technical themselves. It's likely this initial stage will be done by someone doesn't understand the *meaning* of any of these keywords, but who is just doing the equivalent of a text search.

Tailoring the application

It's also important to tailor your application to the specific job and company in question. This needn't involve hours of work, but you should at least do a little research on the company, so you know at least something useful about them, and then *demonstrate* that knowledge in your application.

We look closely at the cover letter for evidence that the applicant really wants to work for us. We don't want to see a generic cover letter talking about me, me, me: we want to see a coherent argument as to why they've thought about this seriously and concluded that Fog Creek is the place they want to work.

If I only have time to interview six people, all else being equal, I'd rather interview six people who really want to work for Fog Creek, not generic smart people that are also applying to a hundred other jobs.

—Joel Spolsky, “Sorting Resumes”

A little effort here can go a long way to boosting your chances, especially since few other candidates will take the time to do it. If you can talk to someone who actually works at the company, perhaps with the help of your network of contacts, that's a great way to get this kind of information.

What not to say

What *shouldn't* you put in your resume? Too much detail about jobs you did years ago. If they're relevant to your current application, say why, but otherwise summarise all your non-relevant experience very briefly (a job title and perhaps a sentence or two describing it).

Omit all irrelevant and non-notable information. For example, if you're the chair of your local dog-walkers association, that's wonderful, but employers don't need to know about it. If you enjoy cooking and reading detective stories, I salute you, but don't bother saying this in a job application. If you have a *notable* hobby, such as naked mountaineering, use your best judgement about whether it's appropriate or helpful to mention it in your resume.

If you started a degree course but failed to complete it, then you're better off not mentioning it at all. You might feel that you should get at least partial credit for *starting* the course, and I agree with you, but employers are more likely to take a negative view of the fact that you didn't finish it. Mention only things that show you in a positive light and that you'd be pleased to talk about in interviews.

Because your resume will likely be read in a terrific hurry, and with more of a view-ing to *rejecting* you than selecting you, it's important to keep it short, punchy, and focused. Think of a stand-up comic delivering a “tight five”: it needs to be all A-material, no filler.

References

You don't have to provide references as part of your resume, but it's a good idea to make a list of suitable people in advance. Employers may contact your references at any point during the hiring process, often as the last step before making you a formal offer. So whom should you list as a reference?

Obviously, it should be someone who's likely to give a good report of you. If that's true of your current or previous manager, great (ask their permission first, but it's unlikely they'll refuse it if you ask nicely).

But it may be that you'd rather not have your boss contacted by potential future employers just now, for perfectly creditable reasons. In that case, you can cite other managers from previous jobs, co-workers, or people who otherwise have an informed professional opinion of you.

If you have a teacher, coach, or mentor, don't neglect them as a reference. They will almost certainly be delighted to provide confirmation of your skills and general employability. If they agree to be a reference, let them know when they can expect to be contacted, and by whom.

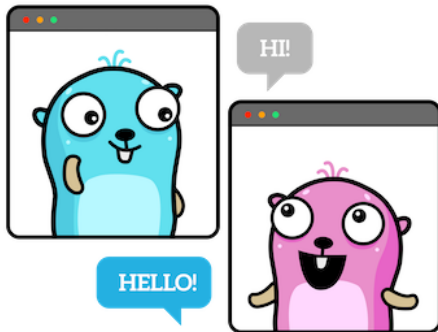
When you provide contact information for references, let the employer know what the person's connection with you is—for example, that they were your direct manager at Company X, or that they're your programming tutor. If possible, list two or three different people; each good reference adds weight to your application.

A great resume, well targeted at the specific job and company, with solid references, has a good chance of getting you an interview, or at least a pre-interview phone screen. How should you prepare, and what kind of challenges will you face in the interview? That's the topic of our next chapter.

3. Interviews

Smart people can generally tell if they're talking to other smart people by having a conversation with them on a difficult or highly technical subject, and the interview question is really just a pretext to have a conversation on a difficult subject so that the interviewer's judgment can form an opinion on whether this is a smart person or not.

—Joel Spolsky, “The Phone Screen”



The first thing to say about interviews is that you’ll have a lot of them over the course of your career. Most of them—the vast majority—will result in you not getting the job. I say this not to depress you—it’s true for everybody—but to prepare you.

An interview is not just some brief formality that you occasionally need to go through to keep your career on track. It’s one of the most important conversations you’ll ever have, and investing a little time in interview prep and skills could mean the difference between success and failure.

Preparation

Most of us get very nervous about interviews. It’s stressful preparing and waiting for the interview, it’s stressful *during* the interview, and it’s also extremely stressful waiting to hear the result. When that result is a “no”, as it usually is, the inevitable consequence is even more stress.

Plan for the long haul

You need to be realistic about the pressure that interviews can put on you, and the resultant toll they can take on your physical and mental health. Putting yourself through too many interviews in a short time can lead to burnout.

As we'll see shortly, tech interviews often involve tasks for you to do at home, and these can be just as demanding as formal interviews, if not more so. If you can only work on them evenings and weekends, it can be difficult to balance this with your personal or family life. Taking on too many such tasks in a short period is unsustainable.

Instead, treat the job search as a marathon, not a sprint. Assume that you will be searching, applying, and interviewing over a period of, let's say, three to six months. Of course, it may not be that long: you might get the first job you go for. But rather than rely on that, you should *plan* for the long haul.

Interviews are more demanding than you think

Be prepared for each interview to demand more of your time and energy than you might have expected. You probably need a day or two to prepare yourself adequately for a serious interview, the whole day for the interview itself, and at least two or three days to rest and wind down afterwards.

That suggests a working maximum of about one interview per week, and one every two weeks is probably better. It's no use showing up frazzled, distracted, and disengaged. And if you're interviewing over a period of months, your performance in those interviews is likely to get worse rather than better unless you manage the scheduling well.

Keep the job listing for reference

Whenever you get invited to an interview, it's a good idea to save a copy of the original job advert or listing. It will help you to refer to this in the interview, and should you end up getting the job, it will be useful to compare the reality against what was originally advertised.

I like to save the job listing as a PDF in case it gets taken down during the interview process. It can also be immensely valuable after you get hired, as a measure of what you were originally asked to do when you joined the company.

So if you find that 18 months down the line your tasks and responsibilities have grown, but your title or salary hasn't, you can have an informed conversation with your manager and ask for your compensation to be adjusted to match what you actually do.

—Carolina Gomez Gilabert, [“Looking for a Job While Looking After Yourself”](#)

Strike while the job is hot

When companies hire, they want to hire quickly. Let's suppose they arrange to interview a dozen people, but it so happens that the second or third person they see is a perfect fit for the role.

Will they ask the candidate to hold on for a few weeks, while they schedule interviews with the remaining ten people, just in case there's someone even better among them? Maybe, but it's much more likely that they'll simply move ahead with the candidate they have, and skip interviewing the others.

That means you want to be among the first people interviewed, if not the very first. Make sure that when you're offered an interview you schedule it as soon as possible. Even the same day, if that can be arranged; there might not *be* any more interviews tomorrow.

The initial phone screen

Depending on the kind of job you're going for, the interview process may involve many stages, but the first is likely to be a phone screen or a video call of some kind. How should you prepare for something like this?

Be on time and ready

First, make sure you know the exact day and time for the call; double-check this if necessary, and set whatever kind of reminder or alarm you need so that you absolutely will *not* miss the appointment. If you have trouble remembering appointments or being on time, get a friend to be your wing-person, so that they can remind you on the day.

A day or so beforehand, check your computer is working and up to date, there are no problems with your internet connection, and that you have installed whatever conferencing or chat software the interviewer will be using. If there's a test call facility, use it, or have a trial run with a friend, just to make sure there aren't any weird issues with your sound or webcam.

For example, some USB devices can cause electrical noise that is inaudible to you, but very audible to the person you're talking to. Make a recording of yourself talking on the microphone you'll be using, and check that it sounds okay.

Double-check your setup

Check that you know how to mute yourself if you need to cough, sneeze, or drink water. Put your phone on silent and turn off all other possible distractions. Interruptions won't help your focus.

Don't bother with things like last-minute revision or technical study; if you don't know the tech by now, you won't learn it half an hour before the interview. Instead, relax as

much as possible. Go for a walk, play with the dog, or read a cheerful book. By the time of the call, you should aim to be rested, refreshed, and energised.

The conversational interview

Try to sound relaxed and friendly, even if you don't feel that way, but be polite, not casual. Treat this conversation as though you were meeting your future boss for the first time, because you might be.

Talking about yourself

If the interviewer is any good, they will know how to ask you *open questions* that give you an opportunity to talk. If the questions are closed, though (“So how long have you worked at XCorp?”) don't just answer them in a few words and then stop.

Instead, use their questions as a jumping-off point: answer them, and then segue neatly into something you want to talk about. “I've been at XCorp for about nine months now—I started out in the mobile division, but pretty quickly moved over to the enterprise software side. Right now I'm working on an interesting project that...”

On the other hand, be careful not to hog the conversation. When you've been speaking for four or five sentences in a row, you've probably said enough for a while, and it's time to throw the ball back to the interviewer.

The note to strike is friendly, yet professional. Imagine yourself already *in* the job and discussing the details of some task with a manager or colleague. One of the many things the interviewer will be trying to decide about you is whether you'd *make* a good colleague.

The interviewer's body language will tell you a lot about how you're doing, so pay attention to it. If you can make the interviewer smile, you're probably 75% of the way to getting the job, because it means you've connected with them as a human being, not just a software-producing robot.

As a candidate, you should treat interviews as an exercise in communication. Do not just focus on algorithms, data structures, and architectural designs. Use your communication skills to ensure the interviewer would love to work with you after this interview.

—Chris Laffra, “[Interviewing](#)”

Telling your success stories

When you're asked to talk about your previous experience, it's a good idea to have one or two specific stories prepared (but, again, not over-prepared).

Ideally, the story will be one where you come out well, but not in a self-serving way. For example, if your team solved a critical engineering problem under pressure, emphasise

the “team” aspect of it. Don’t say “And then I figured out how to...” Say “We figured out that...”

Behind the boring list of past employers on the resume, there’s always a story. What I’m looking for is the story of how the candidate handled challenges in the past. I’m looking for people that got things done, even in the face of opposition. I’m looking for people who challenged the status quo, who overcame objections, and who made things happen. So when the resume says, “drove the adoption of .NET,” I want to hear what “drove” means. In detail.

—Joel Spolsky, “The Phone Screen”

You don’t need to wait to be asked to tell an interesting story about your career or experience. Just look for a suitable lead-in and tell it. The interviewer *wants* to hear as much as possible about the way you work and the kind of things you’ve done, but they may struggle to find the right questions to get you to open up. Don’t make them struggle: be ready to talk, and prepare something interesting to talk about.

Some examples of the kind of stories you might be asked for:

- “Tell me about a recent project that was very challenging, Explain the challenge and tell me how you went about solving it.”
- “Tell me about a project where you had to work with cross-functional teams and that caused problems. What was the problem? How did you solve it?”
- “Tell me about a situation where you had a conflict with a more senior or a more junior member of your team. What was it about? How did you successfully resolve it?”
- “Tell me about a project where you weren’t able to meet your commitments for some reason. Why not? What was the impact of that? What did you do?”
- “Tell me about a time when you discovered a problem with your design in the middle of a project. What happened? What did you learn?”

And, assuming you’ve prepared a few stories along these lines, you can now launch into the appropriate one for the question.

Stick to the point

The secret to telling a good story, in an interview or anywhere else, is to keep it short and punchy. Don’t get distracted or dwell on irrelevant details. It shouldn’t take you more than a minute or two to tell the whole story. If the interviewer wants to know more, they can ask follow-up questions.

Avoid the temptation to be critical, either of your former colleagues, or the company as a whole. Don’t say “The idiot engineering manager changed her mind at the last minute and put the whole project in jeopardy.” This may be the simplest and most accurate description of what happened, but it doesn’t reflect well on you, especially if you’re *talking* to an engineering manager. Instead, be diplomatic. This is a job skill, too.

“There was a last-minute change of requirements that led to some severe schedule pressure. So we decided to...” expresses the same context without judging or criticising individuals. After all, your story should focus on what *you* did, not on other people’s mistakes.

The tricky questions

Having thrown you a softball or two by inviting you to talk about your work achievements, the interviewer may now move on to a few more challenging, but still predictable questions.

“Why do you want to change jobs?”

The correct answer here is to say that, while you’ve enjoyed the work and learned a lot, you now feel ready to take the next step and look for new opportunities.

You’ve discovered in the course of your work that you have an aptitude for (insert skill here) that you’d like to develop further, and that while you’ve explored options within your current company, you think the most exciting opportunity is with (insert company you’re talking to).

The idea to leave your interviewer with is that you’re the kind of employee who values development and growth, and you’re simply growing a little faster than your current position can accommodate. The fact that you’ve come to *them* as a result is a compliment: it suggests you think they’re the kind of go-ahead company that’s just a little bit smarter than its competitors.

An interesting question to ask in response, in a gentle way, is “Why is this position open?” Is the team expanding to handle new business? Did the last person quit? Why? Have they been trying to fill the post for a while without success? Why? Answers to these questions can be informative, but a *lack* of convincing answers can also be informative in a different way.

“What’s your greatest weakness?”

What this question is really probing for is your level of self-awareness. Everyone has weaknesses, so are you aware of yours? And, more importantly, what are you doing to improve them?

Whatever your *actual* weaknesses, avoid citing something that the job description requires as a strength. For example, if the job is as a database developer, don’t say that your greatest weakness is that you know nothing about databases.

Instead, pick something that you know you could improve, but that isn’t necessarily central to this job. In your answer, rather than focusing on the weakness itself, focus on what you’re *doing* about it.

For example, you might say something like “One thing I’d like to improve is my knowledge of system design, because I haven’t had much direct experience of it in my previous role. I’m studying this from textbooks in my spare time and working on a side project where I can practice this skill.”

“What salary are you looking for?”

Don’t answer this question now. That can wait until they’ve offered you the job. In most jurisdictions companies aren’t allowed to ask you what your current salary is. All the same, they want to know how cheaply they might be able to get you. Politely decline to mention any numbers.

Every employer will ask about expected salary and every experienced professional knows to not answer. This is a standard part of the hiring dance, don’t be afraid to stand your ground.

—“Salary Negotiation Strategies Everyone in Tech Already Knows — But You Don’t”

Turning the salary question around

Ideally, you won’t even *be* here in the interview unless you already know that the salary will be in the right range. But if you’ve had no luck getting answers up to this point, ask again. Don’t be deflected by an answer like “it depends on the candidate”. Ask “Well, then, take me for example. What would be the lowest salary you’d offer someone of my experience?”

If you’re in the *interview* and you can’t get a clear idea of the compensation for this job despite repeated enquiries, you should be concerned. Either they genuinely don’t know what they should be offering you, which is a red flag in itself, or they know it’s well below market rate, and they don’t want *you* to know that.

The technical interview

You’ll often face a technical test or quiz, especially in early-stage interviews. How you approach this will depend on whether you’re talking to someone who’s technical themselves, or not.

Non-technical interviewers

Especially at an early stage of the interview process, you might be talking to an HR employee or a hiring manager who doesn’t have a technical background at all, but who has been given a sheet of technical questions to ask you (and, hopefully, the expected answers).

This kind of situation can be difficult for both of you. It’s easy to feel a little insulted if you’re a highly-experienced engineer and they’re asking you about things that a begin-

ner ought to know. Don't take offence. It's not that they doubt your experience, just that if a non-technical person is to administer a technical quiz, the questions themselves must necessarily be fairly basic. Otherwise, they probably wouldn't be able to understand your answers.

Don't be tempted to be too clever, by giving a detailed and unusual answer that's *correct*, at least to real experts, but doesn't chime with what the average person would expect to hear. Your interviewer may be *very* average, so keep your answers simple and straightforward. Save the deep stuff for when you're talking to fellow engineers.

The dreaded whiteboard task

Coding on a whiteboard, whether real or virtual, can be a scary prospect. It's difficult to think clearly and accurately when you're under pressure. A good interviewer will do their best to put you at ease, and to start you off with one or two simple tasks to build your confidence.

On the other hand, they may give you a really hard problem that they don't even expect you to be able to solve. The idea here is to see whether you're able to think out loud, discuss ideas, or ask for help, and to assess your strategy for tackling difficult problems.

It's not about your program

The key to doing well in whiteboard challenges is to remember that it's not really about the program, it's about the *programming*. The interviewer wants to see your process, how you work, and how you think about the code you're writing.

It's not that they really care whether you'd be able to reverse a list of numbers, let's say: that's likely just a prompt to stimulate an in-depth technical conversation. If you know how to solve the problem, great, but what's even more important is that you know how to *talk* about solving it.

Even though the format of the interview is, superficially, just a candidate writing some code on the whiteboard, my real goal here is to have a conversation about it. "Why did you do it that way?" "What are the performance characteristics of your algorithm?" "What did you forget?" "Where's your bug?"

—Joel Spolsky, ["The Guerilla Guide to Interviewing"](#)

Show you know how to collaborate

If you get stuck on the task, stop. Explain your thinking up to this point, describe the problem you're having, and ask for suggestions or hints. Even better, suggest an approach you *could* take, and ask the interviewer if they think it makes sense.

When things haven't gone well, don't beat yourself up too much. Whiteboard challenges are unrealistic, stressful, and difficult, and none of us can perform to the best of our abilities under these conditions. Sensible interviewers know this, and will interpret your results accordingly.

One of the best ways I know to evaluate whether someone can program well or not is to *pair program* with them. Really smart companies will include this in their interview process, especially if they're serious about code quality.

You might be invited to pair with another engineer on an example task, or even a real task. If not, ask if you can do this. It'll be more enjoyable for you than a whiteboard challenge, and very likely more informative for the interviewer.

A truly successful interview resembles a technical discussion between engineers already working together. One of you ran into a problem. You are using the whiteboard to solve it together.

A successful interview does not look like a one-way test where only one person asks questions, and the other only gives answers. Use your communication skills to manipulate the interview into a meaningful discussion.

—Chris Laffra, “[Interviewing](#)”

Learn some CS basics

It's not so much that you'll be *doing* computer science every day in your job, unless it's an academic one. It's more that computer science gives us the tools and the language to *talk* about programs: their efficiency, scalability, correctness, and so on.

This doesn't mean you need to memorise a whole bunch of complicated algorithms; after all, when you're doing your work, you can always look those up if you need to. For the purposes of the interview, though, it's a good idea to read a few introductory textbooks and familiarise yourself with the important *concepts* involved, especially if you don't already have a CS background.

For example, if you're not sure what a tree is, or how “Big-O” notation works, or why you should never use a bubble sort, you might be well advised to hit the books a little. These things are nowhere near as complicated as they sound, and you can master everything you *need* to know about them in a few days of study.

But practice beats theory

A decent, though by no means extensive, theoretical understanding of common CS topics will help you a lot in interviews, and give you a big advantage over those candidates who haven't bothered to study them at all.

Practical experience is even more important, though. It doesn't help much to know what a linked list is in theory, if you can't actually write the *code* for it.

Interview tasks, if they're well-chosen, will not require you to know the details of some arcane high-performance algorithm. More likely they'll give you a problem for which you need to come up with a *decent* algorithm to solve.

For example, you might be asked to write a program that takes a binary number and prints it in decimal format. This isn't an especially hard task (at least, when you're not

under the pressure of an interview), but there are lots of different ways to do it.

What the interviewer will primarily be looking for is not simply the best solution, but your ability to *evaluate* and talk about different solutions, their relative efficiency, and what factors affect it. Putting a little time into learning the basics of things like [algorithmic complexity](#) can really make a difference to your chances.

The take-home task

While whiteboard problems can be a good way to stimulate technical conversations, they don't really say much about someone's actual programming ability. Instead, a *take-home task* is a challenge you can complete at home, in your own time, using your own computer, development environment, and so on.

Scope and time management

It's better to produce a basic, but working, program than one that is complex and ambitious, but unfinished.

That said, you should take all the time you need to get the program right. It's not about endless polishing and tiny refinements, but showing that you can engineer software to a high standard. That means careful thinking about the design and architecture of the code, detailed testing, and refactoring the program until it is as clear, elegant, and logical as you can make it.

Don't worry too much about any time limit you've been given; this is usually just a suggestion. Take as long as you need to finish the program to *your* standard.

It's about the programming, not the program

The goal of the take-home test is to find out how you cope with semi-realistic programming tasks. Can you break the problem down into manageable sub-goals? Can you avoid wasting time on irrelevant things, and focus on the key requirements instead? Can you solve a problem in the simplest possible way, or are you easily lured into over-complication?

Pay careful attention to exactly what is asked of you. If the task description stresses the importance of unit tests, for example, make testing your priority. If your program is supposed to produce output in a specific format, make sure you match this precisely. Another thing the take-home task will examine is your ability to understand and work within a set of constraints.

You don't need to put absolutely *everything* you know into the program itself. Instead, you can write down all the things you considered, and list all the improvements or extensions that you think are appropriate. For example, you could mention that you know about certain possible optimisations, but explain that you've omitted them for clarity, and show that you know when or why they might be needed in production.

Documenting your process

A take-home task isn't a memory test: you can use Google and other resources to re-search all the information you need. That's just sensible software engineering, and unless there are any specific instructions to the contrary, you should take advantage of all the resources that you would in your normal programming work.

Use the results of your research in your program, but use comments and documentation to show where you found it, what you learned, and what other possibilities you considered. Provide links, citations, and other resources so that your conclusions can be checked.

Show your ability to recognise engineering trade-offs, and to make reasoned decisions about them. You could say, for instance, what changes you would make if the program needed to handle more than a given amount of data, or scale to more than a certain number of transactions. This is another opportunity to show the breadth of your knowledge and understanding.

If possible, get a friend or colleague with more experience than yourself to review the program before you submit it. Talk over the problem with them, and practice talking about your decisions, and explaining how and why you made them. Get them to ask a few questions about the program, and if you find any of them difficult, spend a little time working out the best possible answer.

Aim to impress

There's a tendency for some candidates to treat the take-home task as a kind of mundane box-ticking. They aim to complete the task with the minimum of effort, doing exactly what's asked for and nothing more. I think this is a missed opportunity, and I think you can do better.

With that in mind, I suggest that you aim not just to complete the take-home task, but to *knock it out of the park*. Go above and beyond the requirements. Ask yourself what would impress you in a solution to a task like this, and do just that.

Spare a thought for the poor interviewers, who may have to sift through a hundred solutions, most of them rather mediocre or just plain bad. What will make *yours* the best? You should produce something so eye-catchingly brilliant that it goes straight to the top of the pile.

When the interviewers are discussing your application, they should be saying "Ah, yes, the one who did the incredible take-home task." And if they're saying that, your chances of getting the job are pretty good.

The in-depth interview(s)

Let's say you've passed the initial screenings and technical tests with distinction, and you're now invited to a more in-depth interview, probably with a more senior person at

the company.

Be prepared for a multi-stage interview process

If you get called back for further interviews, you can take this as a sign that you're doing very well indeed. There would be absolutely no point in continuing to interview you otherwise.

Technical hiring can be a lengthy and somewhat exhausting process for both parties, and this partly reflects the fact that there's a big downside risk for the company if the hire doesn't work out.

It is much, much better to reject a good candidate than to accept a bad candidate. A bad candidate will cost a lot of money and effort and waste other people's time fixing all their bugs. Firing someone you hired by mistake can take months and be nightmarishly difficult, especially if they decide to be litigious about it. In some situations it may be completely impossible to fire anyone.

—Joel Spolsky, [“The Guerilla Guide to Interviewing”](#)

But don't persist if it doesn't feel right

It's fine for you to stop the process at any point, if you have any doubts about whether you *really* want this job. You'll be saving both yourself and the company a lot of wasted time, so don't hesitate to pull the plug if you feel it's right. Trust your instincts.

It's easy to convince yourself that you're just imagining things, or being oversensitive, but the truth is that your subconscious is usually a pretty good judge of the situation.

If it doesn't feel right, then it isn't right, and persevering with the process in that case is always a mistake. Cutting your losses now is better than taking the job and then finding out that it's not for you.

It's about who you are, not what you know

If you've reached this stage, the company has already satisfied themselves that you can probably *do* the job, in terms of the day-to-day work. The purpose of further interviews is now to select the very *best* out of the set of plausible candidates, and eliminate those who won't work out for reasons other than basic competence.

This is likely less about what you *know*, or have done, than it is about the *person* you are. For example, are you someone who's friendly, approachable, and collegial? Can you discuss technical issues with others in a productive, constructive way?

How do you deal with people whose opinions differ from yours? Can you take direction and guidance from more experienced engineers, and be a helpful resource and support for your juniors? Can you control your emotional reactions and stay focused when things don't go your way?

Show that you're a smart person who gets things done

Most companies are looking for someone who is fundamentally (a) smart, and (b) gets things done. It's the combination of both qualities that makes an effective engineer.

People who are smart but don't get things done are completely impractical. They would rather mull over something academic about a problem than ship on time.

People who get things done but are not smart will do stupid things, and somebody else will have to come clean up their mess later. Not only do they fail to contribute, but they soak up good people's time.

—Joel Spolsky, “[The guerilla guide to interviewing](#)”

In the later-stage interviews, then, focus on demonstrating these two qualities, rather than trying to show off specific technical skills. If you didn't have the technical chops, you probably wouldn't still be here: the process is now about showing who you are, not what you know.

“Do you have any questions for me?”

Even if the interviewer doesn't prompt you to ask your own questions, it's a good idea, and shows that you're engaged and interested. The ideal question does all three of the following:

- Learn new information
- Show why you're a great candidate for this job
- Show that you can ask a penetrating question

Some example questions

If you're interviewing for a development job, for example, you can ask about how the team operates: do they use pair programming? What's the code review process? One thoughtful and nicely open-ended question is:

What is the entire process for a one-line change in code to make it to production?

You can also ask specific technical questions like “I see you use Technology X; how do you deal with Problem Y?” Or “I'm really interested in learning more about X—can you tell me what kind of opportunities there will be for me to do that?”

More broadly, you can ask about the decision-making process. What happens when people disagree about the best way to solve a problem? Are decisions made by consensus, or handed down by management? How much input does the team have into product management and design? And so on.

A very high-level question you can ask is “How does the company make money?” On the other hand, if the answer to this is obvious, you can ask instead “How does what I'll be doing connect to the way the *business* makes money?”

After the interview

At the end of the interview, make sure you're clear about what the next stage is and when you can expect it to happen. A good interviewer, for example, might say something like "We'll let you know by next Friday whether or not we'd like to move forward with your application." If they don't give you a timeline, ask for one.

Thank your interviewer

A nice thing to do, that can only work in your favour, is to send a thank-you message the day after the interview. Tell your interviewer that you enjoyed the conversation, and that you appreciate them taking the time to speak with you. Be brief, but sincere.

Interviewing can be a challenging and demanding process from the other side of the desk, too. Very few people ever bother to thank their interviewer for their time, and it makes a favourable impression when they do.

If you don't hear from the company within the agreed time, it's fine to send a follow-up message. You can say something like "I'm just checking in to see whether there's any news on my application—I was told to expect to hear a decision by X date."

When you get ghosted

Sadly, it's quite possible that you may never hear anything at all from the company after your interview. So-called *ghosting* seems to be more and more common these days. Rather than contacting you to say you didn't get the job, the company might simply drop all contact with you, and stop answering your calls or emails.

This is unacceptable, but it happens all the same. Don't sweat it; just make a note of everything you learned from this particular round, and move on to the next.

Surviving a "no"

When the answer arrives and it's a "no", that can be very upsetting and demoralising. When you've put a lot of time into a particular company's hiring process, and feel that you've genuinely given of your best in tech tests and interviews, it's devastating to be told that you haven't made it.

I can't take the pain away, but I *can* tell you that it happens to us all, many times in the course of a career. You won't get most of the jobs you ever apply for, and that's simply a statistical inevitability. It doesn't mean you're not a good engineer, or a nice person, or that your applications or interview performance are substandard. It just means that you didn't win this particular lottery draw.

In fact, if you *did* get every job you ever applied for, you should worry. That would mean you're not setting your sights high enough.

Getting feedback

Technical feedback can be difficult to hear, especially if you're pretty experienced. But don't be tempted to answer back: arguing never turned a no into a yes. Instead, figure out what you're going to do differently next time.

For example, in a coding test where you suspect your approach may be misunderstood, plan to explain yourself better. Say that you're aware of the "standard" pattern, but you made an engineering decision not to use it because of reasons A, B, and C. Say why you think your approach is a better choice.

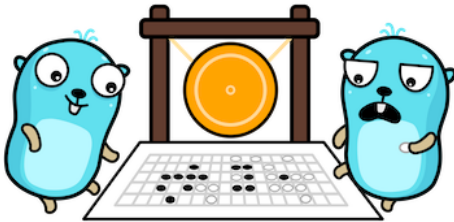
While a "no" needn't be a disaster, and, as we've seen, a "no, because" can be very valuable if you process it the right way, a "yes" is obviously the best outcome of any hiring process. And it might seem like the end of that process, but it's not.

There's one more stage to come, and it's arguably the most important of all: *negotiating* the terms of your employment. That's the topic of our next chapter.

4. Negotiation

The most dangerous negotiation is the one you don't know you're in.

—Chris Voss



So it's a yes! Congratulations, you've won the interview process. Take a breath, and give yourself a mental pat on the back. But don't start celebrating just yet, because things are about to get interesting.

Every job offer involves a negotiation, but it's one that many people don't succeed in, often because they don't even realise that it's happening.

The offer

Let's suppose you've survived, somehow, the seventeen interviews and the twenty take-home tests, you've waited anxiously for the company's decision, and it finally arrives: they want you. What happens next?

You don't have to accept right away

The first thing to say about a job offer is that you're not obliged to accept it. This might seem obvious, but if you know that the company's invested a lot of time and trouble in your interview process, you might feel that in a way you owe them something. You don't.

It's perfectly possible that, at the end of this process, and having done some thinking about what you want, you may decide that this particular job isn't for you. And that's perfectly okay. The whole process is about helping you *make* that decision correctly.

If you're not sure, it's worth asking if you can have a brief chat with someone who already works at the company, ideally in the team you'd be joining. The answer to this should always be yes. After all, if people enjoy working there, why *wouldn't* the company want you to hear from them?

If you decide, on balance, that you're going to turn down the offer, do so gracefully and with courtesy.

You can simply thank them for their time and for the offer, and say that having considered it seriously, you've reached with regret the decision that you're unable to accept it at this time. However, you wish them well, and if circumstances should change, you would not hesitate to reach out at some future date.

Negotiation is not a dirty word

An offer is just an offer: it invites negotiation. And this is the ideal time to do it.

The best time to negotiate your salary for a new job is the moment AFTER they say they want you, and not before. Then it becomes a game of chicken for each side to name an amount first, but it is to your advantage to get them to give a number before you do.

—Kevin Kelly, [“Excellent Advice for Living”](#)

Some people feel shy or awkward about negotiation, but it's not a dirty word. Nor is it rude, greedy, or unprofessional. It simply means discussing something, in order to reach a mutually acceptable agreement. Who could object to that?

The negotiation

To negotiate well, you need to understand the goals of everyone involved. The company's objective here is to hire the best person for the least amount of money. So their initial offer is likely to be somewhat less than the amount that they'd *really* settle for, if pushed.

So *your* objective is to push them, by making a counter-offer. Here's how to do that.

Knowing the numbers

First, do your research in advance. What is the market rate for this position? What about other compensation such as bonuses, stock options, health insurance, paid vacation, remote working, and so on? Talk to friends in comparable jobs, especially if they've been hired recently, and review a range of advertised salaries for your area and

industry sector. You should end up with a good idea of the upper and lower bounds of the compensation range available.

Next, cross-reference this against what you want to get to make it worth your while taking the job. What is the real *minimum* that you could possibly accept? If you're currently rather well paid, but you really want *this* job because you like the work or the company, you might be willing to accept less than you're making now. Or you might not, because you already have a better offer from another company.

Either way, figure out your lower bound in advance (and of course keep it to yourself). Now decide on the number you actually *want*, and find reasons to justify that number.

Making your counter-offer

Here's where your prior research pays off. State your counter-offer in the form of the number you've settled on, along with the reasons you've assembled to suggest it's at least plausible.

For example, you might say that having researched a number of comparable job vacancies or information sites, you think this amount is the current market rate for the job. Be able to produce your evidence if necessary, and show your working.

If you feel a little embarrassed at asking for your chosen amount, then it's probably about right. If not, you should be asking for a bit more.

The best way to actually make the counter-offer is to say your number, together with the reasons you've chosen it, and leave it at that. Don't waffle on past this point: that gives them a breathing space to think and prepare their reaction. Instead, just let the silence work for you.

What if they won't negotiate?

They might say something like "Sorry, but we can't negotiate on this." Don't take that for an answer. Politely enquire whether they mean that they don't *personally* have the authority to adjust the offer. If that's the case, ask them whom you should speak to instead.

But if they're saying that the *company* won't negotiate, well, that should be a red flag. If they're serious about wanting you, they'll at least be willing to *discuss* a better price. A "we can't negotiate" may sound like an ultimatum, but it might just be a placeholder.

Don't push back right away in that case, but just say something like "I understand it's difficult," and change the subject slightly. If you force someone onto the defensive, you make it hard for them to change their mind without losing face.

Instead, give them an opportunity to make a graceful retreat. Return to the topic a few minutes later and ask "Is there any way we can find to get closer to the figure I'm looking for?"

Be prepared to (politely) walk

If they still refuse to budge even an inch, then they're either bad negotiators, or just not really that interested in hiring you. The right response here is to say something like "Well, thank you for your time. I'm sorry we weren't able to come to an agreement on this occasion."

It's no good bluffing here. You need to really be ready to walk if you don't get what you want. If you're not, then maybe you're not in a strong enough position to negotiate as hard as you'd like to. That's okay: successful negotiation is partly about knowing when you can push things and when you can't.

Keep negotiating as long as necessary

More likely, though, they'll make a counter-counter-offer, which should logically be more than their original offer, but less than your counter. If you think this offer is really their upper limit, then accept it (if you want to). If not, keep negotiating as long as you think you can possibly get them to go higher.

The process doesn't all have to happen in a single conversation. It might take a few conversations, emails back and forth, crunching some numbers, and so on. To some extent, time is now on *your* side, because the company can't really interview anyone else until they know whether or not you're going to take the job.

Don't deliberately slow-roll them, though, because if you leave it too long, they *might* find someone cheaper than you, or at least less good at negotiating. Just be calm, polite, and measured, but persistent.

Let them win a little too

Eventually, and perhaps after a few more rounds of negotiation, you should reach a number that both sides can at least live with. A successful negotiation leaves both parties feeling that they've gained something, so it's wise to let them beat you down a little.

You don't *have* to accept an offer, however high or low. If you feel, on reflection, that you could do better elsewhere, then walk away.

After all, it's the job that really matters, not the money. You'll have plenty of time to earn lots of money over your career: what's important is that you earn it for doing something you want to do, in a place you want to work.

Equity as compensation

It's common in the tech industry for part of your compensation to include stock in the company, usually in the form of *options* that give you the right to buy a certain amount of stock at a favourable price.

Alternatively, you might get Restricted Stock Units (RSUs), which are essentially a promise to make you a free gift of a certain amount of stock at some specified future time.

Let's see how these things work, and how you should value them as part of the company's offer.

You're getting a stake in the company

Offering you equity in the company itself, as well as just money, makes a certain amount of sense. It literally gives you a stake in the company's success—or, more precisely, in maintaining or improving its stock price, which isn't always exactly the same thing.

From the company's point of view, stock options and RSUs help to encourage employees to stick around, rather than moving on as soon as they find a better-paid job. There's usually a *vesting period* that has to elapse before you can exercise your stock options or receive your RSUs: for example, you might have to stay with the company for a year or two before the first vesting.

Usually, the longer you work there, the more stock you get, and the amounts can eventually become quite significant.

What are stock options worth?

How should you value these options, then, in considering your total compensation? That depends on the company. If it's a large, well-established firm with a long track record of success, then its stock price is usually pretty stable. You can have a reasonable idea, then, what its options are worth: the difference between the *strike price* (what you'll pay for the shares) and the current market price.

On the other hand, as we all know, markets can take a downturn, and these can be both sudden and long-lasting. Individual companies can experience turbulent times too, regardless of their size or profitability. For example, in October 2022, Meta's stock price was down 61% on the year, putting a sizeable dent in the value of its options.

But, in general, and in the long term, stock prices usually go up. That's not *guaranteed*, sadly, which means that you can't consider stock options as valuable as cash. But they certainly have *some* value... provided that you stay long enough for them to vest, and for you to *exercise* them—that is, to actually purchase the shares at the agreed price.

You may not get to exercise options anyway

If you quit, or are laid off, for example, you may be able to keep options that have already vested. But you'll lose those you would have been entitled to if you'd stayed longer. And if you're fired for cause—or if you leave the company on bad terms for any other reason—you'll most likely lose any options that you haven't yet exercised.

Even if you *are* able to exercise your stock options after leaving the job, you may not want to. It's quite likely that you may need to conserve your cash, for example, rather

than putting a big chunk of it into purchasing stock, even at a favourable price. Or the market value may have fallen below your strike price, meaning that you'd lose money on the deal. RSUs, on the other hand, are *always* worth something, as long as the stock price stays above zero.

Valuing options in startups

It's wise to remember that the price of a stock is essentially the market's estimate of the company's future profitability. If the company is small, such as a startup, or its future is otherwise uncertain, then its valuation can have a very high margin of error.

Startups that are drastically undervalued by the market usually don't survive long enough to hire, because they can't get funding. Statistically, therefore, we should assume that most startups are considerably *overvalued*, and price their stock options accordingly.

Since most startups fail pretty quickly, we should discount the value of stock in them by a factor of perhaps ten to a hundred. In other words, if you're offered stock options currently valued at \$100, treat them as equivalent to \$1 of hard cash.

You might feel this is overly conservative. If so, I have some stock options to sell you. Seriously, though, it doesn't seem unreasonable to be pretty risk-averse when the company is effectively gambling with *your* money.

Beyond cash and equity

Negotiations like these are often focused on direct compensation, such as salary and equity. This makes sense, but these are also areas where a company usually has the least flexibility.

Creative negotiation

For example, the company may say, quite reasonably, "We just can't go any higher than X at the moment. Times are tough, and the board has put a hard limit on salary offers."

"Fine," you can say, "so let's talk about other kinds of compensation." For example, what if they were to give you more paid vacation time? Or to work from home one day a week? Or to work a four-day week? Or a more generous relocation allowance, a more important-sounding job title, or even just a bigger monitor? All these things are usually within the discretion of even the most financially constrained hiring manager.

Creative negotiation, then, means finding ways to improve the deal even when it seems like it can't be done. No company should object to you suggesting novel or unconventional ways to reach an agreement.

If anything, it supports the idea that you're a smart and thoughtful person who can get difficult things done. Exactly the kind of person they're looking for, in other words.

Can it ever hurt you to negotiate?

Some people are hesitant to negotiate hard because they fear that it might put the company off them. Maybe it will. What should you infer if that happens?

Only bad employers will view you negatively

If negotiating marks you out as a troublemaker, that's a sign of an oppressive and dysfunctional company culture. Maybe it's not just low pay that people are expected to endure without complaint.

In what *other* ways might you be mistreated if you ended up taking the job? You probably don't want to find out.

Therefore, you should *always* negotiate a job offer: politely, persistently, courteously, but determinedly. You simply can't lose by it. Even if it fails completely, that very failure teaches you something critically important about the company. Specifically, that you should walk away from it as quickly as you can.

Negotiation is a key skill for any job

Resolving disagreements peacefully and productively, and being able to reach tough decisions in a way that preserves good relationships between all parties involved, is far from easy. But, if you can master it, and you will, negotiation is the key to a successful career.

I can't promise that, even if you're a terrific negotiator, the results will always be positive. You may try really hard and still come away with nothing. Ultimately, though, you'll gain far more than you'll lose from negotiating skilfully but fairly throughout your career.

Negotiating your contract

If and when the company offers you a draft contract of employment, remember that this is only an opening offer. Just like your salary, you can and should negotiate it assertively, but courteously.

Check the fine print

You might assume that the job has normal office hours: nine to five Monday to Friday, or whatever is usual in your country or sector. But the contract might say something different. It might say that, actually, the company can change your hours whenever it wants to, or "occasionally and when necessary", which amounts to the same thing.

It might say that, while you're required to be in the office or online for work at certain times, you're also expected to be available out of hours or on weekends in some situations. For example, software developers might be required to take on-call shifts on a

regular basis, or operations engineers might be paged at *any* time if something goes wrong (and it will).

If you're okay with this, then it's okay. But be sure you know what you're signing up to, and don't feel that you just have to say yes to anything they ask.

Get the whole agreement in writing

Don't be tempted to take someone's verbal assurances about the contract terms, if the document you're given to sign says something different.

For example, they might say "Oh, don't worry about that, it's just legal language—we never actually enforce it." Fine. If that's true, then they won't mind putting it in writing. But you may find them curiously reluctant to do so.

Look carefully at the language describing the work that you'll be doing. Some companies use a bait-and-switch technique: you think you're being hired to do X, and you end up doing Y, which wasn't what you wanted. This is where being able to refer back to the original job advertisement can help you.

Again, if the language around what you're actually expected to do in the job is fuzzy, open-ended, or missing entirely, that's a problem. Watch out for all-purpose weasel words such as "such additional so-and-so as circumstances shall occasionally require".

Don't be afraid to ask for such loose wording to be tightened up in your favour; that won't be possible after you've signed.

You'll have no power to change the agreement later

Once you sign the deal and you're *in* the job, you have very little negotiating power. The company can more or less make you do whatever they want.

Get your own professional legal advice. Have a lawyer review the contract and discuss it with you. They will suggest a number of potential questions, clarifications, or changes that you probably would not have thought of yourself.

This may cost a bit more than you imagine, but a lot less than you stand to lose by signing a bad contract. Consider it a very sound investment.

Take your time and don't give in to pressure

As with any deal, there's an incentive for them to get you to sign as quickly as possible. They may put pressure on you in subtle or not-so-subtle ways, such as saying "We're so excited to have you on board—as soon as we've got this silly paperwork out of the way, we can start getting you all your great benefits and company swag."

Smile politely and decline to fall for this kind of thing. Take all the time you need to read and consider the contract, and if at all possible, get legal advice on it. *Never* sign it right away, in the meeting.

The anatomy of a contract

Now that you know how to handle the *process* of negotiating your contract, let's turn to the substance. What kind of things can you expect a typical contract to cover, or not cover?

What doesn't it say?

Contracts can be notable for what they *don't* say, as well as what they do. If you don't regularly draft or read employment contracts, you might not even realise what's missing.

Most conditions of employment are covered by statutory law, which would apply even where the contract is silent, but don't assume that law will necessarily be in your favour. Laws, after all, are generally made by people under intense lobbying pressure from big businesses with deep pockets.

And, since you probably won't be in a position to enforce the law anyway, it's far better to get all important conditions explicitly agreed as part of the contract. Don't see any entitlement to sick pay or parental leave? Then you won't get them—unless you ask.

Compare with similar contracts

Ask your friends in similar jobs if they're willing and able to share the text of their contracts with you. Compare them with yours. They may be very different in all sorts of ways, but this is useful information. You can go back to the company and say "I'm told it's usual for a contract to include X—how come this one doesn't?" Or you can say "This seems like it's not in line with industry practice—would you object to changing it?"

The contract doesn't matter, until it does

Most of a contract's provisions will be about what happens when things go wrong. For example, it says what action the company can take if you do something you shouldn't, or fail to do something you should.

It may even try to control what you can do after your employment ends. For example, it might contain a "non-compete" clause that prevents you from working for one of the company's competitors for a period of time, or from trying to hire away the company's other employees after you leave.

These stipulations are not unreasonable in principle, but they are often worded with breathtaking scope and longevity. A one year non-compete is defensible; ten years isn't. And you shouldn't assume that the company won't act aggressively to defend its rights, even long after you've left it. On the contrary, assume that whatever terms you sign will be enforced to the full extent of the lawyers.

Moonlighting and spare-time work

One question that may be important to you later on is whether the contract forbids you from doing work for *other* companies during your employment. It makes sense that the contract would ban you from working directly for a competitor, but it may say more than that. It may say that you can't do any other work *at all*.

If your career goals include independence, it might be important to you to be able to take on the odd side hustle. For example, if you'd like to become a consultant one day, you might want the freedom to do small, one-off jobs, such as helping someone resolve a problem with their project. Naturally, you wouldn't expect to do this on *work* time, but your contract may also restrict what you can do in your own time, including evenings and weekends.

Intellectual property

You might think that anything you develop by yourself in your own time belongs to you, regardless of where you happen to work. And I'd agree with you, but that's irrelevant: the question is, does your *contract* say so?

It's not uncommon to see contracts stipulate that *any* intellectual property you create belongs to the company, if it's created during your employment. That is, not just during the *hours* of your employment, but during the whole period that the contract covers.

Even if it only covers the use of company equipment, such as a laptop, the onus might well be on you to prove that you *didn't* use company resources to develop your project, and that could be a hard thing to prove.

Making the final decision

Just like the interview process, actually negotiating an employment deal can be a time-consuming and stressful business. That doesn't mean you shouldn't do it, of course, but just be aware what to expect.

Expect pushback, but don't yield to it

As with your salary discussions, expect the company to negotiate back at you, perhaps pretty hard. Or they might try to deflect you by making light of the whole thing. "Look, don't worry, none of these things ever actually happen in practice, so it doesn't matter. It's just legal boilerplate." Sure.

Or, more aggressively, "Don't insult us by acting like we're going to break the agreement. We're not. You're acting like you don't trust us." On the contrary, since contracts hardly ever make it to court, you're both extending each other a lot of trust. The contract exists to protect you both if that trust should ever break down.

And you will learn a surprising amount about your potential employer from negotiating your contract with them. How they act when you're still on more or less equal terms

tells you a lot about how they'll act later, when they have considerable power over you.

Remember you can still walk away

You won't get everything you want, even if the contract negotiations go as smoothly as they should, and that's okay. What's important is that you and the company reach an agreement you can both live with.

And that's not always possible, especially if you're too smart to sign whatever paper is put in front of you. If it turns out that you can't strike a mutually acceptable deal, that's too bad, but it happens. Chalk it up to experience, walk away, and try again with another company.

Don't be suckered into signing a bad deal because you don't have any other offers right now, or even because you've spent a ton of time on it. Failure *is* an option. Right up to the moment ink hits the paper, you still have the option to say "thanks, but no thanks" and move on.

Assume this is the best the deal will ever be

You should proceed on the assumption that what's in the contract is what you'll get: no more, no less. By all means *hope* for more, but hope isn't a strategy. Don't take a bad deal on the assumption that you'll be able to improve it later. Chances are you won't.

Even negotiating and signing the perfect deal doesn't guarantee that you'll find your work rewarding, that you'll get on well with your colleagues or management, or that you'll make the progress you want to make towards your ultimate career goal.

That all depends on what you do in the job, and this is the subject of our next chapter: *work*.

5. Work

The only thing harder than trying to design software is trying to design software as a team.

—Joel Spolsky, “[The Big Picture](#)”



Welcome aboard! You’ve just started a new job, maybe even your first job. What should you expect? How should you act? What do you need to know to succeed? In this chapter, let’s explore the world of work.

Starting out

You might be feeling nervous about the prospect of starting a new job. If so, don’t worry, you’re not alone. There are lots of things to worry about, as with any major life change, but the chief worry in most people’s minds is probably:

Can I even do this?

The fact that you’re feeling this way is a good sign, oddly enough. If you’re going to make progress in your career, then each new job should at least *seem* slightly beyond your current ability.

This might seem obvious, but it’s something many people fail to take into account. When they look for jobs, they’re looking for jobs they *can already do*. In other words,

jobs where the requirements more or less match their current set of skills.

Impostor syndrome

So *impostor syndrome*—the feeling that you’re not really up to the job and sooner or later people will find out— isn’t necessarily a problem. In fact, I’d say it’s a sign that you’re taking on the right kind of challenges. You should be concerned if you *don’t* feel a bit of impostor syndrome in any new job.

Also, everybody else feels just the same as you, however experienced or senior they are, and however relaxed and confident they might look on the outside. Everyone is secretly thinking “I hope nobody finds out I have absolutely no idea what I’m doing!”

You can do this

Having evaluated you carefully, and probably at considerable length, the company decided you were the right person for this job. Getting that decision wrong would be unfortunate and expensive, and companies that consistently make bad hires don’t survive for long.

Now, I’m not saying you should be *over*-confident; that would be silly. What I *am* saying is that you should regard a modest amount of impostor syndrome as normal, and that while it’ll never go away altogether, it’s also not a sign that anything’s wrong or that you shouldn’t be doing what you’re doing. Instead, just keep in mind that this is what *growth* feels like.

And then I thought to myself, “You know, what they think of you is so fantastic, it’s impossible to live up to it. You have no responsibility to live up to it!”. I am what I am, and if they expected me to be good and they’re offering me some money for it, it’s their hard luck.

—Richard Feynman, “[Surely You’re Joking, Mr. Feynman!](#)”

Don’t show off

In the first days or weeks of the new job, you may feel a strong impulse to *demonstrate* to everyone around you just how skilled you are and how much you know. This often takes the form of talking a lot in meetings, making points to try and sound smart. Or, worse, pointing out what’s wrong with everybody else’s code and telling them how it could be improved.

This usually doesn’t go well. When someone in the team is constantly showing off and acting like a know-it-all, we don’t tend to assume they must be over-compensating for hidden feelings of inadequacy. We just assume they’re a deeply annoying person.

It can also be very exciting to join a team of smart people who know a lot, and sometimes this excitement can overwhelm you. If you’ve been missing some intellectual stimulation and the company of interesting people, you may find that your new position generates a lot of fresh ideas. That’s great! You don’t need to tell them to everybody

all at once, though. Capture them in a notebook and revisit them after a month or two to see if they're still as interesting as you thought.

Listen before you speak

Spend your first few weeks of any new job or role in *listening* mode. Absorb as much information as you can about how things work and who does what. If you see something that you think isn't optimal and could be fixed, keep it to yourself—for now.

Think about how irritating it is when you tell a friend about some problem you're having, and they say "Why don't you just...?" Well, you probably already thought of that, and it doesn't work. But it's annoying that they assume you hadn't even looked at the most basic and obvious solutions.

So don't be *that* person. Instead, assume that if something seems off to you, it's probably because there's some important factor you're not aware of yet. Try to learn more about the situation and the problem. Talk to people who've worked on it. Ask questions like "Can you help me understand why...?"

How to make suggestions

Once you've spent a decent chunk of time in listening and learning mode, and if you *still* think you have a good idea that someone else hasn't already tried, you can start gently and diplomatically introducing it. For example, you could say "I was thinking that maybe it would be good to do X. Does that sound reasonable to you, or is there some issue I don't know about?"

If the first two or three people you ask can't instantly find a reason to reject the idea, then maybe you're on to something. You could ask them to help you try it out. Note the words "help" and "try out". You don't want to come across as saying "Hey, losers, I fixed your code for you. Yeah, you're welcome," even if that's how you really feel. There are ways and ways of doing things.

Settling in

If it's natural to feel apprehensive before starting a new job, it's even more natural to feel a bit overwhelmed when you do actually start work.

Coping with "Day 1 Panic"

Don't panic. You *will* cope. Just look around you: all your co-workers must have felt exactly the same when they started, and now they're doing fine. They probably don't even look stressed. It's amazing how quickly we can adapt to new circumstances and start to thrive.

Spaaaaaaace engineers talk about [max q](#), the point of peak dynamic pressure experienced during a rocket launch:

$$q = \frac{1}{2} \rho v^2$$

At ground level, the atmosphere is thick, but the rocket is moving slowly. Higher up, the density of the air decreases, but the rocket is also moving through it faster. So there's a point fairly early in the flight where the speed and density curves intersect, and the rocket experiences its maximum structural load: $\max q$.

If the vehicle is going to fail under stress, this is when it's likely to happen, and you may feel like you're going through this phase too. How do rocket scientists deal with this problem? They throttle back the engines slightly, to reduce the loads on the structure. You can do the same.

In other words, throttle back your expectations of yourself slightly. Don't assume you should be in a position to start delivering value right away, or even within a few weeks. It takes time to learn your way around a complex codebase (and is there any other kind?)

The trial period

There may be a formal trial period, or probation, during which it may be easier for the employer to dispense with your services if it turns out you're not a good fit. This can add to the stress of feeling like you have to prove yourself, because in this case you do.

One way to manage it, and to improve your chances of making it through the probation filter, is to talk to your manager and have them clarify exactly what's expected of you. Will you be evaluated on specific things, or is it more of a general feeling? If people have washed out in the past during the trial period, why? What can you do to avoid the same result?

Check in regularly with your manager during the trial period to see how you're doing, and whether you need to adjust course. Don't wait for them to tell you.

Instead, be pro-active, and get regular feedback about your performance. You'll learn a lot from these conversations about the clarity, transparency, and consistency of your new employer's communications with its staff.

Expect to be slow at first

When everything is new, and nothing is routine, even the simplest-seeming tasks can be a struggle. Need to make a one-line change to some documentation page? There's actually a tool you need for that. How to install it? Well, the instructions are out of date and don't work, so you'll have to find someone who knows and ask them. But they're too busy to help you right now. And so on.

"This is ridiculous! It should only take five minutes," you may be saying to yourself. Sure: when you have all the tools you need, you know how to use them, and you're ex-

perienced enough to deal with any little glitches, it's a five-minute job. But when none of those things are true, it could easily take you all day.

So don't be too tough on yourself, and don't assume that others will be feeling impatient if you're not reaching orbital velocity just yet.

In fact, your experiences as a new person are very valuable input. Things like documentation *get* out of date because people become familiar with the processes and don't need the docs anymore. So any time you find something like this, offer to update it (with the help of someone who can explain what it *should* say).

This is what learning feels like

Don't wait till you get stuck before asking for help. Get help so that you *don't* get stuck. If there's no help available, and you're struggling, see if you can break down the problem into smaller steps. Focusing on one small thing at a time helps reduce anxiety, and every tiny task you can check off your list increases your feeling that you're getting somewhere.

You *will* get frustrated at first, and demotivated, and maybe even depressed. But this will pass. You will feel dumb for not being able to master things right away. But this is completely normal: indeed, that's what it *feels* like to learn something new.

When you lift weights at the gym, your muscles will be sore; that's what it feels like to get stronger. When you lift weights with your brain, it feels sore, too. But little by little, it is getting stronger. And, just as your body needs rest and downtime between workouts, to recover and grow new muscle tissue, your brain needs that, too. Go easy on it, and on yourself.

Being effective

So take heart, you *will* survive that scary “max *q*” period, where your brain feels like it's exploding every five minutes. You may feel convinced that you're going to get fired for rank incompetence at any moment, but that won't happen. At least, let's hope not.

BOSS: *Kramer, I've been reviewing your work. Quite frankly, it stinks. These reports you handed in... it's almost as if you have no business training at all.*

KRAMER: *Well, I'm just trying to get ahead.*

BOSS: *Well, I'm sorry. There's just no way that we could keep you on.*

KRAMER: *But I don't even really work here.*

BOSS: *That's what makes this so difficult.*

—Seinfeld, “[The Bizarro Jerry](#)”

How long does it take? There's no specific moment when you realise you're no longer a newbie, and you start to feel like you're getting a handle on things. But there'll come a time when you look back on yourself from a few weeks ago, or a few months ago, and think “What was I even worrying about? This is fine!”

Social capital

A useful thing to understand about work, or any kind of group activity, is that you have a certain amount of *social capital* to play with. Being helpful and kind increases your social bank balance, if you like. Being honest, trustworthy, reliable, humble, and easy-going increases it, too. On the other hand, being arrogant or unpleasant will drain your capital rapidly.

And it's not just about being a good person, though you should certainly care about that. You *need* social capital to do your job. When you've got a problem you can't solve and you're under pressure to get it fixed right now, you can draw on your capital by asking a co-worker for help. Needless to say, if you haven't helped them in the past, they won't feel much like helping *you* now.

A team is not a group of people that work together. A team is a group of people that trust each other.

—Simon Sinek

Social capital is what makes people listen to your ideas, or give weight to your opinions. It's how people see you as a team player. It's what makes them want to help you, even when that's not strictly their job. It's the currency of trust. And trust is what makes the whole thing work.

Managing your commitments

To be trustworthy, and earn social capital, it's important to prioritise your *commitments*. In other words, if you say you'll do something, you need to do it.

If you don't, people will gradually stop trusting you, and your social capital will dwindle away. The most dangerous thing about *that* is that you may not even know it's happening.

People don't tell you when they stop trusting you.

—Jerry Weinberg, “[Secrets of Consulting](#)”

The biggest threat to your trustworthiness is probably not that you're a horrible, deceitful person who tells lies. It's that a perfectly natural desire to please causes you to *over-promise* and *under-deliver*.

We all want to say “yes” to things. It makes everyone happy, until you realise you've taken on far too much, and you have to belatedly tell them you won't be able to do the thing after all. Now they'll be *unhappy*, not just because you made a promise and then let them down, but because it's probably now too late for them to find someone else to help.

Saying “no” is a superpower

While ill-considered promises might earn you a little temporary capital, your failure to deliver will cost you more later, resulting in a net deficit. On the other hand, say-

ing “no” now might cost you a *tiny* bit of capital in the short term, but in the long term you’ll gain much more, because you’ll be known as a good commitment manager.

When someone asks you for a commitment, they don’t *really* want you to say yes unless you’re actually going to deliver on it. So think carefully about your answer. It’s fine to say “Let me get back to you when I’ve checked my schedule.”

And, if you eventually conclude that this *isn’t* something you can be 95% confident about delivering on time, you should say “I’d really like to help, but it just doesn’t look like it’s going to be possible to fit this in with my existing commitments.”

That’s a nice way to say no, because it underlines the value you *place* on your commitments, and how seriously you take them. The person you’re talking to knows that, even if you can’t help this time, when you *do* say you can, you will.

Finding your feet

One thing that’s guaranteed when you join any new organisation, division, or even team: they’ll do things differently from what you’re used to. There will be unfamiliar and unexpected rules, some written, most unwritten, and you will break them without even meaning to.

That’s okay. If you listen more than you talk, and pay close attention to what everyone else is doing—and even more importantly, *how* they’re doing it—you won’t get yourself into too much trouble.

But there will be friction at some point and about some things, so how do you deal with that?

Fitting in with the way others work is hard, and it takes some time and patience. You have to be sensitive to what’s going on, and tolerant of things that aren’t the way you like them to be.

*A new kid in school, got to follow the rule, you got to learn the routine...
You got to fill out a form first, and then you wait in the line
It’s all his design, no one cuts in the line, no one here likes a sneak
You got to fill out a form first, and then you wait in the line*

—Paul Simon, “[The Afterlife](#)”

When you go from working largely by yourself to working in a team, especially a big team, you’re likely to encounter lots of things that you find hard to understand, or even laughably wrong.

Coding standards, patterns, and practices vary widely: even the layout of brackets and whitespace can be a point of debate, in languages that don’t have a canonical standard format.

go fmt’s style is no one’s favourite, yet go fmt is everyone’s favourite.

—“[Go Proverbs](#)”

All teams have a house style that you need to learn and abide by. You may not like or agree with all its choices, but then, your fellow team members probably feel the same. The point of standardising practices like this is not to do everything perfectly, but to stop people wasting time arguing about trivialities.

Don't rearrange the furniture

A codebase is a little like someone's house: they've probably lived there for a while, and they have things mostly the way they like them. Also, they're used to the arrangements. It can be difficult to have someone new come in and start blundering around the place, falling over the furniture or breaking the crockery.

And you wouldn't be too pleased if your new roommate or partner started taking issue with your choice of decor or furnishings. You certainly wouldn't expect them to just start rearranging things to suit themselves: "I'm used to having the couch face the window—I was sure you wouldn't mind me moving it."

You'd expect them to adapt themselves to your preferences, even if it's not what *they're* used to. Now imagine those roles reversed. *You're* the guest in someone else's codebase or project. Don't complain, criticise, question, or move things around without asking. Things are the way they are for a reason, even if that reason is simply "we haven't got around to fixing that yet".

A little empathy goes a long way here. If something seems wrong to you, it might just be the way it is because other people have thought about it a little more carefully than you have. Or maybe they're fighting a desperate battle that you know nothing about, and compared with that, this issue seems ridiculously trivial. And, if something seems a little half-finished, maybe that's because it is.

Making the right first impression

Don't look for problems your colleagues seem to have missed. Instead, ask *them* what problems they could really use help with. You could say "Do you have any little tasks kicking around that no one's had time to get to lately? Tackling those would help me get familiar with the codebase, while still contributing something useful." This is much more likely to get a favourable, or even rapturous, reception.

It sends out a signal straight away that you're not here to make them look bad. In fact, you're here to make them look good. You're not going to create extra work for them: instead, you're going to *lighten* their load. Once you've established this idea, you'll find your welcome a lot warmer.

Collaboration is your biggest challenge

The biggest challenges you'll face at work won't be build failures or the type system, they'll be about trying to work with other people. If everyone were smart, nice, and competent, things would be easier, but you'll have to work with plenty of the other kind too.

Even when team members trust each other and get along, there will be conflicts and difficulties. Collaboration is just *hard*. This is known. And no one is a born collaborator; it's something you have to learn and practice to get good at. We're all somewhere along that journey, and throughout your career you'll meet people at every level of collaborative ability, from "zero" to "decent".

People skills

While technical skills predominate in the job application process, once you actually make it into the workplace (whether physical or virtual), you'll find that your tech chops are taken for granted. What's going to determine your ultimate success or failure in this job is your *people* skills.

Unfortunately, not everyone realises this, and you'll find a few people in every team or company who, while they may be super smart and experienced, are also anti-social jerks. Sorry about that.

You can do your little bit to redress this problem by learning and demonstrating how people *should* treat one another, at work or elsewhere: with respect, consideration, and kindness. Not everyone will reciprocate, but that's not why you're doing it.

Human relationships

We're told we should act "professional" with our co-workers, but that shouldn't be interpreted as treating them like robots. They're just as human as we are, at least in most cases, and the secret to a good professional relationship is having a good *human* relationship.

LISTER: *See, Rimmer, the trouble is you've never got time for people. You're too busy trying to be successful.*

RIMMER: *I **have** got time for people. What about all the time I spent licking up to Todhunter even though he was a total gimp? And Captain Hollister? I went out of my way to simper round him.*

—Red Dwarf, "[Thanks for the Memory](#)"

We're all very good at knowing when people are just being nice to us because they want something. It only works when you actually mean it. But then it works magnificently.

Overcoming shyness

People are always astonished and dismayed to be told that they come across as unfriendly. Almost no one acts this way on purpose. Instead, they might just not have the knack of making light, friendly chat. By initiating the chat yourself, you'll often be able to break the ice and turn a cool relationship into a warm one.

And no one expects sparkling conversation at work. Just making the effort at all is the compliment you're paying them. "So, how about that local sportsball team?" won't win

you any prizes for originality, but it might win you a smile, and the next time will be easier.

Social bonding

Socialising outside work helps a lot. Sure, this is hard if you work mostly remotely, but not impossible. Remote and distributed teams often have non-work get-togethers for just this reason. If your team doesn't do this, maybe it should. You can have a book club over Zoom or Teams, do a quiz, or even just block out the last 15 minutes of your weekly meeting for general chat. A formula that can help here is to ask each person to nominate something interesting but non-work-related that they read, watched, or heard about this week.

If your company or team doesn't organise social events outside work, try taking it on yourself to do so. Pick some activity that just about everyone can enjoy, even if it's just ironically: bowling, escape rooms, a board game tournament, go-karting, a movie marathon, a charity walk. The main thing is to have the chance to interact with your colleagues outside work: you might be surprised what a difference it makes to your subsequent interactions *at* work.

Listening

Listening is another interpersonal superpower that's often unwisely neglected. Most of us are pretty good at talking, usually at considerable length, but someone who knows how to *listen* is a rare and precious find.

We tend to *think* we're listening to someone, when what we're really doing is working out what we're going to say next, and waiting for them to stop talking so that we can make our devastating point. And even if we're paying attention on some level, that still isn't quite the same as actually listening.

If you listen carefully to what someone tells you, instead of humming a tune inside your head and waiting for them to finish, you might learn something useful. Also, if you can keep quiet long enough for them to work through the process of turning their thoughts into words, *they* often learn something, too.

Experienced programmers call this *rubber-ducking*: sometimes, the simple act of explaining your problem to someone else in sufficient detail is enough to trigger the insight you need to solve it. Rubber ducks make better listeners for this purpose than most human beings, apparently.

What *we* can do that bath toys can't, though, is to add value to the discussion by telling the speaker what we think we understand them to be saying. Our understanding may be right, or it may be wrong. Either way, it's helpful for the speaker to know how we're hearing them.

Reflection

True listening, then, is not a passive process, but an active one:

I recall a time when I was talking with someone who seemed to ignore everything I said. “You are not listening to me!” I accused. “Oh, yes I am!” he said. He then repeated word for word what I had told him. He heard exactly. But he wasn’t listening. He didn’t understand the meanings I was trying to convey.

—Robert Bolton, “People Skills”

Even if we’re showing all the signs of attending, the other person does not actually *know* that we’re listening unless we can show them that we’ve understood the import of what they said.

The simplest way to do that is to *reflect* it, by stating the essence of what they said, but in our own words. That usually means identifying the *feelings* they’re trying to communicate:

THEM: *That code review comment really made me angry. It was like “You’re so dumb for making this mistake.”*

YOU: *You felt attacked.*

THEM: *Exactly.*

When the speaker hears that we have successfully grasped their point, they usually say something like “Right,” or “Exactly”. If not, they will correct us. Either way, they are reassured that we’re at least trying to understand.

Don’t make it about you

The mark of someone who’s *not* listening, by contrast, is that they jump in as soon as there’s a pause. This is like saying “Never mind that, here’s what *I* think...”

A common fault among poor listeners is that they immediately try to relate what the speaker is saying to some experience of their own. “The same thing happened to me once when I was on the way to Shelbyville. Now, I was wearing an onion on my belt, which was the style at the time...”

The listener means well, because they want to let the speaker know that they understand exactly what they’re talking about, by drawing a parallel with their own lives. But by breaking the speaker’s flow and changing the conversation to be about *them*, they inadvertently did the opposite of listening.

Instead, just let the person say what they want to say in their own time, and in their own way. And if it seems that they *don’t* want to talk, don’t try to force it: just stay quiet. If they want to fill the silence, they will. Otherwise, you can just be comfortably silent together. Not all conversation has to involve talking.

Being a good co-worker

It's a curious feature of modern life that we're expected to have opinions on everything at all times, whether we're particularly well-informed about that topic or not. People have a bad habit of constantly probing and auditing each other's opinions, and challenging those that don't fit perfectly with their own.

The office isn't the place for this kind of thing (that's what the internet is for). Just let people think what they want, however nutty it might seem to you, and resist the urge to educate them on everything they're wrong about.

Keep your opinions to yourself

This isn't to say, of course, that you should *have* no technical opinions about, for example, the right way to structure a particular piece of code. Naturally you will, and the more you become a master of your craft, the more such insights you'll have. But the way to use that understanding is by *doing* the right thing, not by arguing with other people about what the right thing is.

And it's always wise to remember that, however sure you may be about something, others may think differently, and that's okay. You don't need to badger everybody until they think the same way you do. After all, you've been wrong about things before, and now you know better.

You can't learn anything if you don't believe others have anything to teach you. Seek to understand, not to persuade.

Don't be an advocate

One of the most annoying people in the workplace is the Advocate for something. You know of whom I speak. It's the person who thinks Linux is the greatest thing ever, everybody should use Linux for everything, and if you've made the mistake of buying a Mac instead, you're either misguided or an idiot.

Or maybe they're an Anti-Advocate: "Go is a terrible language, and you should feel bad for using it." Sure, buddy. Whatever you say.

These people are just *boring*, and their co-workers pretty soon learn to avoid their favourite subject, or to avoid talking to them altogether. But it's no good, because they will somehow manage to bring *every* conversation around to the topic.

Exercising empathy

It doesn't matter how nice a person is in other ways: if they have no empathy, they'll be a bad co-worker.

PETER: *Milton, hi, uh... could you turn that down just a little bit?*

MILTON: *I was told that I could listen to the radio at a reasonable volume from nine to eleven...*

PETER: *No, I know you're allowed to, I just thought maybe as a personal favour...*

MILTON: *I told Bill that if Sandra's going to listen to her headphones while she's filing then I should be able to listen to the radio while I'm collating, so I don't see why I should have to turn down the radio...*

—“Office Space”

Perhaps this is less of an issue now that remote working is becoming more common, but when you share a workspace with other people, do be considerate about what noises, smells, or other distractions you might be emitting.

I once had a manager who played all his voicemails back on speakerphone, audible to everyone around. And he got a *lot* of them, presumably because he was the kind of person that people try to avoid face-to-face conversations with.

Everyone in the office hated this, but no one ever said anything. People don't tell you when they start hating you.

Meetings

For some people, meetings *are* their work, but for most engineers they're simply an annoying time-suck, to be avoided whenever possible. When you can't avoid a meeting, though, here are a few tips for getting through them with as little collateral damage as possible.

Less noise, more signal

Some people love the sound of their own voice, and ideally all those people would just get together and have one giant meeting, while the rest of us get stuff done. Until that happens, you can at least avoid making the problem worse by keeping your own contributions to a minimum.

Don't feel you need to say something just for the sake of it. Indeed, if you get through the whole meeting without being *required* to say something, then you probably don't need to attend those meetings in future. It's a win-win.

On the other hand, if you're only in meetings when you need to be, and you only speak when you have something to say, then you have a better chance of getting people's attention.

The stakes can be high

Work meetings are an interesting social dynamic, because you're not just having a conversation with someone: you're talking to them *in front of their peers*, that is, effectively, in public. People are more sensitive in this situation than they might be otherwise. Something they might perceive as a slight, or that they think *others* might perceive as

a slight, can give offence in a meeting situation when it might not matter at all in a private chat.

If, as often happens, the boss is present too, that raises the stakes even further. Now it's not just your peers who are listening and judging you, but the person who has power over your compensation and promotion. Maybe even your continued employment. Under these circumstances, a word or two from your co-workers can do you great harm, or great good. It all depends.

The presence of a boss is like an amplifier for the risk or reward of what we say in meetings, and the bigger the boss, the greater the amplification. Group chats such as Slack or Teams are like a meeting that never ends, in this and other respects, so be careful. Treat anything you say or type in such a context as being said to everyone present, whether they appear to be listening or not.

And, it's important to add, treat anything you write to someone *privately* (for example, in an email or direct message) as though it will be shared with the whole company. One day, it might be. If you say something critical about a third party, assume they will see it sooner or later (and you may never know).

No one likes a doom-monger

One of the most endearing qualities you can have as a colleague is *enthusiasm*. We all know the person who shoots down every suggestion in meetings with the response “Well, that won't work, because...” And we are not fond of that person.

Yes, it can be useful sometimes to visualise potential problems, especially if you can come up with solutions. But just rejecting all ideas out of hand isn't constructive by itself.

Even if you think the proposal is fundamentally flawed, you needn't phrase your response that way. You could say “That's a really interesting idea. I wonder if...” and you can talk about the pros as well as the cons.

We just want to be visible

Have you ever wondered why some people seem to constantly speak up in meetings even when they don't have anything useful to say? The answer is that they want what we all want: recognition and validation. They want someone to say, in effect, “Yes, we see you, Bob, and we think you're a valuable member of the team.”

It's easy to deny someone this validation, even when we don't really mean to. Consider an exchange like this:

FEMI: *I was wondering if it would be a good idea to store the what-nots in a shared database?*

ERIC: *What we should really do is cache that data in a key-value store local to the node, or maybe in a CDN.*

(MURMURS OF AGREEMENT)

See what Eric did? He trampled right over what Femi just said, erasing their suggestion and replacing it with his own. Even though the one idea leads logically on to the other, Eric subtly managed to suggest that Femi was wrong, and that, in future, people shouldn't listen to Femi, but to Eric instead.

Maybe he didn't consciously intend to send that message, but that's what people *heard*. We all know an Eric. How can we avoid *being* an Eric? By building others up, not shooting them down.

Validating and being supportive

After this meeting, Femi may think twice about voicing their ideas in meetings in the future. They may also think, with some justification, that Eric is a jerk.

Here's a better way to handle this exchange:

FEMI: *I was wondering if it would be a good idea to store the what-nots in a shared database?*

ERIC: *That's a great suggestion, Femi. Could you talk about it a bit more?*

FEMI: (TALKS ABOUT IT A BIT MORE)

ERIC: *Yes, I like the sound of that. And what if we used a local key-value store, or even a CDN? Do you think that would increase performance?*

FEMI: *Yes, that's a great idea.*

Almost exactly the same conversation, at least on the surface, but with a radically different *subtext*. Eric first of all recognised Femi by name and validated that their suggestion was acceptable, and worth discussing. Rather than explain it further *himself*, as though it were his own idea, he invited Femi to talk about it, implying that he values their opinion.

Then he presented his own point, not as a substitute for Femi's plan, but as a natural evolution of it: *"Yes, and..."*.

Rather than imply that he's correcting Femi because he knows better, he asks Femi for their opinion about *his* suggestion. By deferring in this way, he sends the message that he thinks Femi is qualified to pronounce on other people's ideas, as well as originating them.

Femi will remember *this* conversation very differently to the first version. They will feel that they contributed something valuable, that the contribution was recognised and appreciated by the group, and that it spurred a discussion leading to an even better plan.

As a co-worker, *this* version of Eric was supportive, listening to Femi's idea and inviting them to talk further about it, rather than trying to shut them down and talk over them.

"Say my name"

When you're in a discussion like this, pay attention and watch for opportunities to do the same kind of thing yourself. If a co-worker makes a point, acknowledge it, support

it, mention them by name, and if the discussion leads to a fruitful result, show that you remember whose idea it was in the first place.

When you follow up on what someone else has said, tie it back to them by name, saying “To Helen’s point, it might be worth considering...” or “What Nasir said earlier reminded me that...”

When someone does this for you in return, take note of the warm feeling it generates in you: “I’ve been heard!”

Being a good co-worker is your job now

I think we all realise that it’s bad manners to actively shoot down a colleague’s idea or code in a group setting. But ignoring them, erasing them, or even just minimising them is a more subtle, yet no less effective way of making yourself an unpleasant person to work with. The behaviour may not be malicious in intent, but that doesn’t matter: it’s the effect that counts.

In general, you should always be seeking out ways to be kind, friendly, and supportive to your co-workers. *That* is your real job, regardless of what your business card says. And it’s not difficult. All you need to do is be the person that *you’d* want to work with, if you had the choice.

Courtesy is a sign of strength

Don’t expect everyone to respond in the same way, though. Some people won’t notice your kindness, some won’t value it, and some may actively exploit it. This is a shame, but there we are. Treat everybody with impeccable courtesy, no matter how they act in return, and you’ll have the high ground in every situation.

Never be suckered into tit-for-tat rudeness. When two people get up in each other’s faces, and the spittle starts to fly, no one watching will remember “who started it”. They’ll just remember two angry, rude people, and try to avoid both of them.

You can’t *like* everyone you work with: that’s impossible. But you *can* be polite to them, even those who don’t return the favour. *Especially* to those people, indeed. It’s hard to imagine that anyone actually sets out to be a rude jerk, but this just seems to be some people’s default way of interacting. It’s important to show them that it doesn’t work on you.

By being rude to you, they hope to engage you in a who’s-the-biggest-jerk contest, and drag you down to their level. Instead, you can wrong-foot them by being extra nice. That worries people, because they won’t understand what you’re up to.

“Never think that being polite is a sign of weakness,” Chen said as he shook his head. “It is really a sign of great strength. When people are rude, they expose their weaknesses. When we are polite, our enemies cannot see our weaknesses.”

—Phillip Starr, [“The Making of a Butterfly: Traditional Chinese Martial Arts as Taught by Master W. C. Chen”](#)

We'll have more to say about the game of office politics, and how to win it by not playing, in a later chapter. For now, though, let's turn to something that's very likely to be the limiting resource for your effectiveness at work, in your career advancement, and in your own life: *time* itself.

6. Time

Some people live more in twenty years than others do in eighty. It's not the time that matters, it's the person.

—The Tenth Doctor, “[The Lazarus Experiment](#)”



In this chapter we'll discuss some useful ways of thinking about time, some tips for using it intentionally, and how to avoid having life just happen to you while you're busy making other plans.

It starts with taking back control.

Being a Time Lord

How do most of us manage our time? I think it's fair to say that we don't, in any conscious sense. Instead, what tends to happen is something like this: we start the day knowing that there are, let's say, three tasks we really need to get done. Before we get a chance to start on these, though, some emails arrive, and in the process of dealing with them, and other matters arising, stuff happens. All of a sudden, it's time to go to bed. What the heck happened to the day?

This seems unsatisfactory on many levels. First, we didn't achieve what we wanted to. Second, we probably spent a lot of time on tasks that aren't really important to us

(maybe not even to other people). Third, we weren't really in control of what we were doing.

Control your schedule

The First Law of Time is:

If you don't take control of your time, someone else will.

You've probably heard of the "big rocks first" strategy, popularised by Stephen Covey. The idea is that you're trying to fill a jar with rocks, pebbles, and sand (not sure why you'd need to do this, but never mind). If you put the sand and pebbles in first, you'll find there's no room for the big rocks. In other words, if you're going to get the big rocks in there, you need to put them in first, and let other things fill up the spaces around them.

Your day is the jar. The things you really want to do are the big rocks. The stuff that comes at you, unasked and often unwanted, is the sand. If you wait until the jar is almost full of sand, it's hardly surprising that most days you won't be able to fit in a single rock.

Make a plan

For each hour of the working day, write down what you plan to be doing during that hour.

Some of these hours will already be filled up; perhaps you have meetings, lectures, dates, or other appointments that represent commitments to other people, or even yourself. But there will also be blank spaces.

Congratulations: you've just won your first small victory for time management. Thinking about your time in advance is the first step to using it wisely. Now you have the relative luxury of deciding what to do with those blank spaces.

Make time for what really matters

No doubt you have a number of commitments to meet, which will use up some of that time. And some of it should be reserved for yourself, to rest, to regroup, to breathe, to go for a walk, to sit quietly with a cup of coffee, to read a book, watch the birds, or to spend quality time with those you love. In other words, to *live*.

The aim of managing our time more effectively is *not* so that we can cram more work into it. There's no point merely running faster on the hamster wheel of death. Instead, let's make sure we're spending time on what really matters: that is, the things we'll wish we'd done more of when, as they inevitably will, our days come to an end. Today is a gift, as the saying goes; that's why they call it the present.

Things to do: make a list

If you've ever been hiking in difficult terrain, you'll know that it requires two quite separate skills: *vision* and *focus*. Vision, because you need to be able to scan the landscape at long range to see where you're heading. Focus, because you need to look where you're putting your feet.

You can't exercise both vision and focus at the same time, so you need to alternate. First look ahead at the landscape, then down at your feet, and so on.

The same applies to your to-do list. Some people say they don't believe in to-do lists, but the fact is that everybody *has* one. Whether or not you like the idea, you *have* a bunch of things to do. Some people just keep the list of those things in their head, that's all.

And that's a terrible place to keep it. Your brain is a marvellous organ, but it's not designed for retaining lists of arbitrary items and prompting you about each of them at the right time, in the right place, in the right way.

Do you have a flashlight somewhere with dead batteries in it? When does your mind tend to remind you that you need new batteries? When you notice the dead ones! That's not very smart.

If your mind had any innate intelligence, it would remind you about those dead batteries only when you passed live ones in a store. And ones of the right size, to boot.

—David Allen, “[Getting Things Done: The Art of Stress-Free Productivity](#)”

Get everything out of your head

Instead of cluttering up your brain with dumb things like “buy batteries”, give it a good clear-out so that it can really start working. Grab a piece of paper, or a text file, and start jotting down all the things that you know you need to do or remember.

At first this will be easy, then hard, then easy again. What I mean is, there are probably four or five things buzzing angrily around your brain right now like trapped bees, making you anxious in case you forget them. These will come straight away.

Once you've freed those bees, it may seem like there's not a whole lot else you need to remember. But just sit for a minute and let your mind spin gently in neutral. One or two more things will pop into your consciousness; write them down. As one thing occurs to you, it may trigger a few more. Keep going with this process until you *really* can't think of anything else.

You will probably find you've covered one or two sheets of paper with things—perhaps an alarming number of things. More than you could reasonably do in the next month, even if you spent every minute of every day on them. Some of them will be quick and immediate (“buy batteries”), others more vague and ill-defined (“learn time management”), and some long-term and strategic (“have fulfilling career”).

That's okay. It's not a problem. While it may be a shock to see so much stuff written

down, the truth is that your to-do list hasn't changed. It's just that you're *seeing* it for the first time. Up to now, all this stuff has been buzzing round your head, cluttering it up and blocking more interesting or valuable thoughts. And, whether you know it or not, it's also been stressing you out.

Deep work

Many people believe that they can multi-task, but the truth is that's just not something the human brain is good at. If you think you're multi-tasking, you're probably just half-assing a bunch of things at once, instead of really doing any of them. Half-watching something on TV while half-reading something on your phone; half-listening to someone on a Zoom call while half-thinking about how to fix your code.

If you have a normal-sized brain, then half of it, or even less, won't be enough to do good work on any particular task. And what's the point of doing *bad* work? Better to skip that task altogether, and focus on something else.

Schedule deep work deliberately

There's a popular myth that we only use 10% of our brains. It *feels* true to us because we're perfectly well aware that we tend to use our brains in an inefficient and unfocused way.

The truth is simply that your brain is a lot more powerful than you realise. You just haven't really given it a chance to show what it can do. And that means dropping the attempts to multi-task. Instead, focus on just one thing for a short period, and do it *really well*.

This is exactly what you're going to do in your day planning. You're going to start by planning sessions of *deep work*: that is, using your brain at full power to achieve something, free of distractions, interruptions, and irrelevant activities.

Remove all distractions

At first it feels scary to put your phone on "do not disturb" and quit Slack and email. What if you get an urgent message? Well, what if you do? You can't spend your whole life on pause, waiting, just in case someone needs you. And it's usually possible to configure these things so that the people you really care about—your child, spouse, partner, parent—can reach you in an emergency, even when others are temporarily blocked.

The next step to deep work is mental attitude: preparing yourself to focus and work without procrastination. Like yoga or jogging, this is always hardest on the first day. You may need to move some toys out of reach; if you tend to use the internet to give yourself a quick break whenever you feel bored or stuck, use a browser extension to block access to your favourite time-wasting sites.

You can still take screen breaks during a deep work session, and that's a good idea, but don't just switch from your monitor to your phone or tablet. Go and stand outside for a few minutes and listen to the birdsong, or (depending where you live) the traffic.

Don't overdo it

How long should you schedule for deep work sessions? This depends on you, but it's wise to start small. If you're not used to working deeply, block out half an hour to an hour to begin with. That's enough time to achieve a surprising amount, if you're really focused.

However, when you feel yourself starting to flag, don't push it. Listen to the signals from your brain and body telling you that you've done enough for now. If you're really flying, the temptation is always to push on a little more: add one more feature, one more chapter.

Instead, make an intentional decision to stop before you start feeling tired and burned out. When you're making scrambled eggs, if you wait until they *look* done, they'll actually be overdone. The same applies to your brain. Learn to take it off the heat just *before* it turns into an unappetising, rubbery mess.

You should also time the end of the session so that you have a reasonable gap before the next appointment. Don't try to jump straight into something else. After a session of deep work, you'll need a break, and for once you'll have really earned it.

Not all work is deep work

Depending on your job, you may need to spend a greater or lesser amount of your day just doing things like triaging bugs and emails, responding to chat messages, making phone calls, and so on.



This “shallow work” can be important, too. But it doesn’t take much concentration, and it can be done in short time slots, whenever you have a few minutes. When you know in advance that you won’t be able to get long, uninterrupted blocks of time for deep work, use the time you do have to crank through your shallow work queue.

A certain amount of shallow work is part of everyone’s job. What’s going to make the difference between you and the next person is your ability to do *deep* work.

Some people never do deep work at all. It should be relatively easy to outperform them. Then you can concentrate on outperforming the people who are also doing deep work, but just aren’t as good at it as you are.

Your ability to do this work is completely dependent on the prevailing interrupt rate. If each interrupt robs you of fifteen minutes of flow, and you get interrupts every fifteen minutes, then you should expect to achieve absolutely nothing of value.

Sadly, that’s the case for many people. To avoid this, reduce or eliminate interrupts.

Make yourself un interruptable

Should we simply hope that people will realise we're busy and not interrupt us? Good luck. A better idea is to take control of your interruptability, and set boundaries for access to your attention.

That's actually something you're allowed to do. Everyone does not have the right to interrupt you and claim your attention.

The first line of defence against these interruptions and attempts to claim your attention is, as we've seen, to make yourself physically un interruptable by turning off the various communication channels through which the attacks will come: Slack, Teams, phone, email, and so on. Not forever; just for an hour or two during your scheduled deep work session.

If you've inadvertently trained your co-workers to expect instant access to you at every hour of day or night, and you probably have, it may surprise them at first to find that you've gone dark.

Where this is the case, have a conversation with them about it and explain what you're trying to do. The odds are, they'll be fine with it once they know what's going on.

People will usually respect your right to become inaccessible if these periods are well defined and well advertised, and outside these stretches, you're once again easy to find.

—Cal Newport, “[Deep Work: Rules for Focused Success in a Distracted World](#)”

Attention is a muscle: flex it

Your ability to do deep work is critically dependent on your ability to sustain your attention for long periods. Unfortunately, almost everything in modern society seems designed to shorten and weaken our attention spans: tweets, TikTok videos, even text messages.

If you never use your muscles, they'll get weaker. If you never have to sustain your attention, it will get weaker. If you find, whenever you try to do some cognitive work such as writing or coding, that your concentration fades after a few minutes and you start craving breaks and distractions, then you know what I mean.

But the good news is that even weak muscles can get stronger, if you train them. You can train your attention by using it. Read long books; spend some time in quiet meditation; watch a whole movie from start to finish without checking your phone. This will be very difficult at first, but you can start small and build up to more challenging attention workouts over time.

Quit social media. That sounds blunt, but it's an essential step if you want to become better at using your brain for a living. Social media is not only a waste of your *time*—it's just a great babble of other people's worthless opinions—but it positively harms your ability to concentrate, too.

It's also extremely habit-forming, and that's not a coincidence. Social media companies maximise their profits (or minimise their losses) by using every trick in the book to grab and hold your attention. They are *mining* you for your attention, and this ruthless extraction has roughly the same effect on your brain as strip-mining has on the landscape.

*Once your brain has become accustomed to on-demand distraction, it's hard to shake the addiction even when you **want** to concentrate. If every moment of potential boredom in your life is relieved with a quick glance at your smartphone, then your brain has likely been rewired to a point where it's not ready for deep work.*

—Cal Newport, “[Deep Work: Rules for Focused Success in a Distracted World](#)”

Beware of the long-term consequences of carrying an always-on distraction machine in your pocket. The smartphone makes a good servant, but a bad master. Consider having a more intentional relationship with it.

Protect your time

Let's assume that you've completed a productive and focused session of brainwork on whatever it is you do best, and now you've reopened your comms channels to find a queue of requests awaiting you. Fulfilling them all would take more time than you have. What should you do?

I'm sure you've noticed that if you ever actually *complete* all the things that people want you to do, it just seems to make room for even more demands on you.

You don't want to spend your life just barely keeping ahead of your to-do list, so what can you do? You can learn how to say no.

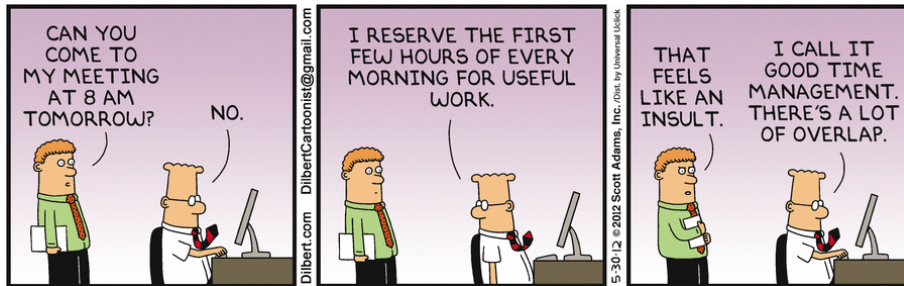
Sometimes helping means not helping

What requests can we *not* refuse? Clearly, it would be a bad idea to say “no” to your boss. The same goes for spouses and partners. And there will be requests to do things that we genuinely *want* to do, and that are important and valuable. A considered “yes” to these is fine.

But it's absolutely okay to say “no” to almost everything else. Some people have a bad habit of asking for help with trivial things instead of thinking for themselves. If you get a question from someone that they could have easily answered themselves with a little research, or even just a little mental effort, you don't owe them an answer.

Indeed, by answering their question, you're not really helping them. It might feel that way, but the truth is that you're just further reinforcing their behaviour. You're training them to reach out for help every time they're momentarily puzzled about something. If you give it, they'll never develop the mental muscles necessary to solve the problem for themselves.

Skip useless meetings



Why do people request meetings with us? In many cases, the meeting may just be a regular fixture on the calendar, and no one really knows why or can remember why it was scheduled in the first place. If these meetings really deliver value for all those who attend, great. But if they don't, don't attend them.

Deliberate non-attendance of a meeting, especially a work meeting, is a bit of a power move. You may want to wield it cautiously, especially when you're new to the organisation. But the reality is, especially with regular status meetings and other pointless schedule-fillers, most likely no one will notice that you're not there. And that just goes to prove that you *didn't* need to be there.

If someone does notice, such as your boss, and asks you why you didn't attend, you can just say that you had some urgent work to deal with. If you can point to something valuable that you achieved while everyone else was wasting time in a dumb meeting, that will certainly help. Indeed, it might make others reflect on whether the meetings are making the best use of their own time.

Almost nothing worthwhile has ever been achieved in a meeting. By contrast, some very worthwhile things often happen in a *conversation*, which is a very different thing, especially if all the parties involved are there because they want to be. Conversations tend to be spontaneous, voluntary, and self-organising, which is the exact opposite of a meeting.

Decline most invitations

The best way to avoid a meeting, or any other kind of synchronous discussion, is to say no to it in the first place. Decline the invitation, or delete the appointment from your schedule. If someone asks you directly for an appointment, apologise and say your schedule is full. A nice way to phrase this is something like:

I appreciate the invitation, but I'm afraid I can't make the time for this at the moment, due to pressure of other work. Thank you for thinking of me, and I'll be sure to reach out to you if anything changes.

Nine times out of ten the reason people ask for a meeting with you is because they need answers to some questions. A much more efficient way to get them would be to simply

send you the questions; then you could answer them at a convenient time, without having to rearrange your day. So asking for a meeting is usually a sign that the person doesn't really *know* what questions they want to ask. They're hoping you'll tell them that, too.

Pushing back on the request will force them to be clear and explicit about what they want, and it usually isn't really a meeting.

Unfortunately my schedule this week doesn't allow for us to have a face-to-face chat, but I'll be happy to answer any questions you have by email, if that works for you?

Most likely you'll never hear from them again, which is fine, but if they really do have something concrete to ask you, they'll let you know and you can answer it in a much more efficient way than by having a meeting.

Sometimes people will want a conversation with you that can't really be replaced by an email exchange. For example, planning, brainstorming, exchanging and refining ideas: these things are most effective in person. And if it's about something that's important to you, do it. Much more often, though, it's something that we don't really care about or want to do, but we just feel it would be too rude to say no.

Saying “no” nicely

They say tact is the ability to make a point without making an enemy, and saying no is a bit like this, too. Declining someone's request without being rude or making them feel bad is a superpower, and it's one that even ordinary humans like you and I can cultivate. It gets easier with practice, too.

Even if what you're really thinking is an unprintable epithet, you don't have to *say* it that way. You can let them down gently, with face saved on all sides:

Thank you so much for your request; I really appreciate you thinking of me. Unfortunately, pressure of time means that I'm not able to accept on this occasion. I wish you all the best, and I look forward to being able to meet with you at some point soon.

How nice is that? It's almost more gratifying than a “yes”. Some variation on this formula will deal with almost all the time-sucking requests you receive in a way that preserves goodwill and good relations with the requesters.

“Not yet” is almost always fine

When you really can't say “no”, something that's almost as effective is to say “not yet”. If someone sneaks through your comms defences and asks for your help, it's perfectly fine to say “I'm on a deadline right now, can I get back to you in an hour / after lunch / next Tuesday?” (delete as appropriate)

Identifying a specific response time is the key here. It means the other person can get on with something else while they're waiting, instead of spinning their wheels uselessly,

because they know how much time they have to play with.

If you'd just said "I'll get back to you later", on the other hand, that leaves them in an awkward limbo, unable to proceed until they know whether you'll actually be able to help. So commit to responding at a specific time, and stick to it.

Unless you're an on-call trauma surgeon, or in some equally time-sensitive line of work, postponing interruptions like this is absolutely acceptable, and makes a big difference to your effectiveness.

The postponement should be for as long as you think you can reasonably get away with. When you ask "Can I get back to you tomorrow?", nine times out of ten the answer will be "Sure, that's fine." In the rare case where the request is really urgent, though, they'll let you know.

We love to say "yes", and it can be linked to our self-esteem. "I'm the kind of flexible, can-do person who can cope with any number of last-minute tasks," we like to think, but where does that end? The more you say yes, the more people will ask of you. At some point, you'll collapse under the weight of expectations and burn out. I don't recommend that.

Instead, be firm about your commitments, but courteous; definite, but realistic; accommodating, but assertive. Never take on more than you *know* you can achieve, and you'll set yourself up for success.

Respect other people's time, too

Protecting and making effective use of your own time is one component of good time management. Another is having just as much respect for *other* people's time, too. If you don't observe other people's boundaries, why should they observe yours?

Don't say "hi", just ask the question

Naturally, that means not requesting time-wasting meetings, or lazily pinging your co-workers with questions that you could have answered yourself. In particular, **don't say "hi"**. When you need to ask a question asynchronously, such as in Slack or another chat app, just ask it.

Don't, in other words, open the conversation with something like "Hey, do you have time to answer a quick question?" and then wait for them to reply before you actually ask it. We all know a person who does this. No doubt they mean to be polite, but they're actually being annoying.

They're trying to turn an asynchronous interaction into a synchronous one, and that's the key mistake. We have asynchronous protocols for a reason. Waiting for the other person to synchronize with you before asking your question is unnecessary, and a waste of time for both parties. You could have got an answer sooner, and no less politely, by just asking the question right away.

Be punctual to be polite

Another way to respect people's time is to be *on time*. *L'exactitude est la politesse des rois*—punctuality is the politeness of kings, the saying goes. To turn that around, unpunctuality is a form of rudeness. If it's never struck you that way before, then you've probably been rude to people without realising it.

By being late in attending an appointment, or in delivering on a commitment, you're essentially saying to the other person "Your time isn't as important as mine. I don't respect you, and I don't place much value on our relationship."

Now, that may well be how you really feel. The question is whether that's the message you want to *send* them. Being deliberately rude is occasionally the best way to deal with something, though it's a weapon you may want to weigh carefully before deploying. But being *inadvertently* rude, especially by being late, is never in your interest.

Five minutes early is on time, tai-pan.

—James Clavell, "Noble House"

Being a little ahead of time has advantages from the point of view of gamesmanship, too. You can scope out the location, and make a note of the nearest exits (if you're a spy, or just easily bored). You can catch your breath, and get a coffee. When your counterpart arrives, you'll be relaxed and ready to get started, and you'll be sitting in the best seat.

If you're chronically unpunctual, it's worth examining what lies at the root of that problem. Is it that you're not keeping track of your commitments? Store them in some reliable system that's outside your head and can send you reminders, such as a calendar or to-do manager. Don't rely on your brain reminding you when it's time to do the thing. It won't. That's not what brains are good at.

Don't renege, renegotiate

We often find ourselves in a situation where we've inadvertently committed to an unrealistic or unachievable deadline. Maybe the person requesting it has no idea of the length or difficulty of the task involved. If it's your boss, this is probably the case.

Bad bosses assume that tasks they don't understand are easy, so they won't take very long. *Really* bad bosses plan tasks using [Scotty time](#). You know how that goes:

HARRIED SUBORDINATE: *I can have it running in 20 minutes.*

BOSS: *You've got five!*

Don't be tempted to pad your estimates in advance when you're dealing with the captain of an enterprise like this. Padding simply leads to a runaway inflation of estimates and deadlines that doesn't help anyone.

When presented with an impossible deadline, it's tempting to just cave in and accept it, avoiding a potentially tough conversation. And maybe you want to impress, by achieving the task in record time, but that's unlikely.

In situations like this, it's best to avoid giving a firm time commitment at all. The moment you *do* commit to a specific deadline, people will structure everything around that, and bad things will happen if you don't meet it.

Better to say "I'll have it done as soon as possible", or even, "I'll give this my most urgent attention", which sounds very energetic, but actually makes no specific commitment at all.

BERNARD: *An official reply just says "The Minister has asked me to thank you for your letter", then we say something like "The matter is under consideration", or even, if we feel so inclined, "under active consideration."*

HACKER: *What's the difference?*

BERNARD: *Well, "under consideration" means we've lost the file. "Under active consideration" means we're trying to find it.*

—*"Yes, Minister"*

But suppose you've been incautious enough to accept a commitment, and now you realise that you can't possibly deliver it on time. What should you do? Just hunker down and hope no one notices?

Bad idea. People will get mad at you if you're late, but if you just don't show up at all, they'll be *really* mad, and with some justification.

Instead, it's always okay to renegotiate the agreement when circumstances change.

Suppose I'd told you I would meet you Thursday at 4:00 P.M., but after I made the appointment, my world changed. Now, given my new priorities, I decide I'm not going to meet you Thursday at four. But instead of simply not showing up, what had I better do, to maintain the integrity of the relationship? Correct—call and change the agreement. A renegotiated agreement is not a broken one.

—David Allen, *"Getting Things Done: The Art of Stress-Free Productivity"*

If you can become a master of time, you'll have everything you need to become a master of your chosen profession. And that's the subject of our next chapter: *mastery*.

7. Mastery

Life is like playing a violin in public and learning the instrument as one goes on.

—Samuel Butler, speech at the Somerville Club, 1895



So far in this book we've talked about how to find a job, how to pass the interview, and how to make the right impression in your new role. That's great, and if you've already achieved these stages in your own career, you're doing really well. Congratulations!

Sadly, though, that's where most people's attention to their careers just... stops. Everything else, they think, will sort of happen by itself? Maybe?

Choosing your path

It might be smart, though, to think about where you'll go next. In the first chapter of this book, we discussed three different kinds of career destinations that you might envisage for yourself: seniority, management, and independence.

You might experience all three of these during your career, or just one or two of them. It seems reasonable to assume, though, that you'll need at least *some* degree of seniority—that is, mastery—to be eligible for either management or independence. So let's talk about mastery and how to achieve it.

Suppose you've been in your job for six months or a year, and you're doing fine, where "fine" means you're meeting expectations, you've got through the scary max- q point,

and things are generally going okay. What now?

Should you stay or should you go?

Should you aim to find a better job at another company, or to be promoted to a higher grade at *this* company? Or is it wiser to build your skills and career capital right where you are?

It partly depends on what you want. If you want more money, which most people do, your best chance of getting it might not be to stay with your current company. Sure, you'll *probably* get some kind of raise at the end of each year, but it's unlikely to be much. Indeed, the size of the increment may have very little to do with your performance, and a lot more to do with the prevailing economic conditions and the company's profitability (or lack of it).

On the other hand, provided that you're at least earning enough money to make the rent and feed your family, it might make more sense right now to focus on building your long-term career prospects. And that simply means getting better at what you do.

A master engineer is employable anywhere, and can make a better than decent living in all but the most hideous economic circumstances. With mastery comes security, financial and otherwise. Mastery is both rare and hard to acquire. You can't become a master of anything worthwhile overnight, so once you're *at* that level, there are fewer people who can possibly compete with you.

The culture of the company you're in is also important. If it's good, and one sign of that is a focus on nourishing and developing engineers, then by all means stay and benefit from it as long as you can. But if it's bad, then don't expect it to change: company cultures rarely do.

You'll need to supply your own challenges

If you stay where you are, doing what you're doing, the value of the experience you gain will decrease over time, as what you're learning becomes less relevant. As we saw in an earlier chapter, it's important to avoid the trap of becoming an expert in an obsolete technology.

There's also the question of what you can learn if you do nothing more than what's asked of you. *Something*, for sure, but unless you're in a very unusual job, you won't be constantly required to learn and master new things. While any new role is demanding at first, after six months to a year you should be feeling as though you've more or less got the hang of it.

New challenges will come along once in a while, but the more experienced you are, the less likely you are to find yourself seriously stretched by anything you're asked to take on at work. If there's to *be* any stretching, then, you'll probably need to do it yourself.

Broadening your mind

Let's take your programming skills, to start with. What languages do you know? How competent are you with each of them? How much commercial experience using each language would you be able to demonstrate to a future employer?

A deep knowledge of one or two well-chosen languages or technology stacks makes you more employable than a passing acquaintance with a dozen. To avoid the fate of the non-avian dinosaurs, though, you shouldn't put all your eggs in one nest. The language that's super hot today might well be forgotten in ten years time, and you wouldn't want to have to start again on the bottom rung with something new.

Being T-shaped

Instead, valuable employees are what's delightfully called *T-shaped*. In other words, they're both specialists *and* generalists. Yes, they've mastered one particular trade, but they can also get along fine in a bunch of others, though not at the same level of expertise.

At the start of your career, then, you should try to get exposure to as many new languages or technologies as possible. Any experience at all is valuable at this stage, and you'll also learn what suits your own style of thinking and working. Gaining some elementary skills in a wide variety of languages will serve you well throughout your career, and it will also give you plenty of options for choosing your main specialty.

"Wonderful," I hear you say, "but which language should I choose to specialise in, then?"

Unfortunately, I can't answer that for you. But I can give you some potentially useful ways to narrow down the available options.

Try as many languages as you can

Eventually, you'll find one language or framework that seems to make more *sense* to you than others do, and that fits the way your particular brain thinks about problems. Expressing yourself in this language feels easy and natural, and you'll become quite good at it quickly, and without much apparent effort. Given the option, you'll choose this language for tasks without hesitation.

If it hasn't happened yet, don't despair. It might take a while, and this kind of lightning might even strike you fairly late in your career. Sooner or later, though, if programming is your thing, you *will* know which language you want to become a master of. In the meantime, the best thing you can do is to get at least a little programming experience in as many different languages, and different *kinds* of languages, as possible.

For example, if you already know C++, try learning Clojure instead. If you know JavaScript, explore Go. If you're a decent Python programmer, spend a weekend diving into Rust. You get the idea. In fact, even if you've already found your one true love language,

you should still keep learning new ones. It broadens the mind, and gives you new ways to think about problems.

Finding your love language

When looking for something to specialise in, you should choose a language that is commercially valuable. There's not much point being the world's foremost expert in a language that no one uses. I mean, that could be a whole lot of fun as an intellectual pursuit, and I yield to no one in my passionate enjoyment of learning obscure but interesting things.

From a *career* point of view, though, you should keep at least one eye on whether people will be likely to want to pay you for exercising this skill. You might think that the amount of time and effort it takes to acquire a skill is directly correlated with how much you can earn from it. And you would be one hundred percent wrong about that; ask any musician.

That said, mastery involves both head *and* heart. You need to love the thing you're going to spend the rest of your life doing, and that doesn't usually happen just by picking something from a list. Instead, the thing will find *you*.

If you don't love writing software, it's a lousy choice of career. If you do, it hardly seems like work at all.

*“Once I had written my first computer program in high school, then all other things in the world seemed a lot less interesting,” Toy recalls. “There was something really rewarding about the process... when I got out of college, it felt like the fools were **paying me money** to do what I would have done for free if only they had known.” He pauses, deadpan. “That still continues today.” —Michael Toy, quoted in Scott Rosenberg’s “[Dreaming in](#)

[Code](#)”

Walking the path

Here are some practical steps you can take today that will get you closer to becoming a master of your chosen craft, even if you haven't quite pinned down the details of *what* you want to master yet.

Be prepared to train hard

If you're to become a master of your craft in a reasonable time, you'll need to take control of your learning, too. As we saw in the first chapter, waiting for the company to train you is a strategy for stagnation. Instead, decide for yourself what you want to learn, and learn it.

That means spending significant time on your craft practice *outside* work, and that's where a lot of people start to lose interest.

“Today, people don’t like to train so hard,” Chen said calmly. “Everyone has other things to do. They have time for movies, for golf, for watching television... but no time for training.”

—Phillip Starr, “[The Making of a Butterfly: Traditional Chinese Martial Arts as Taught by Master W. C. Chen](#)”

Maybe you feel like you don’t have a lot of spare time outside work, and I get that completely. We all have many calls upon our time, including family and other non-negotiable commitments. But, beyond those, you probably *do* have time for watching television and so on.

It’s just that you have *already* decided how to spend all your discretionary time, and it doesn’t include work on solo software projects, for example, or studying craft skills. That’s okay. The important thing to recognise is that you *have* a choice. Now you can re-evaluate it in an informed way.

Write every kind of program imaginable

Just as you should get exposure to as many different programming languages as possible, you should also try to work on as many different kinds of *programs* as you can. As we’ve seen, most business applications are pretty samey and boring (even if they do pay well). That’s not the same as “easy”, for sure, but you’ll never become a master if you only work on one or two kinds of problems.

Write command-line tools, write API servers, write HTTP middleware. Write browser extensions and microservices. Write web scrapers, 3D games, and machine learning tools. Write firmware for microcontrollers. Write high-frequency trading tools, Slack bots, and physics simulations.

Write your own operating system, implement device drivers for it, and give it a text editor. Invent a language and write a compiler for it. Don’t wait for someone to pay you to do all these things: they won’t. Instead, do it because it’s awesome.

Programming is fun, and you should have fun. After all, isn’t that why you got into this business in the first place?

If you leave your love of programming and the feeling of the computer revolution romance on autopilot, the industry will burn through you. It’s your responsibility to know exactly why you’re doing the specific job you’re doing, what your long-term goal is, how to develop a credible plan to achieve it, and what you need to do (and not to do) to keep your underground spring of creativity alive.

—Slava Akhmechet, “[Surviving Disillusionment](#)”

Learning how to learn

Another great reason to keep learning new things throughout your career is that learning *itself* is a valuable skill. In the fast-changing tech industry, the difference between success and failure in a given job is often your ability to absorb information, pick up

new skills, and be productive with them quickly. Being smart and determined helps a lot, of course, but *effective* learning is mostly a matter of skill and practice. You need to learn *how* to learn.

The most important skill for any engineer to possess is the ability to learn quickly.

—Cindy Sridharan, “[Know How Your Org Works](#)”

Work on solo projects

Commercial developers are rarely involved in the *design* of the systems they work on. Usually, that gets done at a higher level, and eventually, smaller, more specific tasks filter down to the development teams: add this feature, fix this bug, update this dependency.

There’s nothing wrong with that: it’s honest work, and you can take pride in doing it to the highest possible standard of quality. But if that’s all you ever work on, how will you ever learn to *make* those higher-level decisions? You won’t.

By contrast, when you work on your own project, you get to be the product manager. You can design a program from scratch to solve user problems. You decide what features go in, and what stays out. You get to be the system architect: you choose how to implement the program’s behaviour and how to structure it into packages and modules.

You are the API designer, the chief test engineer, the documentation writer, the lead maintainer, the release manager, the head of operations, and the principal data scientist. If you wait for your *company* to rotate you through all these roles, you’ll be waiting a long time. Instead, you can start learning them now, and no one can stop you.

Where does mastery come from?

Taking the *decision* to master your craft is the critical first step, and you can’t take it too soon. But clearly that’s not enough on its own: you need to start putting work in, and get serious about your learning. What should you learn and who or where should you learn it from? Let’s talk about some of the options.

Do you need a college degree?

A degree is a lovely thing to *have*, and will undoubtedly open doors that would otherwise remain closed to you, if only because employers need to filter on *something*. But it usually costs a lot of money to go to college, and then there’s the opportunity cost of what you might have been earning while you were studying instead.

A computer science degree is often regarded as a ticket to a good job in the software industry, and it can be, but the actual *knowledge* you acquire in that degree program probably won’t be that helpful for your day-to-day work. Computer science is not really about computers, as Dijkstra observed, any more than astronomy is about telescopes.

There's more to learn at college than just CS 101, of course, and if you have the means and the opportunity to go there, I strongly recommend it. If you don't, or if you just don't feel like it's your thing, no big deal. If you're smart, you'll be able to teach yourself just about everything you need to know. If you're *not* smart, college won't help with that.

Should you buy software engineering books and courses?

You can learn useful things from books, to be sure, but you won't become a master unless you also spend a lot of time *applying* that knowledge to practical projects. And once you've completed a few projects, you'll soon develop your own ideas that won't agree with what's in the books, and that's okay. Better than okay, it's essential. And you'll also find, confusingly, that the books don't always agree with each *other*.

When I got my first computer as a kid, there was no internet. If you wanted to look up some information, you had to go to the public library, but they wouldn't have a book on programming anyway. Fortunately, my computer came with a truly excellent manual, and it said this:

Whatever else you do, keep using the computer. If you have the question "what does it do if I tell it such and such?" then the answer is easy: type it in and see. Whenever the manual tells you to type something in, always ask yourself, "what could I type instead?"; and try out your replies. The more of your own programs you write, the better you will understand the computer.

—Steven Vickers, "ZX81 BASIC Programming"

I've followed that advice ever since, and it's served me pretty well. To this day, when it occurs to me to wonder "How does such-and-such work in Go?", I'll just type it in and see. I've upgraded the *computer* once or twice in the meantime, though.

If you want to learn by reading, as well as by doing, you should read *programs*, not books. Reading other people's code is the most effective way to learn to write your own. It's astonishing to me that this isn't a standard part of software engineering education, but here we are.

"What do people do when they're being trained, for example, to get a master of fine arts in poetry? They study great works of poetry. Do we do that in our software engineering disciplines? No. You don't look at the source code for great pieces of software. So you don't study the literature of the thing you're trying to build."

—Richard Gabriel, quoted in Scott Rosenberg's "[Dreaming in Code](#)"

And we live in a world where almost all the great programs that have ever been written are available for us to read and study, for free. If you like, you can read the software for the [Apollo Guidance Computer](#), or the [Unix kernel](#), or the [world's first web browser](#). There's a lifetime's worth of education to be had, and all it'll cost you is a little effort.

Save your money for professional coaching

Something you absolutely should be prepared to invest money in, though, is a *coach*. It's interesting that we take it for granted that we'll need a coach when we want to learn, say, the piano. And people will spend eye-watering amounts of money on professional coaching for things like tennis or golf.

Yet very few people even consider hiring a software engineering coach. Why is that?

Elite performers, researchers say, must engage in “deliberate practice”—sustained, mindful efforts to develop the full range of abilities that success requires. You have to work at what you’re not good at.

In theory, people can do this themselves. But most people do not know where to start or how to proceed. The coach provides the outside eyes and ears, and makes you aware of where you’re falling short.

—Atul Gawande, “[Personal Best](#)”

In other words, we don't know what we don't know, and that limits what we can achieve by self-teaching. A coach can help you reach your desired level much more quickly than you could on your own. Also, with their support and guidance, you can tackle much more difficult projects. And, just as lifting heavy things is the only way to get stronger, solving hard problems is the only way to become a master engineer.

Good teachers are surprisingly good. I had a masterclass where within thirty minutes I figured out things I couldn't figure out before in 10 lessons or more. To this day the few lessons I had with that particular teacher resonate with how I play.

—Elad Bogomolny, “[Choosing a Music Teacher](#)”

But while a teacher can help you *use* your time much more effectively, they can't do the work for you. The Chinese word *gōngfū* (“kung fu”, we might say, mangling Mandarin pronunciation a little) refers to craft mastery in general, and martial arts in particular, but it also combines the concepts of both *effort* and *time*.

A coach can hugely accelerate your learning

You probably wouldn't try to learn karate on your own, or at least not without some excellent health insurance. While the risk of injury may be slightly less, it's worth considering getting a coach for software engineering, too.

If you're taking your career seriously enough to invest in professional training for it, that already puts you ahead of 99% of the competition. It tells employers that you're the kind of person who is likely to achieve great things. And paying that coaching bill every month reminds *you* how important this work is, for yourself, your family, and your future.

I can personally vouch for the effectiveness of coaching, since that's what I do when I'm not writing books. I don't turn bad engineers into good engineers; a coach can't do that.

I help turn good engineers into *great* engineers, and I say “help” because they really do most of the work themselves.

What’s my contribution to the process, then? Well, I can provide support, encouragement, motivation, feedback, and advice, all of which is valuable. But more importantly, I can guide my students in a way that maximises the effectiveness of the limited time they have available for learning.

For example, I can help them choose projects that are difficult enough to stretch them, but not so hard that they’ll get demotivated and quit. I can steer them away from things they *don’t* need to worry about, and towards the questions that really *do* matter.

I can answer questions that would otherwise lead them down blind alleys of bad internet advice and obsolete blog posts. I can get the student un-stuck when they don’t know how to move forward. And I can share with them the fruits of many decades of experience, not just about engineering, but also about building a *career* out of engineering. You’re reading some of them now.

Potential mentors are all around you

Okay, end of commercial. While it’s great to have a professional personal trainer giving you one-to-one lessons on your own schedule, not everybody needs or can afford that level of coaching all the time. There are other ways to get mentoring and learning support, and many of them are free.

Your colleagues, for example, are an amazing learning resource if you know how to make the most of them. Early on in your career, you’ll probably feel like *everybody* you work with knows more than you do, and perhaps that’s the case. But it’s good news, because it means you can learn from them.

You can, and should, ask for help and advice from your team-mates all the time; that’s what they’re for. The best way to *get* help from someone is to show them that you’ve made a serious effort to solve the problem yourself, and you’re not just asking them to do your homework for you.

Keep an eye out for people who are really experienced in your language or field, and talk to them. If you build a good relationship with them, they may offer to give you some mentoring or other assistance in their free time.

It doesn’t have to be a one-to-one relationship; why not start a “code club” among your co-workers, for example? Every week you can get together, physically or virtually, and discuss an interesting programming book, or write some code together.

You could *read* code together and discuss it, which is something all software engineers should do more. This kind of group learning is very valuable, and it can also be a lot of fun. Don’t wait for the company to organise it; just go ahead and arrange it yourself, informally.

You can mentor others without being an expert

As soon as you've learnt a little about the subject yourself, you can start mentoring others, too. You might feel like you have to wait until you're a highly senior expert before you're qualified to be a mentor, but that's not true.

Mentoring really just means "helping", and if you think of it that way, you'll see that of course there are things you can do to help your co-workers learn.

It doesn't mean "lecturing", though, and no one appreciates a back-seat driver who keeps pointing out all their mistakes and explaining what they should have done instead. So don't help someone unless they *ask* for help.

If they didn't hire you, don't solve their problem.

—Jerry Weinberg, "[Secrets of Consulting](#)"

But when they do ask, be generous with your time and effort. What goes around comes around, and mentoring others not only helps you become a better craft master yourself, it makes you a more valuable employee, too.

Pair programming

The best way to transfer knowledge back and forth between people is through *pair programming*, where two of you (or maybe more) work on the same piece of code at the same time. This is a subject that tends to divide opinion: some people love pairing, and think it's the only effective way to develop software. Others hate it, and think it's a waste of time.

Good pairing is about trust

The fact is, as with anything, there's a right way and a wrong way to pair program. It's also a skill that needs to be learned and cultivated: no one is good at it until they've had some practice. And, since everyone is different, it takes time to learn how to pair well with each different partner. Even more importantly, it's a matter of *trust*.

Pairing is a surprisingly intimate process. It means exposing your vulnerabilities and your unfiltered thoughts, your weaknesses and your uncertainties. It's almost like a Vulcan mind meld.

SPOCK: *You begin to feel a strange euphoria. Your body floats.*

GELDER: *Yes, I begin to feel it.*

SPOCK: *Open your mind. We move together. Our minds sharing the same thoughts.*

—Star Trek, "[Dagger of the Mind](#)"

When you first learn to pair program, it's best to stick to doing it with just one other person. Working in a larger group (known as *mobbing*) is possible, but it's a more advanced

technique, and it requires that all the participants have some practice with pairing beforehand.

How pairing works in practice

If you're pairing in person, you should sit next to each other looking at the same screen. When you're pairing remotely, use the screen-sharing facility of your call software. One of you will be the *driver*, the other the *navigator*, and you should switch back and forth between these roles fairly frequently.

The driver does the typing and mousing, while the navigator keeps an eye on the bigger picture, catches errors, helps the driver stay focused, runs interference on any potential distractions, and makes sure that both partners have a shared understanding of what they're doing. You could think of pairing as a kind of very focused conversation that produces code as its output.

For example, this could be an excerpt from a typical pairing session:

DRIVER: *So we need to add a method here...* (TYPES)

NAV: *Actually, maybe it should be a function instead, because it doesn't need to access any information on the object.*

DRIVER: *Right, right.* (TYPES) *And it needs to take...*

NAV: *It takes the user ID as a string...* (DRIVER TYPES) *...and returns a User struct, maybe?*

DRIVER: *What about a pointer to User?*

NAV: *Yes, that's better, because User has pointer methods.*

You get the gist. Even though the driver is doing the *typing*, they're not in charge: it's a joint venture. You're both doing the thinking together, with two temporarily conjoined brains. Both of you need to fully understand the meaning and purpose of what the driver is typing; as soon as one partner loses the thread, pairing is no longer happening.

Pairing problems, and how to fix them

Problems to watch for when pairing, then, include:

- *Loss of sync.* If the navigator gets distracted by their phone, or starts chatting about a movie they saw, or just doesn't understand what the driver is doing, then they're no longer helping the driver, only hindering them. A good rule for both partners is "Don't type when they're talking. Don't talk when they're typing."
- *Silence.* When neither of you are talking for an extended period, something's wrong. If you're silent because you're thinking, think out loud so that your partner can join in. If it's because you don't understand what's happening, say so. Your partner *must* stop and explain, and not resume until you've confirmed you understand and agree this is the right thing to do.

- *Arguing.* There will be differences of opinion about what *is* the right thing to do, and that's okay. When the partners trust and respect each other, these disagreements are usually resolved quickly. Nothing is final, after all. Try it one way and see how it looks, and if that's no good, try it the other way. If the discussion looks like being a long one, just capture the issue to deal with later, and move on to something else.
- *Sullen acquiescence.* Sometimes one of you isn't convinced, but rather than keep pushing back, you just lapse into a sulky silence, or say something like "Whatever, dude, I don't care." Or even "You know best," which is a passive-aggressive way of saying that you don't agree, but you're also bored with arguing about it. Don't do this: instead, stop and resolve the issue or misunderstanding before trying to press on.
- *Person overboard.* When one partner doesn't quite understand what's happening, they may hesitate to say so right away. You need to feel quite safe with someone before you can say "I don't understand this," but the more you both say it to each other, the easier it will become. If you don't, you'll just drift further and further behind in the other's wake, until it's too late for them to circle back and pick you up.
- *Dictating.* Although pairing can work well even between partners of very different skill levels, there's a danger in this case that it can turn into one person simply dictating what the other should type. Instead, let your partner think and work at their own speed, even if it's a bit slower than yours. Don't jump in to correct them straight away. Let them type something out, look at it, think about it, *then* correct it. It's very irritating when someone keeps picking you up on the typos you're *just about* to notice.

Pairing distributes knowledge around the team

You'll not only learn a lot about each other by pairing, you'll also learn a lot *from* each other. Everybody knows something useful that you don't know, and the knowledge transfer is almost always bidirectional. If you're the most junior person on the team, try to arrange to pair regularly with the most senior, and so on down. That way, everybody levels up together.

You'd think that everyone would be in favour of this idea, but that's not always the case:

I once had a team member who got visibly upset after I facilitated our first mob programming session. He stood up and said: "Look, I've spent many years working hard to get to where I am today, and I sure as hell am not going to share my knowledge here for free!"

—Alex Bunardzic

Another nice thing about pairing is that it tends to produce better code than any individual could write on their own. Sure, it ties up two developers on the same job, but if

the result is more than twice as good as a single developer could write in the time—and it often is—that’s a win.

Don’t try to pair all the time

Pair programming is *intense*, because it’s a kind of two-person deep work, and deep work is always intense, but also highly effective. You shouldn’t expect to be able to pair program continuously for eight hours a day; nobody can do that. Instead, just like solo deep work, aim for short but focused sprints on a very specific task.

Between 60 and 90 minutes is probably about the right pairing session length for most people. It’s hard to get much serious work done in under an hour, and after 90 minutes of pairing you’ll both need a break before your brains overheat and shut down. It’s fine to resume again later.

It’s always a surprise to me when companies don’t routinely encourage and expect developers to pair on most of their tasks. It’s so effective when it’s done well that I think it should be the default mode of development. But because it’s easy to do badly, and because *bad* pairing is such a miserable experience, I quite understand why some people are sceptical about it.

Code review

The best software companies *do* use pair programming as their standard workflow, and some have a requirement that all mission-critical code needs to be written this way. Not everything is mission-critical, of course, and there are some tasks that are really better done on your own, because they just involve a lot of thinking. And even when code is written as a pair, it should still be *reviewed* by someone else.

*If the programmers like each other, they play a game called “pair programming”.
And if not, then the game is called “peer review”.*

—Anna Nachesa

Code review, then, is a necessary part of software development, but it’s one that can cause a great deal of friction, anxiety, conflict, and wasted time and effort, when it’s done poorly. So let’s talk about how to do it *right*.

The life and times of a changeset

Usually the workflow is something like this: Aisha writes some code (or Aisha and Bryan write some code as a pair) and it gets checked into version control, usually on a *branch*. Aisha can now make a *pull request*, also known as a *merge request*, which, when approved by Chiku, would merge all her changes—her *changeset*—with the main line.

Alternatively, some teams push changes directly to the main line, sometimes also periodically pulling from it into a *release branch* to build versioned releases, if necessary.

Either way, there's usually a step where the changeset has to be *approved*, maybe by someone more senior, but often just by someone else on the team.

As part of this approval, Chiku should review the code, where “review” means at least “read”, but most likely also to make comments or to ask questions about it. A conversation then ensues between Aisha, Bryan, and Chiku, and usually a few further changes are made, before the merge is eventually approved.

Pair reviewing

Notice that we haven't said this conversation needs to happen through the version control software itself: for example, as comments on a GitHub pull request. Actually, the best way to have it is by *talking*, which is an unbelievably high-bandwidth communication channel compared to those little text boxes.

In other words, code should be *pair-reviewed* as well as pair-programmed. For example, Aisha and Chiku could get on a screen-share and look at the code together, asking questions, making suggestions, explaining ambiguities, and highlighting potential problems.

It's far quicker to have an exchange like this synchronously, by voice:

CHIKU: *Is this `int` down-conversion safe? What if the value is bigger than 32 bits?*

AISHA: *It can't be, because if you look right here, it comes from...*

CHIKU: *Oh, I see now. What if we put in a comment here, just to make that clear?*

AISHA: *Sure... (TYPES)*

That took just fifteen seconds by my watch, but if they'd had to go round this loop a few times via GitHub comments, it probably would have pushed the merge back till Tuesday. Plus, Aisha and Chiku are probably still good friends, whereas lengthy code review debates have been known to jeopardise even the best relationships.

Tone of voice and even facial expressions convey a lot of extra information that would otherwise be lost, and somehow, people are always gentler and more respectful to each other *in person* than they are when typing messages back and forth.

Perhaps for this very reason, you'll sometimes encounter people who insist on trying to make the discussion asynchronous and text-only. The simplest way to foil this tactic is not to join in. If they send you comments on your code via GitHub or email, just reach out to them in person and ask to have a chat about it.

If they're still unwilling, then so be it. Evidently they don't want a *conversation*; they just want to talk while you listen admiringly. That's unlikely to be helpful, so you should bypass them altogether and find someone who *is* interested in a genuine conversation about the code.

Over time, you'll build up a useful network of relationships with people who can “pair review” code with you, and your code will be far better as a result (theirs, too).

The gentle art of code review comments

Pair review like this isn't always possible, though, so when a code review happens textually, that means we need to take even *more* care about what we say and how we say it. Let's start with the "what".

A good way to think about code review is as a process of *adding value* to existing code. So any comment you plan to make had better do exactly that. Here are a few ways to phrase and frame the different kinds of reactions you may have when reviewing someone else's code:

- *Not my style.* Everyone has their own style: their particular favourite way of naming things, arranging things, and expressing them syntactically. If you didn't write this code, it won't be in your style, but that's okay. You don't need to comment about that; changing the code to match your style wouldn't add value to it. Just leave it be.
- *Don't understand what this does.* If you're not sure what the code actually says, that's *your* problem. If you don't know what a particular piece of language syntax means, or what a certain function does, look it up. The author is trying to get their work done, not teach you how to program.
- *Don't understand why it does that.* On the other hand, if you can't work out *why* the code says what it says, you can ask a question: "I'm not quite clear what the intent of this is. Is there something I'm not seeing?" Usually there is, so ask for clarification rather than flagging it as "wrong".
- *Could be better.* If the code is basically okay, but you think there's a better way to write it that's not just a style issue, turn your suggestion into a question. "Would it be clearer to write...? Do you think X is a more logical name for...? Would it be faster to re-use this variable, or doesn't that matter here?"
- *Something to consider.* Sometimes you have an idea that *might* be helpful, but you're not sure. Maybe the author already thought of that idea and rejected it, or maybe they just didn't think of it. But your comment could easily be interpreted as criticism, so make it tentative and gentle: "It occurred to me that it might be a slight improvement to use a `sync.Pool` here, but maybe that's just overkill. What do you think?"
- *Don't think this is right.* If it seems to you like the code is incorrect, or shouldn't be there, or there's some code missing that *should* be there, again, make it a question, not a rebuke. "Wouldn't we normally want to check this error? Is there some reason why it's not necessary here?" If you're wrong, you've left yourself a graceful way to retreat. If you're right, you've tactfully made a point without making an enemy.
- *Missed something out.* The code is fine as far as it goes, but there are cases the author hasn't considered, or some important issues they're overlooking. Use the "yes, and..." technique: "This looks great for the normal case, but I wonder what

would happen if this input were really large, for example? Would it be a good idea to...?”

- *This is definitely wrong.* The author has just made a slip, or there’s something you know that they don’t know. This is your opportunity to enlighten them, with all due kindness and humility. Don’t just rattle off what’s wrong; take the time to phrase your response carefully, gracefully. Again, use questions and suggestions. “It looks like we log the error here, but continue anyway. Is it really safe to do that, if the result is `nil`? What do you think about returning the error here instead?”

Accentuate the positive

We often think about code review as a process of pointing out what’s wrong, but there’s also a great opportunity to praise what’s *right*. Indeed, your review should begin with a description of everything you like about the code. “This is really well organised, and I like how you’ve used X, Y, and Z to give extra context for readers. The function names are great, and this seems like it would be a nice API to use.”

A spoonful of sugar helps the medicine go down, and even when you have to give someone feedback that’s hard to hear, there’s always something good you can find to say, too. Lead with the positive, follow up with your tactful, respectful comments and suggestions, and finish the review with something encouraging: “Great job! This is really shaping up nicely.”

If you’re reviewing code from someone who’s senior to you, though, it’s probably not appropriate to tell them they did a great job. They might feel, rightly or wrongly, that you’re not yet in a position to judge that. Instead, you could say something like “I really learned a lot from this, thanks for giving me the opportunity to look at it.”

When your vanity is wounded

Your self-esteem can take a hit from code reviews, especially if they’re not done tactfully, and that hurts.

You put your heart and soul into a piece of work, you send it off with fond expectations that it’ll be hailed as a triumph, and then it comes back with brutal comments all over it.

You feel wounded, you feel angry and resentful, and you’re tempted to hit back. “Oh yeah? Well let me tell *you*, buddy...”

Needless to say, this isn’t a good idea. Your reviewer almost certainly *means* well, even if it doesn’t come across that way. If they *don’t* mean well, and they intend their remarks to be dismissive and hurtful, then sure, they’re a jerk.

But the world is full of jerks. It’s not your job to turn them all into decent human beings, and you’re certainly not getting paid for it.

Instead, just suck it up and move on. There’s probably more truth in their feedback than you’re prepared to acknowledge at first, even if it’s expressed in an unnecessarily

rough way.

Not everyone is a master of people skills, and even the kindest of colleagues can have a bad day for reasons totally unrelated to you or your pull request. *Tout comprendre*, as they say, *c'est tout pardonner*: if we could truly know the secrets of someone's heart, we would never think harshly of them.

“I see what you tried to do here”

Some people's code review comments, while not exactly rude, can be very condescending, which is almost as bad. You know the kind of thing: “Hey, you probably didn't realise this, but you should use a pointer receiver here. Let me show you how that works...”

Perhaps they're trying to be nice, but actually, what we *hear* is “You don't have a clue what you're doing. Lucky I was here to save you from making a total ass of yourself.” Indeed. The funny thing is that people who act like this usually don't know very much themselves, and maybe that's why they're so quick to point out to *others* what they don't know.

This happens a lot in the tech industry, and you'll find that those who know the most often don't say much, while those who know nothing seem to talk all the time.

Maybe that's because, in tech, people can often get away with being unaware of their own incompetence. Not all industries are so forgiving.

In aviation, for example, people who greatly overestimate their level of skill are all dead.

—Phillip Greenspun, quoted in Jessica Livingston's [“Founders at Work”](#)

Stylistic nitpicks

We saw earlier that “not my style” comments don't really add value to code; they just create churn. Every time this particular section of code is updated, it will also accumulate wasteful and random changes to suit the “style” of whoever happens to be reviewing it. So, while you won't be *giving* these comments, what should you do when you receive them?

My suggestion, which may seem a trifle radical, is simply to *ignore* them. Just because someone has made a comment, it doesn't mean that you automatically have to respond to it. If you don't, they're unlikely to follow up.

If they do, and if they continue to insist on the changes (assuming they're in a position to insist), you can have a private conversation with them to clarify exactly what they think the blocking issue is.

Is it purely a matter of style, or is that masking something more important that they haven't adequately explained? You're entitled to ask them politely if there's something they can't live with.

The end of the rainbow

How do you know when you're becoming a master? Probably because other people will start referring to you that way. And the more of a master you *are*, the more you'll politely correct them: "No, I'm no master. Just a student."

Mastery, in fact, is one of those things that creeps up on you. There's never a day when you wake up and say to yourself, "Wow! I'm a master!" It doesn't work like that. Rather, the more of a master you become, the less it *feels* like you know.

That makes sense, because the deeper you go into any subject, the more things you realise there are that you don't yet know. That's what it *feels* like being a master: it feels just like being a perpetual beginner.

8. Seniority

The primary skill of a senior engineer is to be bringing up the junior engineers. If your job title says “senior”, and there’s no juniors around, you’re not a senior engineer.

—Lars Lehtonen, “[Burnout and Ops](#)”



To keep moving in the right direction, you’ll need to do more than just be great at your job; you’ll need to know how to navigate and survive the dangerous and poorly-charted waters of the organisation.

Beginnings

If this is your first job, or if you have less than 2-3 years experience, you will effectively be a *junior* engineer. Sometimes this level is called something like “early career” instead, but it means the same thing: entry level.

Juniority

After a year or two of satisfactory progress as a junior, and acquiring a lot of knowledge pretty quickly, you may expect to be considered *mid-level*. Whether or not this is actually reflected in a change of job title, or even a raise in pay, nonetheless, you’ve taken the first step up.

You're no longer a beginner, and you can (confidently or not) take on most development tasks, with occasional support, but perhaps you're not yet quite ready to lead a team or execute major projects by yourself.

Doing your One Job

The more time and focused effort you put in outside work on solo study and programming, the faster you can expect to learn the stuff that a mid-level developer needs to know. And, of course, there are lots of other important things you'll be learning at the same time, such as how the business works, how to deal with your manager, your co-workers, and so on.

But you also need to *execute* well on what you're asked to do, and that's something that people often neglect. No matter how helpful or useful you are in all sorts of other ways, the company is unlikely to reward you for it if you're not also doing the thing they *hired* you for.

In other words, focus first on your One Job:

When someone isn't meeting the basic expectations for their core responsibilities, it doesn't matter how many other wonderful things they are doing. In fact, those things can become strikes against them.

When you are meeting expectations for your One Job, then picking up other work is a sign of initiative and investment. But when you aren't, you get no credit.

—Charity Majors, “[Know Your One Job and Do It First](#)”

It's not always easy to know exactly what that One Job is, but you can clarify this by talking to your manager. They should be having regular one-to-one meetings with you, at least weekly, and if they aren't, ask them to.

When do you become senior?

Fast-forward a few more years, and maybe even a few jobs in various different companies or industries, and there comes a time when you *can* pretty much do everything without help, and indeed you're able to lead or train others to do what you do.

And, again, while it may not necessarily be accompanied by a formal promotion, at some point in this process you will be recognised as a *senior* engineer. This is a gradual process: you'll come to notice that over time you're relying less and less on the support and guidance of those around you, and *you* are instead providing more support and guidance than you're receiving.

Whichever team or group you're part of, by the time you start to become senior you'll probably be one of the most experienced people in it. Even if you're not officially the team leader, you may find that people naturally look to you for leadership.

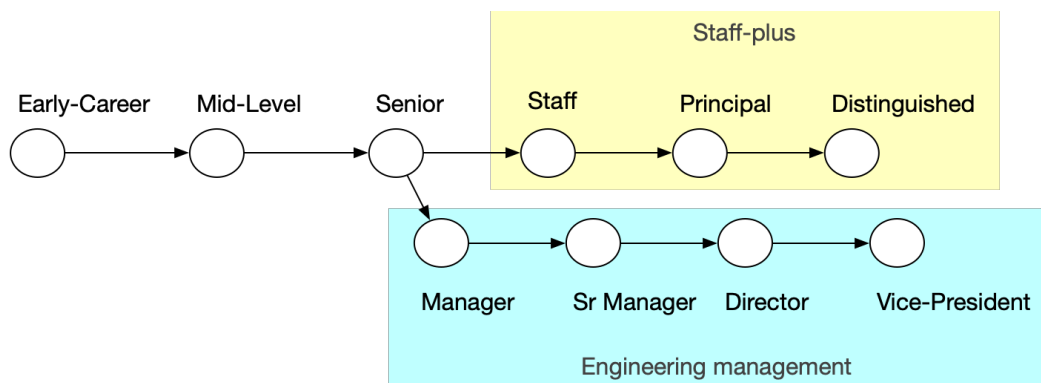
Once you reach the senior level, the road ahead forks into two possible paths. You can choose to stay on the technical side, and advance further through those ranks, or you

can opt to climb a different ladder, by moving into management.

Going further: management or staff-plus engineer?

If you want to stay an engineer, but still keep advancing, your next level will be what's often called *staff* engineer (don't ask me why). Beyond this, depending on your company, there may be “staff-plus” levels with titles like “principal engineer” and “distinguished engineer”.

On the other hand, if your inclination is more towards management, you may first become a *manager*, then a *senior manager*. Above this, again depending on the company, you may be able to progress to *director* and then perhaps *vice-president*, or even the *chief executive*. That's where the buck stops, and so will you unless you want to start your own firm and become independent.



This diagram, from the excellent [StaffEng](#) site, gives a good outline map of these two main routes for advancement.

Recording, thinking, and planning

Your one-to-ones with your boss are the best way for you to find out what you're supposed to be doing, whether you're doing it well, and what you can do to improve. Simply *asking* these three questions in each meeting would be a great start.

You should also regularly ask *yourself*: “Am I doing a good job? What would that look like? What should I be doing differently? What am I still missing?”

It's a great idea to keep a work diary (just a text file or a set of notes to yourself is fine). Every day, write a few sentences about what you worked on, and how it went. Record any useful new things that you found out, or problems that you solved. You might be surprised how quickly you can forget these day-to-day details, amid the stress and flurry of a busy development team.

Take careful note of any conversations you have with your manager, and what you learned from them. Use the diary to make a plan for how you're going to apply these lessons at work, and then record how that worked out.

Over time, you'll build up a valuable record of what you did, what you thought about, how you changed your approach, and so on. When the time comes for you to be considered for raises or promotion, you'll be able to use the information from your diary to explain in detail, and with references, how you've become a more capable and valuable employee.

These things can be rather intangible and hard to prove, especially after some time has elapsed, so being able to point to specific things that you achieved or contributed to will help you a lot. Of course, if something goes wrong and you get in trouble of some kind, it will also be very handy to have an accurate record of what exactly happened, and when.

When things go *right*, you potentially have a story to tell in your next job interview (see the earlier chapter on interviews). Make sure to extract any highlights from your diary and write them up with supporting detail. Over the course of a few different jobs you'll gradually amass a file of killer stories: they'll come in handy.

Visibility

You'll probably have a team meeting of some kind at least once a week or so, where you can discuss progress and problems and what-not. Most people just mumble something indeterminate when it's their turn to report, but you can do better. Because you've kept daily notes on everything you've done, you can give a clear, brief, but informative summary of your week's activity.

The more you pair-program, collaborate, and co-ordinate with the other people in your team, the more visibility your work will have to start with, and you can use the status meetings to communicate it more widely. It's also a good idea for your *team* to give regular updates to the company or division as a whole, on the same basis.

This is especially important in areas like operations and infrastructure, where people in the rest of the company hardly ever even know you exist—until something goes wrong, at which point you look bad.

There's nothing wrong with sending out regular little info-dumps about what your team has been doing: you installed this, you deployed that important update, you saved the day when payments went down at peak time, you extinguished a burning server.

Even when things don't go so well, it's still worth making your work visible. For example, you might not feel like telling everybody that you've spent the last two weeks trying to recover records from a damaged database, so far without success. To you that probably sounds like a failure, but for those not involved in the work, especially non-technical folk, it might be just the opposite.

They might think “Gosh, that sounds difficult and complicated. How highly skilled those engineers must be! I could never do that stuff; it's a good job we've got them around. In fact, they should get a pay rise.”

Well, we may hope that's what they think. It does no harm to give them the chance, anyway. As a rueful correspondent reported to me:

I was denied a promotion because I didn't show leadership in a crisis. The reason there were no crises was because I succeeded in preventing them.

A *small* crisis once in a while, then, might be no bad thing. It never hurts to remind people how wrong things could go if you weren't around.

Learning the ropes

You should also seek to learn as much as you can about this specific company and how it works. Not just the formal organisation chart, but how things *really* work, in practice, unofficially.

Who really calls the shots on certain decisions? Who is respected? Who is mistrusted? Who is humoured but ignored? How do you get things done when official channels are blocked? Whom should you call when things really hit the fan?

Every organisation is also a game

In order to advance, you also need to study how advancement *works* in your organisation. What does the company say it values and rewards in its employees? What does it *really* value and reward, even if it says something different in press releases and annual reports?

Advancement is a game, and the rules of that game are different in every company. You need to learn what the rules are, how to avoid breaking them inadvertently, and how to make them work for *you*. Alternatively, you may decide that this game is not one you can win: the system just doesn't suit your particular talents, preferences, or ways of working.

In that case, you might have to look for another company where the game is more to your liking. And if so, that's something you'd want to find out as early as possible. It's no good rising high in the wrong company.

You can either learn how the organization's promotion process works and play your cards right, or if the process seems so arbitrary and unfair you feel you will never have a shot at succeeding, you can try to switch to organizations or companies where you feel you might have a fairer crack of the whip.

—Cindy Sridharan, [“Know How Your Org Works”](#)

Patterns for success... or failure

Organisations tend to settle into certain behaviours and patterns. These patterns can be good or bad, and are often both at once, depending on the circumstances. It's useful to learn what these patterns are for your company, and whether you should adopt or avoid them.

Talk to people who've worked there for a long time and ask them about notable failures and successes they've seen, for example. Does the company always seem to make the same kind of mistakes? If so, why? Do they learn from past failures? If not, why not?

When things go well, is there something that successful projects tend to have in common? Do certain specific teams or people tend to succeed, and others fail, or is it more that certain kind of *projects*, but not others, tend to turn out well for this company?

What kind of experience and expertise is valued in this company? What is *not* valued? Which of them do you have? What kind of experience are you likely to acquire here that might also be valued elsewhere? What skills are held in high regard here, but would be useless or irrelevant at any other company?

Don't be an office politician

There are only so many promotions available, and you are effectively competing for them with all your co-workers.

Now, one possible strategy for succeeding in that competition is simply to be a valuable employee, a good co-worker, and extremely good at your job. That's the strategy I recommend, and I think it's the most effective over the long term.

But not everyone *can* do that. Their only alternative is to play office politics. You know how that goes: always telling everyone how busy they are, sending out emails or messages on weekends or evenings, playing up their own contributions while subtly bad-mouthing their co-workers, burying their own mistakes, and highlighting those of others. Sucking up to those above while bullying those below.

Dirty tricks, and how to avoid them

Some people, indeed, can be infuriatingly good at this game. Somehow they always manage to be present at the moment of success, and to take undue credit for it, while also being mysteriously absent when a crisis spirals into disaster, and there's blame to go around.

As you'd expect, these things often go on in the shadows. You may not even know that someone has fired a political missile at you, until it actually detonates. They can be launched from beyond your personal radar horizon.

You may think you've been doing a terrific job, and that may be the case, but when it comes to review time, you may find that your boss doesn't agree. Instead, someone else has sidled in and taken all the credit for what you did.

ELAINE: *There's this creepy new guy at work. He just—he just comes out of nowhere and he's right next to you!*

JERRY: *So he just sidles up?*

ELAINE: *That's right! He's a real sidler.*

—Seinfeld, “[The Merv Griffin Show](#)”

Winning by not playing

These dirty tricks are often successful in the short term, but over time, they tend to rebound on the tricksters. People learn whom they can and can't trust: who praises you to your face and condemns you behind your back. And backstabbers tend to draw the attention of *other* backstabbers.

Seasoned office politicians easily recognise those who are playing the same game, and usually target them for elimination first. That makes sense, because the biggest threat to a dirty-trickster is a fellow member of their own profession.

What goes around, in other words, usually comes around, if you wait long enough. That doesn't mean that the arc of the career universe always bends towards justice, but it does suggest that honest competence is the most reliable policy.

Moving on up

Examine your own skills critically, to help work out where there are gaps. Write down every skill or competence you can think of that's related to your job, and then score yourself against them.

For example, you might grade yourself out of 10 for each type of skill, where 0 would represent "total beginner" and 10 might represent "expert", whatever that means in this context. If you're not sure exactly how to grade yourself for something, ask someone else who seems knowledgeable in that area. How would they rate your expertise?

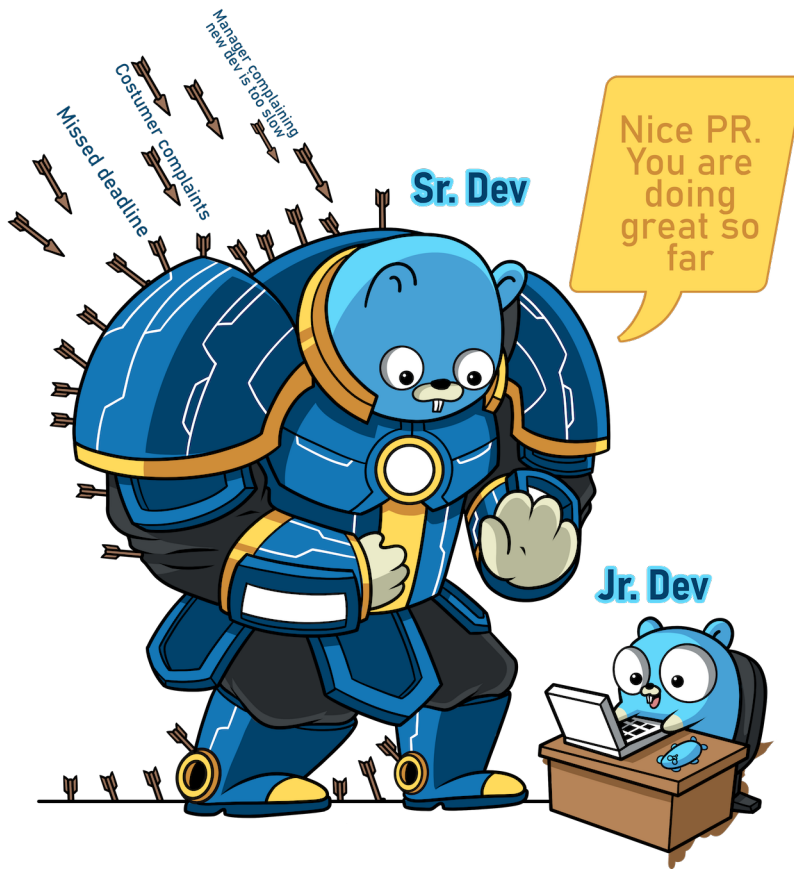
You should end up with a good picture, not only of what you know right now, but also of the areas where you need to improve to fill out your desired T-shape.

Mentoring: from mid-level to senior

Communicating your hard-won knowledge to others is the key to true "senior engineer" status. At this point in your career you'll find that many or even most of the people around you are less experienced (or at least less confident). Your job now will become more and more about encouraging, training, and helping these others.

In other words, mentoring. As we saw in the previous chapter, "mentoring" really just means "helping", and it's useful to approach it in that spirit. Think about mentoring not so much as *telling* people things you know, as *enabling* people to learn those things themselves.

What does "enabling" mean here? It can involve protecting junior team members from some of the stress and pressure coming down from above. A senior engineer should be able to create a bubble of safety and support to enable juniors to learn, try new things, make mistakes, fix them, and improve.



On a busy project, new tasks are always coming in, changes are being requested, and fires are breaking out all over the place. This hurly-burly makes it very difficult to concentrate and do good thinking work, especially for juniors.

As a senior engineer, you can divert a lot of these incoming missiles away from the rest of the team, taking on the most difficult or challenging problems yourself, and doing what you can to push back on the more unreasonable management or customer demands.

Supporting and encouraging

But don't just lock yourself away to do the senior-level work. Make sure that you're giving opportunities for junior engineers to collaborate with you on those tasks.

Even if they're not quite ready to take on these things by themselves, they're ready to learn about them, and to watch how someone more experienced handles them. Encourage the juniors to *contribute* something, too, however small.

When you lead, your real job is to create more leaders, not more followers.

—Kevin Kelly, “[Excellent Advice for Living](#)”

Mentoring can be tremendously rewarding, but also difficult, because in wanting to help people you can sometimes inadvertently hold them back. For example, if you micro-manage someone to the extent of never letting them make a mistake, that can backfire. They positively *need* to make mistakes, so that they can learn from them.

What you can do is match the challenges to the junior’s capabilities, so that they have more successes than failures. You can guide them just enough that the consequences of the inevitable mistakes aren’t too serious, and you can reassure them that it’s completely okay to *make* mistakes.

Absorb blame, distribute credit

Remember how, when you first started, every time you screwed up something you were terrified that you were going to get fired? Well, hold on to that memory, and make sure that your junior engineers don’t feel that way. When the team gets something wrong, take the blame on yourself: after all, you’re the senior engineer around here. Your shoulders are pretty broad.

But when the team succeeds, make sure the credit for that success goes mainly to the juniors. *You* have plenty of credit already, and you’ve earned it: you don’t need any more brownie points just at the moment. And you’ve enough experience now that you pretty much *should* be getting things right. On the other hand, for the juniors this is all new: they’re playing the game on hard mode. When they win, they deserve to be recognised for it.

Just being *called* “senior” means nothing in itself if you don’t *act* senior. And you can start doing that any time you like, without waiting for a promotion. Indeed, the more you fulfil the role of a *true* senior engineer, the more likely it is that people will start treating you as one.

Be a product engineer, not a software engineer

Another way to mark yourself out for seniority is to focus on the *product*, not the code. In other words, when you’re deciding what to do and how to do it, look at how the change affects customers and users, and prioritise those factors over purely technical ones.

An example might be when you’re assessing a performance improvement task, like “Reduce the execution time of function X by 10%”. Most software engineers would instinctively think this is important: code should be fast, and if it’s slow, then they feel they need to make it faster.

But *is* it important, really? When you have the product engineer mindset, you might try to find out by asking questions:

Is this time reduction having a positive impact on any of the stakeholders? Will this make the life of our customers better, reducing their waiting time? Will this

reduce the platform cost significantly?

—Carlo Bertuccini, “[Stop Hiring Software Engineers](#)”

In other words, while a software engineer is interested in software, a product engineer is interested in *products*, and in building a commercially successful product development organisation.

Again, you can start being a product engineer at any time, without waiting for the business cards. Just start thinking harder about what you’re doing, asking questions, and focusing on value.

The more seniority you gain, the more you’ll find that you’re being seen as a leader. What that means, and where it might take you, is the subject of our next chapter.

9. Leadership

If you cannot manage yourself, you have no business managing others.

—Jerry Weinberg, “Quality Software Management, Vol. 3”



Many of the qualities we associate with the senior engineer—coaching, encouragement, protection, support, motivation, and making the work of others visible—could also be described under the heading of *leadership*.

Tech leading

If you embody these qualities in the right ways, sooner or later, you may find that people start looking to you as a leader, and the first step on this ladder is likely to involve being what’s known as a *tech lead*.

What is a tech lead?

“Tech lead” is not a formal title, but a role an engineer is playing. It is not a promotion or a step up in a career ladder. Not everyone has to be a tech lead to grow in their careers.

Being a tech lead is not a permanent state. An engineer can play the role of a tech lead for a project or for a specific phase of a project.

—Gaurav Gargate, “WTF is a Tech Lead.. and What Does She Do?”

In other words, it's not that your boss will call you in one day and give you new business cards saying “tech lead”. It's more likely that you'll gradually notice that the tasks you're given are growing in complexity and difficulty, and they may involve co-ordinating the work of others as well as your own.

What does a tech lead do?

First, given some overall task or goal, it's their job to break this down and figure out the general approach that needs to be taken. The tech lead may do some system design, or they may just carve up the work into chunks that individual engineers or pairs could take on.

The tech lead makes sure everyone knows what they're supposed to be doing, is talking to the right people about it, and that the results come together in the right way, at the right time, to achieve the goal.

Finally, the tech lead serves as an *unblocker*, checking in with everybody to see if they're stuck on some problem, or just waiting for a resource, and doing whatever they can to eliminate the blockage.

Conflict and consensus

Getting work done by yourself is (relatively) easy. It's harder when you're trying to co-ordinate a larger team. Lots of engineers have lots of opinions. How, then, should you handle disputes about the appropriate way to proceed?

A leader can always pull rank, of course, and say “We're going *this* way, like it or leave”, but that's unwise. People might *take* the option to leave, and you probably don't want that.

It's always better to lead by consent. That doesn't mean saying “yes” to every suggestion, but it does mean having a fair and transparent way of making decisions about such issues.

Engineers love to argue the minutiae of engineering issues; that's almost a definition. These arguments (known as *bikeshedding*) are often more fun than writing code, if not as productive.

When fruitful discussions descend into bikeshedding, a good remedy is to say something like “Let's go round the table and get people's views.” Ask each person to (briefly) state what they'd prefer to do, and why.

It's a funny thing, but much of the time what really matters to people is feeling that they've been *heard*. The mere fact that their input has been recognised can make them more accepting of the final outcome, whatever it is.

“Is there anyone who can’t live with that?”

Then you can say “After listening to everyone’s views, the consensus seems to be for Option X.” And now ask the key question: “*Is there anyone who can’t live with that?*”

The phrasing is important. If you simply ask “Does anyone have a problem with that?”, the wrangling will continue. The question is not whether anyone has an objection, but whether anyone’s objection is so serious that they *can’t live* with the proposed decision.

That’s a much bigger deal. Engineers always have *opinions* about technical matters (another definition), but they’re usually flexible enough to go along with the group.

On the other hand, if someone *does* feel so strongly about the issue that they’d (implicitly) consider quitting the team or the company over it, that’s something you need to surface and discuss further.

Steamrolling over such strong feelings, or pulling rank, would be a serious mistake. As a tech lead, even an unofficial one, your job is to make a decision, not an enemy.

Negotiating from values, not positions

Try to draw out *why* the person is so fixated on their particular solution. What values are they fighting to maintain? Don’t attack the position, or the person: this just triggers automatic defensive behaviour that has nothing to do with the merits of the case.

Instead, ask the group “If there *were* a solution we could all agree on, what general shape would it have? What constraints would it have to satisfy?”

Instead of a battle between entrenched positions, we’ve switched the discussion into a collaborative problem-solving mode. This is much more productive and less stressful for all involved.

Are you a potential manager?

Being a tech lead, at whatever scale, is a good chance for you to find out whether you enjoy dealing with people, and whether you’re good at it. You may find that while you have no trouble juggling lots of complex technical issues, you struggle to maintain harmony between the various different personalities on the team.

As more of your time is taken up by meetings, mediation, and generally getting square pegs to fit into round holes, you may grow to resent this, and to wish that people would just leave you alone so you can write some code.

This is totally okay, and not everyone who’s a great software engineer would also necessarily make a great manager.

Promoting a good developer to management is often a twofold bad move: you’ll lose a good developer and get a poor manager.

—Mario Fusco

On the other hand, you may find that you have an unsuspected talent for getting the best out of people. Engineers tend to be smart and opinionated people with pretty independent minds, and managing engineers has often been compared to herding cats. If you've never tried this, it's awfully hard work, the results usually aren't that convincing, and the cats don't enjoy the process.

The trick of engineering management is to be involved with what the engineers are doing, but at the right level. If you get *too* involved in the details, you're not really letting your people do their jobs, and you run the risk of actively interfering with their work:

The org was run by a VP who kept his fingers a little too deep in the day-to-day product decisions (e.g. "I think you should add a menu there. Drop what you're doing and get it done asap"). By itself, that's not a horrible thing, but it was coupled with a culture where pleasing your manager always trumped pleasing the customer.

—Alan Page, “My End at Microsoft”

So if you do have the rare knack of inspiring and guiding people to focus their energies on a common goal, maybe that's something you should explore further. While developing software is fun, developing *people* can be even more fun, if you're good at it.

In your current company, the “standard” promotion path from senior engineer may be to management, and if that suits you, great. It might even be something that you haven't seriously thought about before as an option.

If you don't feel that management is for you, that's okay, too. There are other options for advancement. For example, in many companies if you want to stay technical, but rise higher in the org chart, that may involve becoming a staff engineer.

The staff engineer track

As a staff engineer, your concerns will expand from individual teams or projects to encompass the business as a whole. In other words, you'll become part of the leadership of the company, but with a technical focus.

You'll help figure out what the company should be doing, how it can operate better, which direction it should go in the future, and what steps it needs to take now in order to get there.

What does a staff engineer do?

The kind of problems that come your way at staff-plus level will be new and challenging, and they'll also be unique to this company, these people, and these products. They're not the kind of things you can look up in books how to solve.

More and more, *you* will be the one identifying the problems and calling for them to be solved. Instead of tasks arriving in your inbox, neatly labelled and categorised, it will be up to you to *see* that something needs to be done, and that you're the one to do it.

However, you may also find that you don't have the necessary authority to change things directly, just by issuing commands. That's not how it works. Instead, you'll have to lead by influencing the right people, in the right way, at just the right times.

There's a misconception that you become a staff engineer and then you'll be in control of the work you do, and everyone will listen to you and do what you want them to do. That's absolutely the opposite of what happens!

—Katie Sylor-Miller, “[Frontend Architect at Etsy](#)”

Becoming a staff engineer also doesn't mean stepping away from solving technical problems. The difference is in the *way* you solve those problems. Instead of fixing things with code, you'll be fixing them with *people*:

A few years ago I was at a fork in the road. Do I become a people manager or do I stay in a technical IC role and focus on solving technical problems?

*Trick question! Once you reach a certain level, all problems are solved by people. And we can make code do a lot of things, but making **people** do things is harder and influencing people to do what we want is harder still. This is what principal engineering is about.*

—Silvia Botros, “[Delving into the Other Side of Engineering Leadership](#)”

Your job is not to fix everything

Be careful you don't try to take on too much at once, or start feeling demotivated because you see problems everywhere you look. That will *always* be the case now. Your job is not to fix everything wrong with the system, but to help build a system that can fix *itself*.

This work is exciting and interesting, not to say challenging, but it'll also never be done. Software engineers tend to be people with orderly minds, who don't like things to be messy. They can't know that a bug exists without wanting to exterminate it.

Staff-plus engineering, on the other hand, means operating in a fuzzy and illogical world of problems that can never be completely solved, only mitigated. And that can be hard to adjust to.

It's also not so easy to know when you're getting it right. You can no longer point to specific changesets or bug fixes as evidence of your achievements. The effects of your hard work will be manifested in more subtle ways.

Bridging the gap between engineering and management

For example, as you talk to different technical teams or departments across the company, you may notice that the same kind of issue crops up over and over again. There's no standard or process for fixing or avoiding it, and because of this there's a lot of duplicated effort going on, and it's holding back the company from being as effective as it could be.

In a situation like this, a staff-plus engineer can both call out the problem, and take ownership of it. That doesn't mean that they issue directives telling people what to do. It's more likely that they'll talk to the people affected and find out what *they* think, learn about the various solutions they've tried, and analyse how that worked out.

As a staff engineer, you can now bring your own technical expertise to bear on the problem, and propose some ideas for solving it. It'll be your job to communicate the issues to the right people, to get widespread buy-in for your solutions at all levels, and to co-ordinate the actual implementation.

Staff-plus-plus

The higher you rise in the staff-plus engineering grades (for example, to Principal or even Distinguished level), the closer contact you'll have with the company's senior leadership, and the better acquainted you'll become with how the business actually works.

You'll have to take many things into account in your work, not just the technical or even the people problems, but the wider commercial, regulatory, or even social issues, too.

For example, there are always lots of things the company *could* do at any given time, many things it *should* do, and a few things it *must* do, and these all need to be weighed and prioritised.

The engineering side of the company will understandably want technical issues to take priority, but if the company isn't making money first and foremost, it can't even *employ* any engineers.

Deep down, most people feel like their job is cranking out technical code. I'm guilty of this myself. When I have my hands on a keyboard it feels like work. It's reassuring.

In reality though, a lot of the work that's critical to my success no longer matches that description. This became even more true as I moved up the IC ladder. Now stuff like making sure I'm solving the right problem is at the core of my work.

—Ryan Harter, “[Getting Credit for Invisible Work at the Staff+ Level](#)”

So at staff-plus level, you'll need a broad understanding of the business, together with its incentives, pressures, and constraints. You'll be called upon to make the technical *case* for the things you think should be done, but you'll also need to balance that against all the other things the business has to take care of.

Staying technical

It's your job to bridge the gap between management and engineering, which means you need to keep a foot in both camps. You need to know the business buzzwords and be fluent with the financials, but you *also* need to stay current on coding skills. If you want to influence engineers, you need their respect for your *engineering* abilities, not just your fancy job title.

The best frontline eng managers in the world are the ones that are never more than 2-3 years removed from hands-on work, full time down in the trenches. The best individual contributors are the ones who have done time in management.

And the best technical leaders in the world are often the ones who do both. Back and forth. Like a pendulum.

—Charity Majors, “[The Engineer/Manager Pendulum](#)”

As you rise higher through the ranks and your work becomes more self-directed, you can choose to make your engineering contributions where they’ll be most valuable. But choose wisely. Don’t just arrogate the most fun and interesting technical tasks to yourself: instead, do what *needs* to be done, and can *best* be done by you, rather than someone else.

To get a sense of how well the engineering function is working in this company, you need to be one of its moving parts. You need to be working on the same software projects as the non-staff engineers, using the same tools, deploying through the same pipelines, operating in the same clouds. When there are problems and engineers struggle to get their work done, you should feel the same pain and wrestle with the same issues.

That’s not to say that you have to contribute major features, design huge subsystems, or re-architect the whole suite of products. You just need to be *involved*, intimately, at every level.

Your influence will be subtle: a word in the right ear at the right time, a minor change of emphasis, or a potential problem averted. Your biggest engineering inputs are likely to be made through conversation, rather than a code editor. But for your contributions to be useful, you need to inhabit that world as thoroughly as the other engineers do, and that does mean writing code, even if it’s not every day.

Managing up

Whether or not you’re interested in becoming a manager yourself, you’ll have to deal with many managers in your career. Learning to do this well will have an outsize impact on your chances of advancing in the way that you want to.

What is a manager’s job?

As a manager, your job is not to manage people, but to manage processes and lead people.

—Andreas Klinger, “[Managing People](#)”

In other words, a good manager should ensure that the right processes unfold in the right way so that people can do good work. They should *also* be doing their best to guide, support, unblock, and encourage the people involved, so that they can work well within those processes.

That sounds complicated, and it is. It means not just having a good understanding of the technical issues involved, and ideally having some direct experience of working on them, but also an excellent understanding of *people*, which is much rarer and more difficult to acquire. This may explain why there are so few really good managers, and so many mediocre ones.

It can also be rather thankless work. When a tricky project succeeds despite the odds, hardly anyone remembers to say “Hey, nice management!” Yet everyone remembers who was in charge when things crashed and burned.

How to be managed well

Some managers are great, some not so great. Even if your manager isn’t the best, you can still do a lot to help them and make their job easier, which will in turn make *your* job easier. This skill is sometimes referred to as *managing up*.

We’ve already discussed the importance of regular one-to-ones with your boss, and outlined some questions that will help you get feedback from them about your performance: “What should I be doing?”, “Am I doing a good job?”, and “What can I do to improve?”

Another useful thing to ask is “What can I do, or do differently, to help make your job easier?” This lets your boss know that you want to support them, which is always good to hear. It also gives them an opportunity to tell you a little about how the organisation works, how they fit into it, and what they’re trying to do.

The manager README

It’s also important to learn about the *person* your manager is. The more you understand what they value, what makes them tick, how they operate at work, and how they’re likely to act or react in different circumstances, the better. If you want your manager to understand *you*, and presumably you do, you can aid that process by seeking to understand *them*, too.

Some people go so far as to write a “README” for themselves, explaining how they work, what they value, and the best way to have a productive relationship with them.

Manager READMEs are user manuals written by managers on their management style, philosophies, expectations, communication preferences, and more. It’s like a “how to guide” for employees getting to know—and working for—their bosses.

—Shannon Maloney, “[49 Manager READMEs from Slack, Netflix, Google and more](#)”

If your manager doesn’t have one of these, you should nevertheless try to learn what would be *in* it. Your weekly one-to-ones don’t have to be just about you: find out something about *them*, too.

Keeping your manager informed

Your interactions with your manager shouldn't be limited to just a brief chat once a week or so. You should make a point of checking in with them at least every *day*, even if that's only a single message to say "I'm working on Task X, and everything's fine—I'll reach out to you if I have a problem."

When something *does* go wrong, let your manager know right away, instead of waiting till tomorrow or the next day. Most people are reluctant to tell their boss "Hey, I screwed up!", and that's understandable, but that's not actually what a (good) boss cares about.

What they care about is that (a) you're having a problem and might need some help, and (b) you may be held up on delivering your work. So it's important to cover these points in your message.

For example, you could say "I'm having a little trouble getting this library to work, but I think I know what the problem is. I'm going to tinker with it for another hour or so, and if I haven't got it working then, I might need another developer to help me."

In other words, there's a *potential* slight delay, and while you don't need the manager to intervene *now*, you're giving them plenty of warning about what you *might* need later on.

You might think your boss will be more impressed with you if you just silently soldier on, and then suddenly issue an urgent demand for help later, but they won't. What will impress them is your understanding that you need to keep communicating, whether the news is good or bad.

In your one-to-ones, you have a secure, private channel to your manager's ears with no audit trail. Make the most of that, by encouraging yourself to talk about the *awkward* things: your emotions, the relationship between you and your boss, honest feedback about what they're doing, your worries or fears, honest feedback about what *you're* doing.

Planning promotion

Your chances of being promoted, and promoted to the job you want, also depend on your ability to communicate with your manager. In many companies the criteria for promotion can be murky and poorly understood, or at least poorly explained. It's a good idea, if you're interested in promotion to a specific position, to let your manager know this, and discuss it with them regularly.

Get your manager on your side

First, you can ask if *they* think this is the right next step for you, and if they don't, you should seek to find out why not, and what they'd recommend instead. Assuming you both agree that the promotion is desirable for you, is it *attainable*? If so, when?

It may be, for example, that you're simply not eligible for promotion until you've been with the company for a certain length of time, or ticked certain boxes such as training courses and performance reviews. Or your boss may feel that you'll have a better chance if you apply for the promotion or transfer after you've got six months more experience in your current job, for instance.

Once you and your boss are aligned on *where* you want to go and *when*, you can turn the conversation to *how*. What do you need to do, or demonstrate, to maximise your chances of getting this position? Under what circumstances have other people succeeded in making the same move, or failed?

Your manager should be able to give you specific, helpful, and actionable advice on what you should be doing between now and when promotion season comes around.

Make sure you get this advice in writing. That will help you remember it, and refer to it regularly so that you know whether you're still on track. Make it easy for your boss to promote you, and you'll both be happy.

Don't wait for the annual review

The promotion plan you agree with your boss should include timelines and specific, measurable goals. Check yourself against it regularly, at least every month or so, and discuss your progress in your one-to-ones. If you're on track, get them to say so, again in writing. If you're not, ask them to help you get *back* on track. Most managers would love to have a report who's so keen and engaged, and your manager certainly will.

When the review comes around, you should both have a very good idea of what's going to happen. If, as is likely, your boss needs to make the case for your promotion in a higher court, then all the communication you've had with them, and the written records, should really help with that.

Your manager is eventually going to be in a room with their peers arguing for why you need a promotion. If you've armed your manager with a great story about why your work is hard, they're going to be grateful to you for making them look smart.

—Ryan Harter, “[Getting Credit for Invisible Work at the Staff+ Level](#)”

Advancement doesn't just happen

Some expect raises and promotions to simply arrive unbidden at regular intervals, like the seasons. That probably won't happen. You won't advance in a systematic way unless you make a deliberate effort to find out what's required of you, and deliver it.

Another mistake that some people make is thinking that if they merely do their work well, they'll automatically be promoted. Not so.

Doing good work isn't enough

The main problem with promotion-oriented culture is that it's very hard to align promotion criteria with business objectives, and so engineers end up doing a lot of work that doesn't necessarily most benefit the product, users, or business—or even potentially their own growth.

—Zach Lloyd, “[Problems with Promotion-Oriented Cultures](#)”

In other words, what makes you or your customers or co-workers happy isn't always the same as what makes *management* happy. If your company culture rewards speed over quality, for example, then you could be the best engineer on your team, but still be the last to be promoted, because you take a little longer over your work than everybody else.

Or, if you spend all your time on small but critical internal tasks that aren't visible to people outside your team, you may find it hard to make the case for promotion because you haven't been associated with any high-profile revenue-driving projects.

Quality and visibility

Visibility, then, is critical to your advancement, but it's not enough on its own. *Bad* workers are also visible, but for the wrong reasons.

The decision-makers in your company need to know about you and your work, but for the right reasons. So focus on getting your One Job done well, being the best co-worker you can be, and building a productive relationship with your manager, *then* think about how to make those things visible.

What's the goal?

It's important to bear in mind *why* advancement matters. It's not for the fancy job titles, or the external validation, or the power and status. It's not even for the money. The point of advancement is to stay on track to the career destination you want, whatever that is.

However, you can't do *that* without money. Not just because you need to eat, but because if you don't have money then you don't have many *choices*. To keep your options open and stay focused on what really matters, you'll need to get good at making, and managing, money.

And that's the subject of our next chapter. Cha-ching!

10. Money

It's easy to sit there and say you'd like to have more money. And I guess that's what I like about it. It's easy. Just sitting there, rocking back and forth, wanting that money.

—Jack Handey, “[Deep Thoughts: Inspiration for the Uninspired](#)”



The most lucrative work isn't always the work you want to do, or can do well, and vice versa. So even if you succeed in getting to a place where you earn a lot of money, you may find that you're not happy in what you're doing, and you can't see a path to fixing that.

It's not really about money, then. Rather, it's about *security*. Security gives you freedom, opens up more options, and gives you room to think about what you really want to do. And, while money isn't the only thing that brings security, it can help a lot.

Your financial position

So how much is enough money, and how do you get it?

Making a budget

Every company, division, or even team has a budget, outlining the demands it expects to face and the resources it has available to fulfil them. If you aspire to one day run

your own company, it's time to start writing a budget for yourself, and figuring out how to stick to it. You don't need any fancy software for this: a spreadsheet, or even just a piece of paper, will be fine.

Since you probably get paid monthly, and most of your outgoings are also likely on the same basis, let's frame our budget in terms of a single month. You'll have annual expenses too, but we'll deal with those in a moment.

Let's start with the plus side of the ledger: the money that you have coming in every month. Your salary will usually be the biggest component of this, but you might also have other sources of income: side gigs, dividends from stocks and trusts and so on. Write them all down and add them up, so that you have a single number representing the money that you receive in a typical month.

If you get any income that arrives annually, such as a bonus payment, divide that by 12 and add it to the monthly figure.

Now for the minus side: your monthly outgoings. Let's start with the things that you have absolutely no choice about: you *have* to pay these, or bad things will happen. Rent or mortgage, car payments, fuel, groceries, health insurance, utility bills, debt repayments. These will disappear from your bank account every month whether you like it or not.

Next, total up your *discretionary* outgoings. These are, basically, everything else you spend money on: eating out, entertainment, socialising, weird stuff you buy from eBay, books, magazine subscriptions... it's probably a fairly long list. Check your bank statements for the last two or three months, make a rough sum of everything in this category, and divide it to get a ballpark monthly estimate.

You will also likely have some annual outgoings: house insurance, things like that. Again, divide these by 12 to figure out how much they all contribute to your average monthly outgoings.

Add up your mandatory and discretionary outgoings to get a single figure for how much you pay out in an average month to sustain your current lifestyle and position.

Your net worth

There's another number it's very important to know, and that's your *net worth*. In other words, how much wealth you have. List the current balances of all your bank accounts, savings accounts, deposits, and so on, plus the value of any stocks you own. Write down all non-trivial assets you have that could in theory be converted into cash: anything worth over \$1000, let's say.

If you own your own home, that's certainly a valuable asset, but don't include it in this list. A home isn't usually counted as part of your net worth, because you need somewhere to live. If you sold that house, you'd have to buy another one, or rent one, which would cost money.

Finally, subtract any outstanding debts, such as credit card balances, student loans, or whatever. The result is your net worth.

Your net worth is essentially the cash you have in the bank, plus the money that you could raise by liquidating assets, without making yourself homeless in the process.

Changing the curve

Your net worth will change over time, and we can now say by roughly how much, and at what rate. Returning to your monthly budget, find the difference between your total incomings and your total outgoings. This is your budget *surplus* (or deficit), and that's how much your net worth will increase (or decrease) every month.

Suppose, for example, that after you've paid all your bills and spent as much as you like on fun and good living, you have \$100 left at the end of each month. Most people are probably not in this happy position, but it's just an example. If this were the case, then you could expect your net worth to increase by \$100 a month. Whatever your *current* net worth, positive or negative, we can see that over time it is moving in the right direction.

But suppose the melancholy fact is that, after these calculations, you find that you're poorer by \$100 every month. This may not sound like much, and it isn't, but your net worth is eroding. Over time, you'll just get poorer and poorer, and sooner or later, you'll find yourself in debt (that is, negative net worth) if you're not there already.

This may be depressing news, but it's really important that you have the facts. Once you do, you can start making a plan to change the situation.

If you're running a budget deficit, this is a serious problem. If you also have negative net worth, this is a *crisis*. Being in debt is expensive: it costs you money, and that's exactly what you don't have. You need to take urgent action to change the slope of your net worth curve, from negative to positive. To do this, you need to run a budget *surplus*.

One way to do this is to increase your incomings: for example, by taking a second job. The *first* thing you should look at, though, is reducing your outgoings. Look at your list of discretionary spending. What is the biggest item? If you reduced or eliminated it, would this bring your budget into surplus? If not, what else could you cut?

Money management

Let's suppose that at some point you're able to reach the happy position of enjoying a budget surplus, however small it may be at first. What should you do with it?

Building up an emergency reserve

If you have any debt, then you should pay that off before doing anything else. Debt, as we've seen, costs you money every day that you have it (sometimes a lot of money, de-

pending who you borrowed it from). Eliminating all debt, targeting the highest-interest obligations first, is the best investment you could possibly make at this point.

The next thing you should do, once you're clear of debt, is to accumulate a small *emergency reserve* (otherwise known as an "oh #\$@!" fund). When your net worth is close to zero, this makes you very vulnerable to sudden, unplanned, and unavoidable expenses.

If suddenly having to pay out a few hundred or even a thousand dollars would put you dangerously in debt, then that's the first thing to fix.

Ideally, open a separate checking account for your emergency reserve, so that you can't spend it by accident, but make sure you can still access it quickly when you have to. How much cash should you have in your reserve? A good target to aim for at first is the equivalent of a month's salary. Three months is better: six months is ideal.

After all, you might find yourself unexpectedly unemployed, and it can happen to the best of us.

Don't wait for something to turn up

Sometimes people who are in debt and getting steadily poorer every month can seem curiously unworried about it, even if they know the true sorry state of their finances. They may believe, like *David Copperfield's* Mr Micawber, that "something will turn up". Maybe they'll get a terrific job that pays a ton of money, and all their problems will be solved.

Maybe. But this is a bit like thinking that we can cheerfully ignore climate change because sooner or later some genius will invent a solution to it. That *might* happen, but it's not the way to bet, because the downside risk of being wrong is so enormous.

Systems are fun

If you're not used to thinking about managing your money, it might seem rather boring at first glance. Maybe there's even something a little distasteful about the idea of constantly thinking about money. Shouldn't we have our minds on higher things?

Sure, but you know what's guaranteed to have you constantly thinking about money? Being poor. How many people might have invented wonderful things that changed the world if only they hadn't been so busy working every hour to feed and clothe their families? The paradox is that, to gain the freedom not to think about money, you need to think about money.

But, because we're programmers, we can do it in a smart way, in an organised way, and in a systematic way that means we never waste time thinking about problems we've already solved. That's the way we should approach our software work, so why not this, too?

"Some deranged fold of my lizard brain gets most turned on by nine detailed pages on how to wash your dishes... I only really get interested in something when it involves some kind of fancy system with charts and documentation."

—Merlin Mann, quoted in Scott Rosenberg’s “[Dreaming in Code](#)”

Putting your money to work

Let’s suppose that, sooner or later, you reach the delightful point where you have some *disposable income*. That is, you’re free of all debt, you’ve built up a decent emergency reserve, your sensible monthly budget is balanced, and you find that you actually have money left over. What should you do with it?

Investment plus time equals wealth

In the first chapter of this book, we said that the best time to start a pension is twenty years ago, and that’s because investments *compound* over time.

Suppose you invest \$1,000 at an annual interest of 5%. Let’s see what happens to it after that.

In a year, you’ll have \$1,050. That seems like a ridiculously tiny difference, doesn’t it? Just fifty dollars earned in a whole year, and you had to lock up a thousand dollars to do it? How could that possibly be worth the bother of even filling in the paperwork?

After five years, though, you’ll have \$1,283, and after ten, \$1,647. If you’d invested your initial thousand bucks *twenty* years ago, you’d now have \$2,713. In other words, you’d have nearly trebled your money. It starts to seem like filling in the paperwork might be worth it, doesn’t it?

Now suppose you open an investment account when you’re 25 with the same thousand dollars. What would it look like when you come to retire at, perhaps, 65 or so? Well, something more like \$7,360; more than seven times your outlay.

All for doing nothing whatsoever but reading the statements and watching the money grow... and, of course, for having had the foresight to *make* that \$1,000 available to invest in the first place.

You don’t have to put the money into a pension fund, by the way; all *they’ll* do is invest it in the stock market, and you could do that yourself. Self-investing has the advantage that you can access the money at any time, rather than having to wait until your scheduled retirement date.

How much cash do you need?

A good rule of thumb says that your wealth should be roughly evenly distributed between cash, real estate, and stocks. Cash is convenient, but depreciates constantly thanks to inflation. Most people who *have* any spare money don’t think beyond keeping it in cash, and as a result it drains steadily away every year instead of growing as it should.

You'll need *some* cash in your bank account, of course, to pay bills and so on, and it's unwise to let your balance get too close to zero every month. A small miscalculation could see you overdrawn, and that usually attracts a penalty.

Instead, aim to maintain a *float*: say, a few hundred dollars, so that your balance can survive an unexpected debit or two. Any cash left over after that should go into a deposit account, where it will earn a little interest.

The amount of interest you can earn usually depends on how long you're willing to lock up the money. The higher the interest rate, the longer before you can get the money back if you need it.

If you have things organised well enough that you can spare a certain amount of money for, say, two to five years, then you'll be able to set up a relatively high-interest account for such long-term deposits. Sometimes you *can* access this money before the term expires, but there may be a significant penalty, so check the small print carefully before you invest.

Emergencies don't usually give you advance notice that they're coming, so it might be a good idea to keep the money set aside for your "oh %*\$#" fund in a separate account that allows instant access.

Real estate: buy, don't rent

You probably need somewhere to live, and no doubt both you and your parents sincerely hope that it won't have to be with them. But renting is expensive, and you don't get anything in exchange for your month's rent other than the right to live somewhere for thirty days.

Putting the same amount of money into *mortgage* payments, on the other hand, not only provides a roof over your head for the duration, but you also own a share of the house involved, and it grows year on year until you eventually own the place outright.

So if you don't already own property, try to buy some as soon as you can. Every month you live in a rented property it's like you're just burning that money to keep warm. The sooner you can start *investing* that money in real estate instead, the better.

The stock market

We all know that stocks fluctuate in value, sometimes wildly, but over the long term, the stock market generally goes up, often by quite a bit.

So it makes sense to put at least some of your spare money into stocks, preferably low-cost, low-risk products like exchange-traded funds (ETFs) and unit investment trusts (UITs). These spread your risk across many individual companies, reducing the chance that a single failure will wipe your money out.

Actively managed funds, which involve some coked-up Wall Street trader playing the market with your money, often don't perform any better than the market as a whole. And their fees tend to be fairly expensive (after all, there's all that cocaine to pay for).

ETFs, on the other hand, are low-cost because they're pretty much automated. They just track the stock price of a given bunch of companies, such as those listed in the Dow Jones index. So they're cheap to run, yet, by definition, they'll perform just as well as the market they track.

Bonds, on the other hand, are a way for companies (or governments) to borrow money from the markets. As such, they're very secure, since the bond issuer *must* pay you back. While stock values go up and down, and can go to zero, bonds always pay out (provided the company or government doesn't go bust entirely, and even then bondholders get priority over other creditors).

As befits a low-risk investment, though, bonds don't tend to deliver a high rate of return. They are solid, boring, and reliable, all of which are excellent attributes in someone who's going to look after your money for you.

Pensions

Whether or not you choose to self-invest in the stock market, it's still a good idea to start a pension. In some ways, the fact that you *can't* access your pension until retirement can be a good thing: it sequesters some of your money where you can't get to it.

You usually get tax relief on pension contributions, and the money you earn as your pension grows is also not taxed as income. Thus, the more you earn, the more it will pay you to divert a chunk of that income into your pension.

Even more attractively, many employers will supplement or match your pension contributions, which is essentially free money. And because of the powerful action of compound interest that we talked about earlier, the impact of this extra money on your eventual retirement income can be very significant.

How much money will you need when you retire? Well, that depends on the lifestyle you want. Most countries have some form of income assistance, such as a basic state pension, or a fuel allowance, but these don't usually amount to much. They might keep you alive, but not much more.

If you think it's uncomfortable living on the breadline in your twenties, try doing so in your sixties or seventies. It's wise to make your own arrangements for the future, rather than relying on the state to rescue you from the consequences of lifelong improvidence.

Money buys freedom

The aim of learning to manage your money, then, is not merely to accumulate huge piles of gold for you to roll around in (though that can be fun, provided you keep the curtains closed against curious neighbours).

Rather, it's to give yourself enough security and freedom that your career decisions needn't be *ruled* by money.

In other words, to have *agency*. And that's the subject of our next chapter.

11. Agency

There is a basic incompatibility between any organization and freedom of thought.

—William S. Burroughs



As you advance in your career, accumulating mastery, seniority, authority, responsibility, people skills, and hopefully a decent amount of money, you'll start to bump up against the limitations of working for someone else.

Preparing for the jump

It doesn't matter how elevated your job title is, or how much money you make: in the end, you'll find that you lack either the freedom or the authority to set the parameters of your own work.

As creative people, I believe that most of us are ultimately looking for freedom to work on what we believe is important and to do it in a way that we believe to be right. In short, we're looking for the freedom to be creative.

Early in our careers, we move from low-level jobs toward higher-level jobs partially for the money and recognition but largely for the ability to make decisions.

—Chad Fowler, [“Clone Yourself: The Step by Step Guide to Finding Freedom at Work”](#)

Finding your niche

What can you do? That's easy: you can quit.

Asking the key question that we touched on in an earlier chapter will help you decide: "Am I where I want to be, doing what I want to do?" If the answer is no, and that answer doesn't change over a few months, then it may be time to move on.

Where you should go is a trickier problem. If your long-term career goal is to run your own independent business, it's worth asking yourself if you feel ready to do that yet. If not, what would make you feel ready? Are there some extra skills or experience you still need to acquire? Where could you get those?

Especially early on in your career, you may benefit from switching jobs pretty often, and trying out a whole range of different things. When you find what seems to be *your* thing, then you can focus more on that, but it sometimes takes a bit of experimentation to find it.

Many successful people are like good oil scouts: They spend a lot of time searching for their space, and then they drill deep when they find the right niche.

—Derek Thompson, "Your Career Is Just One-Eighth of Your Life"

On the other hand, it's perfectly possible to have *some* degree of agency and independence within someone else's business. You may find, for example, if you advance a little way down the staff-plus engineer track, that you enjoy the balance of challenge and security that this kind of work offers. In that case, by all means pursue it further.

You might find that you hit a ceiling within one company that you can only break through by going somewhere else. Indeed, changing companies can be one of the quickest routes to promotion and seniority. Instead of lobbying to change your job title where you are, why not walk straight into the job you want at the firm next door?

In practice, it might not be as easy as that, but it's worth considering—if only for the leverage and confidence that such opportunities might give you.

But, for one reason or another, it may not be time to move on just yet. Maybe you need to stay a little longer for your stock options to vest, or the job market is in bad shape, or other events in your life are injecting a little turbulence, and you need stability at work right now.

If something other than sheer inertia or despondency is keeping you in a job you can't stand, what can you do?

Resigning in your head

Some of the happiest periods of employment in my life, weirdly enough, have been after I've handed in my notice.

And that makes sense, when you think about it. What is the major source of your stress or unhappiness in an unsuitable job? Probably the knowledge that you're stuck there. The moment you can see a future opening up beyond this job, you can relax.

The feeling of freedom you get from quitting is delightful, and if you're not *quite* ready to quit yet for one reason or another, maybe you can still channel that feeling. I call this "resigning in your head", and as strange as it sounds, it really works.

If you have no other options at the moment, you can at least promise *yourself* privately that you're leaving this hell-hole as soon as conditions permit. You might be surprised at what a difference that makes to your mental well-being.

Consulting

If it doesn't look like you'll have the opportunity to leave for a while, you can play another game: being a "secret consultant". Consultancy is interesting work, and we'll have more to say about it in a moment, but one useful attribute of it is the ability to maintain a certain emotional distance from what you're doing.

Emotional distancing

A consultant generally arrives as a stranger to everyone at the company, and because they're there to tackle a specific and usually fairly short-term problem, they don't get too deeply involved in the politics, relationships, or unrelated issues that go on there.

They don't have the stress of constantly worrying about their future at this company, because they don't *have* a future at this company. They're just here to complete a task and then move on to another company with another problem. That's very freeing, in some ways.

Sure, the consultant doesn't get the benefit of putting down roots in the organisation, building long-term working relationships, or accumulating social and technical credit. On the other hand, they don't have all the stress, grind, worry, and bullshit that can go along with being a permanent employee. In essence, it's as though they've already resigned in their head before they arrive.

The secret consultant

This is one of the things that makes consultants so helpful: they *don't* work here. They're not prey to any of the groupthink that plagues organisations, and they're not locked in the straitjacket of "well, we've always done it this way".

A consultant can size up the problem, figure out what's gone wrong, and give their honest opinion about what to do, without fear or favour. At least, to *some* extent; consultants, like anyone else, are keen on repeat business.

So why shouldn't you work the same way, even if you're not technically a consultant? The key to the "secret consultant" game is, ultimately, your lack of emotional attachment to what you're doing. It's your job, but it's *just* a job.

Do a stint in consulting

If the “secret consultant” mentality appeals to you, you might like to consider actually *being* a consultant, of one kind or another. Whether you work independently, or as the employee of a consulting firm, the experience you can gain from even a brief spell of this kind of work is invaluable.

Working as a consultant unlocks career gains that are difficult, if not impossible, to achieve as an in-house engineer or engineering leader. A good consulting gig is a constant stream of the most interesting projects on the market, a smorgasbord of tasty ways to improve your skills.

When you consult, there are zero filters between you and the effectiveness of your work. Consultants who don't deliver as promised don't last long. So you get good at listening, at documenting, at showing the value of your work. You learn to solve real problems, not fake ones—because that's how you get paid now.

—Forrest Brazeal, “[Every Engineer Should Do a Stint in Consulting](#)”

And you don't have to be *called* a consultant to do consulting work. Many big companies have senior engineers whose job it is to essentially consult for customers about the company's technology. They might be called a solutions architect, a post-sales or customer success engineer, or a technical account manager, but it's all roughly the same thing.

Don't solve the problem too quickly

So how can you succeed as a consultant? The first thing you'll need to do, oddly enough, is to resist the urge to solve the client's problem straight away. Partly because this is insulting—it's like easily unscrewing the lid of a jar that's defeated someone else—but mostly because your “solution” probably isn't the right one.

And, while the issue will usually *present* as a technical problem, don't be fooled. The reason a technical problem exists is because of something deeper and more fundamental (and less easily solved).

No matter how it looks at first, it's always a people problem.

—Jerry Weinberg, “[Secrets of Consulting](#)”

So you'll need to get good at *appearing* to solve the technical problem, while *really* solving the people problem—or, at least, trying to. You probably won't succeed, but that's okay. The client doesn't really expect you to.

What they're paying for is not necessarily a solution, as such, but *validation* of what a serious problem it is, which explains why they weren't able to solve it themselves. “See? Even an expensive consultant couldn't fix this!”

You can't "win" this game

Most really serious problems are the consequence, not of a single mistake, but of a *system* that's dysfunctional in some important way, and often the entire organisation is involved. This can take a lot of fixing.

It's not that anyone is doing a bad job, particularly; it's that the built-in constraints and incentives are basically setting them up to fail. When the system is faulty, even if everyone involved is doing a good job, they won't get good results.

Things are the way they are because they got that way.

—Jerry Weinberg, “*Secrets of Consulting*”

So your real job as a consultant is to figure out *how* things got this way, and what would have to change in order for people to reasonably expect different results in the future.

You may not be *able* to change it—you almost certainly can't—but you can at least help the client see what it is about the current system that's preventing them from succeeding.

And that may mean telling them some hard truths, which doesn't always go down well. *Any* suggested change is an implicit criticism of the people responsible for the status quo, and they're probably the people you're talking to.

It's only natural for them to feel defensive when someone parachutes in and starts pointing out everything that's wrong. Who wouldn't?

Let them solve their own problem

There's a saying that diplomacy is the art of letting someone else have your own way, and this applies to consultancy too. The more you can get the client to suggest their *own* solutions, the more likely they are to be actually put into practice.

The more people in the client organisation whom you can involve in designing the solution, the more ownership of it they will feel, and the more passionately they will argue for its adoption. We may call this, perhaps, “invented here syndrome”.

So even if you know the right thing to do straight away, don't say so. Don't give directives to be implemented; instead, ask a series of *questions* in just the right way, so that the client arrives at the solution themselves. Then you can say “That sounds like a great idea. Let's try it!”

This way, everyone's happy. The client's managers are happy because their mistakes weren't humiliatingly exposed by some outsider, yet their decision to hire a consultant was proved to be correct, because the problem got solved.

The client's engineers are happy, too, because they have a solution. Indeed, it's the one they already proposed themselves, but couldn't get approved until it was validated by an expensive external consultant.

Most importantly, *you're* happy because you got a nice big check—and you earned it.

Time to leave?

Making the decision to leave a job is never easy. And it's a pretty drastic step, especially if you don't yet have another job to go to. But sometimes it has to be done.

Avoiding the comfort trap

If you're feeling miserable and finding your work unrewarding, you may find it relatively easy to flip the mental switch that says "time to leave". But sometimes you can run into the opposite problem: getting *too* comfortable where you are.

You may have fallen into the expertise trap we talked about in an earlier chapter, where you're the king of a very small hill, and one that will eventually be washed away by rising sea levels. If this is the case, you need to migrate to higher ground before it's too late. But many people will hesitate, fatally, because they don't like the idea of getting their feet wet.

Alternatively, you may find yourself in a job that's so easy you can basically do it on autopilot. This situation is much harder to leave, because it's so pleasant to be in.

And maybe it *is* okay for you to stay here for the rest of your career—if that's really what you want. Is it?

PETER: *I generally come in at least fifteen minutes late, and after that I sorta space out for about an hour. I just stare at my desk, but it looks like I'm working.*

I do that for probably another hour after lunch, too. I'd say in a given week I probably only do about fifteen minutes of real, actual, work.

—"Office Space"

Ultimately, work that's too easy is no fun, and it's not the basis of a rewarding career. All you're doing is selling *time*, and as you get a little older you'll come to realise that time is a non-renewable resource.

It's no good just selling your life; you won't be able to buy it back.

Why is everyone around me getting dumber?

Even if you enjoy your work and find it stimulating at first, especially if you're surrounded by lots of smart and skilled people, you may find that the higher you rise in the organisation, the less this is the case. If it seems like everyone around you is getting dumber, what's going on?

One *might* expect that people at the higher levels of a company would be more competent than those below, but this usually turns out not to be true, because of the Peter principle:

Although some people function competently, I observed others who had risen above their level of competence and were habitually bungling their jobs, frustrating their co-workers and eroding the efficiency of the organization.

*My analysis of hundreds of cases of occupational incompetence led me on to formulate **The Peter Principle**: In a hierarchy every employee tends to rise to his level of incompetence.*

—Laurence J. Peter, “[The Peter Principle](#)”

In other words, if you’re good enough at your job, you’ll be promoted to another job, and another, until you eventually reach a job that you *can’t* do well, at which point you’ll stay in it (possibly for the rest of your career).

Which explains a lot about some organisations, doesn’t it?

The good engineers are evaporating

Another reason you can find yourself adrift on a ship of fools is the “Dead Sea” effect:

The more talented and effective engineers are the ones most likely to leave—to evaporate, if you will. They are the ones least likely to put up with the frequent stupidities and workplace problems that plague large organisations; they are also the ones most likely to have other opportunities that they can readily move to.

What tends to remain behind is the ‘residue’—the least talented and effective engineers. They tend to be grateful they have a job and make fewer demands on management. They tend to entrench themselves, becoming maintenance experts on critical systems, so that the organization can’t afford to let them go.

—Bruce F. Webster, “[The Wetware Crisis: the Dead Sea effect](#)”

If you’re in a company like this, it’s not hard to stand out from the “residue”, and as a result you may be showered with promotions, fancy titles, and maybe even money. That sounds great, but there’s a hidden danger to watch out for.

Stranded by the tide

If you’re promoted too far, too soon, you may find that when you look for other jobs at the same level, you don’t really have the necessary skills for them.

For example, if you’ve already become a so-called “senior” developer at Company A, and then you apply for the same job at Company B, you may find that their definition of “senior” is rather different, and that you don’t meet it. You’re a victim of title inflation: the currency of “senior” has become devalued.

Hence there’s a tendency for you to stay at Company A, because who wants to take a step down in job grade and salary? If you find yourself surrounded by Principal and Distinguished Engineers and Architects who don’t seem to actually *know* anything useful, then they may be suffering from this kind of title inflation. Make sure you don’t become one of them.

The company won't love you back

Companies like to tell themselves pleasant little stories about how they're like a "family", everyone is a valued team member, the company will look after them, and so on.

The truth is that, however benevolent its messaging, a company exists to make profits and increase its own value. If this happens to benefit the people who work there, too, that's nice, but it's not what the company is fundamentally about.

Indeed, when the interests of the staff and the company's profits come into conflict, the profits will always win. Welcome to capitalism.

In particular, the HR department is not your friend. They're not your *enemy*, necessarily, but they exist to protect the company from you, and not vice versa.

HR exist to represent the interests of the company and those interests always have a degree of divergence with employees. It pays to get informed about your rights, because your HR and Legal teams are not going to do that for you.

This isn't to say your HR team are bad people. They're almost certainly not. They're just doing their job. But don't forget what their job is, and it's not to protect your interests, so make sure you have someone at the table who is.

—Colm Doyle, [“Having Friends in HR Is Fine, but HR Is Not Your Friend”](#)

When the company says wonderful things about how much it values you, don't take them quite at face value: after all, they *would* say that, wouldn't they? And when you're laid off, don't take that personally either.

The company simply doesn't have any feelings about you one way or the other, and once you know that, everything else about the way it treats you starts to make perfect sense.

When it's time to quit

When you finally make the decision to leave, whatever the reason, there's a right and a wrong way to go about it.

RICHARD: *So you're going to quit, just like that? How can you **do** that?*

GILFOYLE: *By saying the words “I” and “quit” in conjunction together, i.e. “I quit”.*

JARED: *Um... there's actually some paperwork involved.*

—“Silicon Valley”

The first thing to say is that your departure should not come as a surprise, at least to your team leader or line manager. This would embarrass them professionally—they're supposed to know what's going on with their reports—and there's no need to do that. Indeed, you *want* their goodwill, ideally in the form of a glowing reference, so you should do everything you can to smooth this potentially difficult transition.

In particular, you should give your boss a chance to change your mind—or, at least, you

should let them *feel* that they've had that chance. It's no good nursing your private resentments for years, while telling your boss once a week that everything's fine, and then suddenly walking out on them.

Instead, you should make sure that if you're unhappy about something, your boss knows it pretty much as soon as you do.

Say no to exit interviews

Companies will sometimes ask you to take part in an “exit interview”, usually presented as an information-gathering exercise where you can give honest feedback about your employment, and why you're leaving. Sometimes they'll ask for suggestions that could help improve things for the employees who are staying. Sounds innocent enough, right?

It's a trap. Don't agree to an exit interview: it can't benefit you, since you're leaving anyway. In fact, it could even hurt you. Once you start talking, there's a danger that you might say too much, venting your deepest frustrations and criticisms. That could harm your relationship with both the company and the people involved.

There are all sorts of ways that your former employers might retaliate. They might decide not to give a reference. They might say bad things about you at industry events. They could refuse to confirm your employment to a background investigator. They could call your new company and tell them you were fired for fraud.

These are all real stories: every single one of those hypotheticals is actually something I've seen happen.

—Jacob Kaplan-Moss, “[Exit Interviews Are a Trap](#)”

Instead, just politely decline the exit interview, or any other solicitation for feedback. They can't *make* you answer any questions. If you feel you can't refuse, then just give bland answers, like “nothing comes to mind”.

So, the doors close behind you, and a new world of possibility opens up ahead. After perhaps many years or even decades of doing what you're told, you've reached the threshold of a new relationship with your work: *independence*. That's the subject of our next, and final chapter.

12. Independence

My goal is to build a life I don't need a vacation from.

—Rob Hill Sr.



When you do decide it's time to branch out on your own, that can be a scary prospect. Ideally, as we discussed in the first chapter, you'll have already dipped a toe into the water by taking on some small side gigs or one-off tasks in your spare time.

The best way to make the transition from employment to independence less scary, then, is to make it gradually. You can start to apply all the advice in this chapter about building up your own business *before* leaving your full-time job, and that's what I recommend—providing your contract allows it.

This will give you some valuable experience of independent working, and it can be a good indicator of whether or not there's the potential for this to eventually become your full-time job.

Working to live

But, at some point, you have to take a leap of faith. There is, in fact, no guarantee that your bid for independence will achieve success. To maximise your chances, though, you can start by defining “success” in the right way.

Being rich is not the goal

Will you become vastly wealthy by starting a business? Unlikely. Of course, we hear about people doing so all the time, but it may have already occurred to you that the *reason* we're hearing about it is that it's so unusual. This phenomenon even has a name: *survivorship bias*.

But do you *need* to be vastly wealthy? Not really, as we saw in the earlier chapter on money. All you need is enough money to support you and your family in a decent lifestyle, and provide security for an uncertain future. And *that's* very achievable, if you go about it the right way.

Build a life, not a fortune

If your primary career goal is to earn lots of money, that's easy to achieve: just go to work for a bank, a government contractor, or any global mega-corporation. This is a much more reliable way of getting rich than starting a tiny business and trying to turn it into a big one.

What you *won't* get from working for a big firm is freedom, agency, quality of life, or job satisfaction. If you want those things, you'll need to start your own business. But you should be clear with yourself that you're *not* doing it primarily for the money.

Instead, I suggest you set out to build a *lifestyle business*: that is, one where you work to live, not live to work. If you only need to work one day a week to bring in enough money to live on, great—that gives you the rest of the week free.

Running a business *is* work

It might take some time to build up your business to the point where it can support you, though, and that's okay. Even if you work longer hours for your own business than you ever did when you were on salary, that's okay: you'll enjoy them more.

You'll never have a deeper, more exciting, or more meaningful relationship with work than when you're doing it just for yourself.

Being an independent business owner *can* be tough, though, especially at first, and you'll find there's a whole lot of stuff involved that you had no idea about, or have a clue how to do. Accounts, invoices, payroll, taxes, government filings, insurance, marketing, contracts, vendors; the list goes on.

It turns out when you worked for a company, someone else just took care of all these, probably in a department you never visited. Now *you'll* be that department. Oh, what fun!

But you *can* do these things, or (if you're wise) find professionals who will do them for you. Don't let the prospect of a little admin work put you off achieving your lifelong goal of complete independence. It's cheap at the price, and you'll never regret making the leap.

If it *really* doesn't pan out, well, you can always go back to working for someone else. But I don't think, once you've had a taste of independence, you'll want to do that.

Sources of revenue

So how do you actually get started? It's hardest at the beginning, when you have no clients, no products, no revenue, and no reputation. But, if you've already explored independent working a little, you'll have at least a few satisfied customers and good contacts. Exploit these first, but don't panic if no gigs seem to be coming your way. It might just take a while.

As a software engineer, there are a few different things you can do to make independent revenue. You can be a *consultant*, as we discussed in the previous chapter, helping companies solve software and development problems. You can build software *products*, and make money from them by selling licences or subscriptions. You can *teach* programming and related topics, and you can also make and sell *learning materials* along the same lines, such as books or video courses.

A combination of all these strategies is probably the way to go, at least until you've figured out which of them is the most successful for you, and best suited to your own strengths. Let's look at a few of these options in more detail.

Selling your time

Consulting and contracting—the two are not quite the same, as we saw in the first chapter—tend to attract a good hourly rate, so at first they may be responsible for the majority of whatever income you can make.

What you're selling as a contractor is simply your *time*, and companies will usually turn to contractors not for some special expertise or skill that they don't already have in-house, but just as an additional or temporary resource at busy times.

Pricing isn't an issue when you're contracting, because the price will usually be set by the client and won't be negotiable. If you try to negotiate it, they'll just go to someone cheaper, because contractors are seen as *fungible*: there's no meaningful difference between one and another. This isn't true, of course, as you and I know, but most *clients* don't know that, or don't care.

Consulting is a trickier and more nebulous business, requiring more experience to do well. Indeed, as we've seen, even if you can do it well, you may have to learn to deal with failure.

Clients will come to you as a consultant because they have a problem they can't solve on their own, or for assistance with some project that they don't have expertise about, but you do. This is where the experience you've gained over your career so far, both broad and deep, will come into play.

Making money

Whether you're selling just your time, or your time plus your expertise, one of the first problems you'll face is figuring out how to charge clients for your services, knowing how *much* to charge, and making sure that you actually *get* that money.

No matter how rewarding your work, and no matter how much you enjoy the independence it gives you, your business needs to make money to survive. How do you do that? Let's take a look at some of the problems and pitfalls.

How much should you charge?

The question of how to put a price on your services is an important one, and this may be the first time you've tried to answer it. Clearly it matters a lot what you charge, and you probably don't have many good reference points to reassure yourself that you're not wildly off in one direction or the other.

But it's really quite simple if you go about it the right way:

First you define your absolute floor. What's the minimum you would accept for an hour's worth of work before you thought to yourself "This isn't worth it!" Now add \$50 to that amount.

Every time someone asks you for a price quote on work, add \$50 more to your hourly rate. Keep doing this until someone says no. The price you were charging before you got to the "no" is your going rate.

—Marianne Bellotti, ["Yes, You Can Charge More"](#)

You *should* charge an hourly or daily rate, by the way, even though clients will do everything they can to persuade you into a fixed-price deal. For example, they might say "We'd like you to implement Feature X for us, and we'll pay \$5000 for the job".

That sounds okay, so what's wrong with fixed pricing?

Don't take fixed-price contracts

Well, let's suppose you complete Feature X to the highest possible standard, and deliver it to the client... only for them to say "That's great, but there are a few little issues."

It turns out what *you* thought Feature X meant isn't at all what *they* had in mind. It's more complicated than it seemed at first, because there are all kinds of complications and implications that they didn't really make clear in the brief.

For example, it needs to work with Feature Y that already exists, and the library that you used isn't compatible with that, so you'll need to change it. Plus, when they showed your Feature X to users, they discovered a whole bunch of little changes that weren't foreseen in the original spec.

The problem for *you* is that, from your point of view, you've done what you were asked, but now the client won't pay unless you submit to all their new demands. If you say no, you lose out on a chunk of revenue.

If you say yes, though, you're basically telling them that they can keep asking you for more things, and you'll do them for free. That's a terrible precedent to set. It's like giving in to blackmail: if you do it once, you'll have to keep on doing it, as long as you're willing to be an easy mark.

The best option for both sides, then, is what's called a *time and materials* contract, where you work for as many hours as necessary to deliver what the client wants, and they pay you based on the time worked. That way, you won't be unhappy if the client changes their mind, because you'll just earn more as a result.

Estimates and change budgets

However, no client wants to commit to a deal where they have no idea how much they'll end up paying. Because of the way company finances work, that usually isn't possible anyway: they'll need to get approval in advance for a *purchase order* that specifies a fixed amount for a fixed set of deliverables.

To resolve this apparent paradox, you can give the client a detailed breakdown of the work that's involved in delivering Feature X, how long you think it'll take you, and how much that will cost them at your hourly rate.

Get it clearly agreed in writing, though, that this is only a good-faith *estimate*, and that if the work takes longer than planned, it will cost them more. (On the other hand, it *might* get finished early. Miracles do happen from time to time.)

You should also build in a "change budget": a small number of hours set aside for changes, new requirements, rework, and so on. Ten percent of the total time is reasonable, and this gives the project a little breathing room to evolve over time, without busting the client's budget or putting you out of pocket.

It's wise to agree with the client that if you run into problems, delays, or anything else that might mean the project costing more than expected, you'll contact them straight away and discuss how to proceed.

Don't just go silent for months and *then* send them a huge, unexpected bill. That's not a good way to get repeat business. Instead, keep the client informed at every stage, even when things are exactly on track—because they need to know that, too.

Ratcheting up your rates

The "think of a number and add \$50" trick is a good way to rapidly evolve your hourly rate to somewhere in the right ballpark. After that, you'll be able to gradually increase it further as time goes on and your business grows in revenue and reputation.

And this is a good thing, because once you've sold all the time you have available, the only way to make more money is to charge more for each hour.

If you're getting more offers of work than you can handle, raise your rate a little each time. Yes, you may lose out on some clients because they're not willing to pay your price, but that's fine. The clients you *do* land will value you all the more *because* you charge a lot.

When your rate has reached a level so high that you have as much work as you can handle, but no more, it's probably about right.

Don't drop your price to get the deal

Beware of those clients who ask for discounts. They are essentially telling you "I don't think your work is worth what you're charging for it." By giving them a discount, you're effectively admitting that they're right. Resist the temptation, even if you could use the extra business.

Clients who want cheap rates are cheapskates, and if they *get* your services at a low price, they'll value them accordingly. Indeed, it's precisely the clients that pay you the least who'll demand the most from you because, secretly, they still think they're paying you too much even *with* the discount.

The vendor-client relationship

This may be the first time in your career that you've been a *vendor*; someone who sells a product or service to clients. While it's easy to spot a bad vendor when you're the client, now the relationship is reversed. What does it mean to be a good vendor, and how can you identify and deal with bad *clients*?

When the client doesn't pay

It's not all good news about running your own business, I'm afraid, and one of the more melancholy aspects of it is chasing unpaid invoices. This will occupy a surprising and irritating amount of your time, which could be better spent doing paying work, but it still has to be done.

The terms of payment for your invoices should be very clear: thirty days from date is usual. You should be equally clear with your clients that you expect them to pay on time, and what the consequences will be if they don't.

It's a good idea to agree explicitly with them that you're entitled to charge a penalty for late payments. Most jurisdictions have some kind of statutory rule about this, but you can set whatever terms you want, so long as you can get the client to agree to them. For example, you might charge a flat fee of \$100 for a late payment, plus a certain percentage of the overdue amount per month.

With luck, you'll never have to actually apply this penalty. If you do, that is probably a sign that you should dispense with this client's business as soon as possible—once you've recovered what they owe you, of course.

How to fire a client

If you've never run a business, this might all sound alarming and unnecessary. After all, you reason, *I* pay my bills promptly and in full. Doesn't everyone else? Well, yes, mostly they do. But it's the 20% of clients who don't that will take up the majority of your business admin time in chasing them.

Don't get angry; it's a commercial issue, not an emotional one. The first time an invoice is overdue, send them a polite reminder. The second time, apply the penalty charge. The third time, fire the client.

In fact, you shouldn't hesitate to fire any client who gives you trouble, either because they don't pay, demand too much, constantly ask for changes, quibble over invoices, badger you with emails and phone calls, or simply don't seem to value you and your work.

Trust your first impressions

The best outcome for all concerned, in fact, is for you to never take on such a client in the first place. When you have your initial contact with them, whether it's by email, phone, or whatever, pay attention to first impressions: they're almost always correct.

There's a strong temptation to ignore your instinctive negative reaction to a potential client, especially if you could really use the work. But your subconscious is smarter about these things than you are. Trust what it's telling you.

Even if you can't say exactly what it is that doesn't seem right, and maybe it's just a weird vibe—then this client isn't for you.

How to be a great vendor

A big reason most vendors are horrible to deal with is *poor communication*. You know: when you email them, you get an auto-reply saying that your message is important, and will be dealt with as soon as possible. Five days later, you get a one-line response saying they can't deal with your request by email, and you should instead use the live chat facility on their website.

As well as being obtuse, obstinate, and unresponsive, these companies are usually so big that your business means absolutely nothing to them. It's the smallest imaginable drop in their ocean of revenue. You have zero leverage with a vendor like this, because you need them more than they need you, and that's always frustrating.

So, when *you're* the vendor, don't make this mistake. Make sure you communicate with your clients in the way that they want. That doesn't mean you need to be available and responsive to them 24/7, but it does mean that they need to *know* how to reach you and when to expect a response.

For example, you might set your clients' expectations that they won't necessarily get an instant response to emails or messages, but they will hear back from you within 24

hours, without fail. Then stick to that. Whatever rules you set for your relationship with clients, make them clear, communicate them, and follow them.

“Policy” just means “we don’t want to”

Another thing that drives me to fury when dealing with a poor vendor is *company policy*. Again, you know how this goes: “We’re sorry to hear that your car blew up on the way home from the dealership, but unfortunately it is not our policy to offer refunds in such cases.” Company policy seems to cover everything, and it never seems to work in your favour.

As a client of such a company, it feels like dealing with some huge and heartless machine rather than with fellow human beings. Even if the person you’re talking to is perfectly pleasant and nice, regrettably, there’s nothing they can do to help you, because it’s *not company policy*.

But who sets the company’s policy? They do, of course. “Policy” is just another way of saying “We don’t want to do this, so we won’t.” My own policy, on encountering a vendor like this, is to fire them and look for someone else who seems to value my business a little more.

So the secret of making your *own* business delightful to deal with is being human, not a corporate policy droid. Be warm, be friendly, be interested, be flexible, be responsive, be respectful, and you’ll soon be successful too.

Building products

Selling your time is likely to be the most rewarding activity for your business, especially at first, but it shouldn’t be the only one you pursue.

Running out of time

As a pure time-seller, you can’t take a vacation, you can’t spend time on things that don’t make money, and you can’t get sick, or disabled, or otherwise unable to work for some reason. You also can’t retire without your income dropping to zero, which isn’t much fun.

You need something else you can sell, apart from your time: in a word, *products*.

As a software engineer, you might immediately think of software products: applications, services, frameworks, plugins, and so on. But it’s actually rather difficult to make money from these.

Selling software is hard

One problem is that there’s so much great software available for free. For no money, people can get a very decent development environment, word processor, email service,

and so on. Why should they pay for what *you're* selling? Only a tiny fraction of people will even consider this, no matter how good your product.

To create a software product that's so far ahead of any free alternative that you could actually make a living wage from it, you'll need to invest an eye-popping amount of time and effort. Major commercial software applications often take thousands of person-hours to develop, and many more to maintain.

Doing this kind of work on your own could take many years, and in the meantime you don't have any revenue to support you. So you'll need to sell your time, and then you won't have any time to work on the product. Foiled again.

On the other hand, maybe you have such an ingenious *idea* that it doesn't matter how little time you have for development. It's just a brilliant invention that no one else has thought of, and it's so simple that you can build it in just a few months of evenings and weekends.

Congratulations, but the chances are that your product won't succeed. If you don't have much experience of product development, you'll be amazed at the insanely great products that people simply won't buy.

If it's an idea no one has thought of before, then presumably it's not addressing a problem that many people *have*. Therefore, they won't be interested in buying it, because they don't know they need it, and it will be tough to persuade them that they do.

Expect tough competition

On the other hand, if it *is* something everybody needs and will buy, then even if you're the first person to offer it, you won't be the last. Another, bigger, company is bound to spot the opportunity and swoop in.

Their version of the product might not be as good as yours, but it'll be better marketed, and they can probably afford to sell it at a loss. At least, until they've put you out of business. Then they'll raise the price to a highly profitable level, of course.

It's been done before, many times. If you're lucky, the big company will buy you out, to save themselves the trouble of competing with you. If you're unlucky, they'll just bury you and take all your business. If you're *really* unlucky, they'll also sue you for stealing their intellectual property.

Knowledge products

An alternative that you might not have considered is making your *knowledge* itself into a product. If you can do something that most people can't, such as writing software, then you can *sell* that skill, not by the hour, but in the form of a book, video, or online course.

It takes a long time to write a book, but it takes even longer to acquire the knowledge you *put* into the book, so why shouldn't you turn that time and effort into money? It doesn't matter if you're a particularly good writer or not. People aren't buying a book

on programming for the author's immaculate prose style (thank goodness), but because they need the information it contains.

You don't need to work with a publisher to write a book, either. A traditional publishing firm will take 90% of the profit from selling your book, and in return... well, I'm not really sure what they'll do for you in return.

Yes, they'll fix your typos, but you could do that yourself, or hire a copy editor to do it for so much a page. In *theory*, a publisher should also market the book for you, though I've never actually known this to happen.

If you can put some text together and turn it into a PDF, and I'm pretty sure you can, then you're essentially there. You can make the PDF available for download on your website, and use an online payment provider to process the sales.

Don't use a self-publishing service such as Kindle Direct Publishing; again, Amazon will take an enormous chunk of your revenue for doing basically nothing except web hosting.

You can be your own publisher

If someone else is publishing your book, then it's not really your book: it's theirs. Your name might be on the cover, but if someone else has the final say over the topic, the layout, the title, the cover design, and the price, then you're essentially just a hired hand.

There's nothing wrong with that in itself; many people have written books through traditional publishers, myself included. They may even have gained some valuable reputation and visibility, which in turn can generate leads for consulting, contracting, or teaching work.

If you want to make a *business* out of your books, though, you need to keep 100% of the revenue yourself. You won't sell as many copies as you would through a publisher, because your book won't be on Amazon or in stores and so on. But that's okay, because all the money from what you *do* sell will go to you, not some giant corporation.

Don't over-think the marketing

If the product isn't good, then anything you do to market it would be a waste of time. Indeed, it would actively harm your business: the last thing you want is your name associated with a low-quality product.

Therefore, until the product is *excellent*, focus the majority of your energy on improving it. Having a great product is the best marketing strategy.

A truly delightful product is what you need in order to turn your customers into *fans*: people who love your stuff so much, they'll buy almost anything you make.

To be a successful creator you don't need millions of dollars or millions of customers. If you have roughly a thousand true fans, you can make a living—if you are

content to make a living but not a fortune.

—Kevin Kelly, “1,000 True Fans”

Your true fans are your marketing department

You also don't need to promote your work on social media. If the work is good, your true fans will promote it *for* you, and this is always much more effective than anything you could do yourself. Of *course* the author thinks his book is good, but that's not going to convince anyone else to buy it.

We're all very resistant to traditional marketing, especially online; the harder you work to promote your product, the more we'll simply tune you out. A recommendation from someone we *know*, though, is a far more powerful driver of sales.

Put your product out there, announce it in the right places, and then let word of mouth take its course. You won't see instant results, but over the long term, it will work, and go on working.

Control

While you can sell partial control of your business, you can't always buy it back. That suggests you should keep an even closer watch on your control than you do on your money.

Independence, or co-founders? Pick one

Similarly, you should be very wary about taking on partners or co-founders. Running a business with someone is a very intimate kind of relationship, and it's often built on rather shaky ground. If it's someone you don't know very well, there's a danger that the relationship won't work out, and it's the most crucial relationship you'll ever have.

On the other hand, if you co-found a business with a friend or relative that you *do* know well, that can go even more badly wrong. As the saying goes, when you lend money to a friend, you're likely to end up losing either the friend, or the money, or both. Now imagine that friend is your business partner.

Friends tend not to sign contracts with each other, and that would be a big mistake if you're going into business together. At the beginning, of course, you're both cheerful and optimistic, the future is rosy, and there's no problem that can't be solved.

When things go badly, though, tempers can start to fray, conflicts can arise, and friendships can hit the rocks. What happens, for example, if your co-founder suddenly presents you with a demand for increased equity, because they claim they've put more time and energy into the business than you have? Or what if they decide to kick you out of the company altogether and keep your share?

Keep ultimate control

Never give up more than 50% of the company's voting shares, because with them goes control. Even if you're still the biggest shareholder, that doesn't necessarily mean you have control.

For example, suppose you go into business with two other people, and you split the equity equally between the three of you. Now control of the company resides with whoever has two of these three votes. So, if your co-founders decide to gang up on you, there's nothing you can do. They can fire you, remove you from the board, and replace you with anyone they want.

Professional legal advice is crucial in any situation where you're giving up partial control of the company to someone else. Don't rely on the *company's* legal counsel; get your own. Better still, just keep all control in your own hands and don't take on any co-founders, investors, or "advisers".

Whatever your company structure, make sure it stays *your* company. That means you, and only you, have the final say on everything.

There shall be only one tai-pan at one time and he hath total, absolute authority over the Company, power to employ or remove from employment all others, authority over all our captains and our ships and companies wherever they may be.

The tai-pan is always alone, that being the joy and the hurt of it. Whatsoever he orders, it shall be obeyed, and no committees or courts or inner circles shall ever be formed or allowed in the Company to curb this absolute power.

—James Clavell, “[Noble House](#)”

A business does not need to grow indefinitely

When *should* your business expand beyond just one person? Well, maybe never. Despite what you might read in the financial press, it is not actually necessary for a company to grow. It doesn't *need* to continually increase its sales, its revenues, its profits, or its headcount. It happens that most companies *do* grow, at least if they can, but that's incidental.

Indeed, there's only so much you *can* grow a single-person business, as we've seen, and that's okay. Once you're making enough to live comfortably on, you're fine. Once again, if you want to make big money, work for a bank.

If you want to work for yourself, though, do just as much business as you need to, and no more.

Don't feel under pressure to get more and more clients, or take on more and more work. That will just lead to stress, unhappiness, and burnout: all the things you quit your salaried job to avoid.

Hiring: whether, when, and how

If you do want to grow the business beyond what you can do on your own, when is the right time to think about taking on an employee? Well, you should first of all make sure that hiring *is* your only option. If you can increase revenues some other way, such as raising your prices, or targeting new markets, do that first.

Don't hire someone until the business is making enough profit to pay their salary as well as your own—not just this month, but for at least a year ahead. Don't offer them equity as a substitute for cash; that would mean giving up your hard-won agency, and, as we discussed in an earlier chapter, it's not really a good deal for them either.

When you do hire, do it very, very carefully and thoughtfully. Take all the time you need over the application and interview process, and consider as many possible candidates as you can. Hiring the wrong person is a disaster, and if it's your very first hire, it might be one that the company can't recover from.

Don't just hire the first person who walks in, seems keen and competent, and is willing to work for a low wage. (Actually, that's a bad sign. If their work is any good, why are they selling it so cheaply?) Depending on your local employment laws, getting rid of an employee who doesn't work out can be very difficult, time-consuming, or expensive (likely all three).

In the chapter on interviews, we saw that it's much, much better for a company to reject a good candidate than to hire a bad one, which is one reason that the interview process can sometimes seem so ridiculously demanding. Well, now that you're on the other side of the desk, it may start to make more sense.

If at all possible, don't use recruitment consultants, agencies, or job boards, or even advertise the vacancy. Just put out feelers among your network of contacts, friends, and ex-colleagues. If you know someone who would be perfect for the job, get in touch with them. If not, you probably *know* someone who knows someone.

Hiring someone on personal recommendation, as we saw when looking at the process from the other end, is by far the cheapest, quickest, and most reliable way of hiring.

And if your requirement is likely to be temporary, or even if it's not, using a contractor may be a more cost-effective way to fulfil it. It's also a good way to see if someone's going to work out in the job: hire them on a daily rate for a while. You can always offer them a permanent position later on if you want to.

An army of one

You may never need to hire anyone at all, which is ideal from the point of view of independence. When you have employees, you instantly become a manager, whether that's what you want to be or not, and whether you're any good at it or not (most aren't).

You will no longer be able to spend much time on the main activity of the business; instead, you'll be much too busy supervising those who *are* doing it. They won't do it as

well as you would, because if they did, they'd probably have their *own* company instead of working for you.

So once you take on your first employees, you'll be busier than you were before, but the company's products won't be as good. And most of your profits will now be soaked up by payroll.

As strange as it may seem, you're actually *less* independent when you have employees than when it was just you. You're the boss, of course, but as we've learned throughout this book, being the boss doesn't mean doing exactly what you want to do, regardless of what anyone else thinks. In some ways it's just the opposite.

So you may well find that your maximum independence comes with being a small company, and staying that way. If you thought your goal was to grow the company as fast and as far as possible, staying small might feel like a failure. But if your goal is *independence*, then staying small might be the greatest success of your career.

Making it

What's the end game? When will you be done, when can you retire, and what does that even mean for you? Starting a business can be hard work, though rewarding, and building that business up to a level where it can support you is even harder, and can take many years.

What does “making it” mean for you?

Still, there will come a day when, whether you realise it or not, all your striving has achieved its goal, and you'll have made it. What “made it” means is different for everybody.

One person may feel that they've made it when they have the agency, or independence, in their job that they need to be happy. That doesn't necessarily correspond with seniority, pay, or status, as we've seen. You could be in what seems to the outside world like a low-level job, but to you it might be the most rewarding job in the world, and one that you never want to leave. If that's so, then you've made it.

For another person, making it might mean that they're in a secure enough position to quit their miserable day job and focus on the work they really enjoy, whether or not that's hugely lucrative: rewards come in many forms other than money.

Someone else might feel they've made it when they can turn the day-to-day operation of their business over to someone else, and just concentrate on the major strategic decisions. Or it might even mean they no longer need to work at all, and can retire in comfort to a beach somewhere.

It's worth asking yourself every once in a while: have you made it yet? If not, what would that look like?

Pre-tirement

Perhaps you'll feel you've made it when you're free to do only the work you *want* to, and nothing else. If the project is interesting or fun, you can take it on, and if it isn't, you can pass. You control what you do, how you do it, and *whether* you do it. This is the ultimate agency.

I call this delightful state *pre-tirement*. If you choose to spend all day surfing, or learning Chinese calligraphy, or building houses for the homeless, you can do those things. But if you want to write software and practice your craft, you can do that, too.

When you own your own business, there's no mandatory retirement age. Why *should* you stop, if you're having fun? No one can force you.

The thing about having made it is that you can relax. You've done it. You're there. You may have felt your whole life that you should always be striving, and that if you aren't, you're somehow indefinitely letting people down. Well, you can stop feeling that way now. It's served its purpose.

You need to define the rules of the game explicitly enough that you can tell when you've won, or you'll never be able to stop playing. Once you *have* won, though, pack up the pieces and put the game back on the shelf, because you won't need it any more.

But this isn't the end; far from it. Now there are some exciting new games for you to play.

The beginning

One thing you might like to do with your now wonderfully free time is to find ways to give back a little. If you enjoy writing software, but no longer need to spend most of your time doing it for money, you can contribute to open source and free software projects.

You can teach free classes for kids or anyone else who wants to learn to code. You can mentor young software developers, or even middle-aged ones. You can write books, make videos, and give talks on what you've learned, because there's always someone else who can benefit from it.

You have a lot to share, and a lot to give, if you want to. You may find that, having reached the end goal of your career, you feel in need of a new challenge.

In that case, helping other people discover and achieve their own career goals could be just what you're looking for. And it could be the beginning of a new and delightful phase of your own life.

So let's get started.

About this book



Who wrote this?

[John Arundel](#) is a software engineering teacher and mentor of many years experience. He's helped literally thousands of people to learn to program, with friendly, supportive, professional mentoring, and he can help you too. Find out more:

- [Learn with John](#)

Feedback

If you enjoyed this book, let me know! Email go@bitfieldconsulting.com with your comments. If you didn't enjoy it, or found a problem, I'd like to hear that too. All your feedback will go to improving the book.

I'm not a global mega-corporation, and I don't have a publisher or a marketing budget: I write and produce these books myself, at home, in my spare time. I'm not doing this for the money: I'm doing it so that I can help bring the joy of programming to as many people as possible.

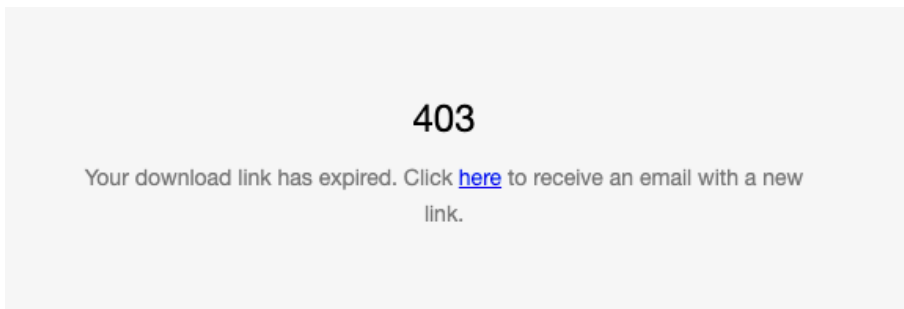
That's where you can help, too. If you liked this book, please tell a friend!

Free updates to future editions

All my books come with free updates to future editions, for life. Make sure you save the email containing your download link that you got when you made your purchase. It's your key to downloading future updates.

When I publish a new edition, you'll get an email to let you know about it (make sure you're [subscribed](#) to my mailing list). When that happens, re-visit the link in your original download email, and you'll be able to download the new version.

NOTE: The Squarespace store makes this update process confusing for some readers, and I heartily sympathise with them. Your original download link is only valid for 24 hours, and if you re-visit it after that time, you'll see what *looks* like an error page:



But, in tiny print, you'll also see a “click here” link. Click it, and a new email with a new download link will be sent to you.

Join my Go Club

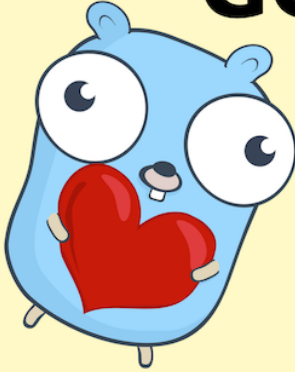
I have a mailing list where you can subscribe to my latest articles, book news, and special offers. I'd be honoured if you'd like to be a member. Needless to say, I'll never share your data with anyone, and you can unsubscribe at any time.

- [Join Go Club](#)

For the Love of Go

[For the Love of Go](#) is a book introducing the Go programming language, suitable for complete beginners, as well as those with experience programming in other languages.

FOR THE LOVE OF GO



John Arundel
@bitfield

"A great way to dive into Go!"
—Max VelDink

If you've used Go before but feel somehow you skipped something important, this book will build your confidence in the fundamentals. Take your first steps toward mastery with this fun, readable, and easy-to-follow guide.

Throughout the book we'll be working together to develop a fun and useful project in Go: an online bookstore called Happy Fun Books. You'll learn how to use Go to store data about real-world objects such as books, how to write code to manage and modify that data, and how to build useful and effective programs around it.

The Power of Go: Tools

Are you ready to unlock the power of Go, master obviousness-oriented programming, and learn the secrets of Zen mountaineering? Then you're ready for [The Power of Go: Tools](#).

THE POWER OF GO TOOLS



ADVANCED SOFTWARE ENGINEERING IN GO

JOHN ARUNDEL

It's the next step on your software engineering journey, explaining how to write simple, powerful, idiomatic, and even beautiful programs in Go.

This friendly, supportive, yet challenging book will show you how master software engineers think, and guide you through the process of designing production-ready command-line tools in Go step by step.

The Power of Go: Tests

What does it mean to program with confidence? How do you build self-testing software? What even is a test, anyway? [The Power of Go: Tests](#) answers these questions, and many more.

THE POWER OF GO TESTS

“Engaging, friendly, informative, and snappy”
—Kevin Cunningham



JOHN ARUNDEL

Welcome to the thrilling world of fuzzy mutants and spies, guerilla testing, mocks and crocks, design smells, mirage tests, deep abstractions, exploding pointers, sentinels and six-year-old astronauts, coverage ratchets and golden files, singletons and walking skeletons, canaries and smelly suites, flaky tests and concurrent callbacks, fakes, CRUD methods, infinite defects, brittle tests, wibbly-wobbly timey-wimey stuff, adapters and ambassadors, tests that fail only at midnight, and gremlins that steal from the player during the hours of darkness.

If you get fired as a result of applying the advice in this book, then that's probably for the best, all things considered. But if it happens, I'll make it my personal mission to get you a job with a better company: one where people are rewarded, not punished, for producing software that actually works.

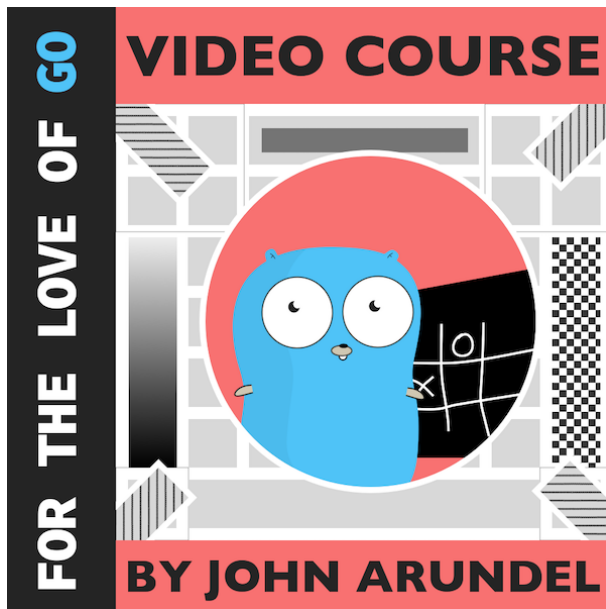
Go's built-in support for testing puts tests front and centre of any software project, from command-line tools to sophisticated backend servers and APIs. This accessible, amusing book will introduce you to all Go's testing facilities, show you how to use

them to write tests for the trickiest things, and distils the collected wisdom of the Go community on best practices for testing Go programs. Crammed with hundreds of code examples, the book uses real tests and real problems to show you exactly what to do, step by step.

You'll learn how to use tests to design programs that solve user problems, how to build reliable codebases on solid foundations, and how tests can help you tackle horrible, bug-riddled legacy codebases and make them a nicer place to live. From choosing informative, behaviour-focused names for your tests to clever, powerful techniques for managing test dependencies like databases and concurrent servers, [The Power of Go: Tests](#) has everything you need to master the art of testing in Go.

Video course

If you're one of the many people who enjoys learning from videos, as well as from books, you may like the video course that accompanies the 'For the Love of Go' book:



- [For the Love of Go: Video Course](#)

Further reading

You can find more of my books on Go here:

- [Go books by John Arundel](#)

You can find more Go tutorials and exercises here:

- [Go tutorials from Bitfield](#)

I have a YouTube channel where I post occasional videos on Go, and there are also some curated playlists of what I judge to be the very best Go talks and tutorials available, here:

- [Bitfield Consulting on YouTube](#)

Credits

Gopher images by [MariaLetta](#), except:

- [Doctor Gopher](#) T-shirt design © Women Who Go

Acknowledgements



My thanks are due to my students at the [Bitfield Institute of Technology](#), from whom I've learned nearly everything useful that I know. The many mistakes I've made in my own career provided some insights, too.

Special thanks to the beta readers who read the book carefully, and criticised it constructively (and kindly). In particular, I'm grateful for the very valuable feedback from Joumana El Alaoui, Ehsan Ashouri, Steve Baros, Maurice Le Cordier, Anish CR, Christian Drexler, Ivan Fetch, Jakub Jarosz, Mythili Kamath, Jackson Kato, Shiv Patil, Andy Pearson, Adam Read, Umang Vanjara, and Miloš Žižić (who also created the “senior engineer” graphic).

If you'd like to be one of my beta readers in future, please go to my website, enter your email address, and tick the appropriate box to join my mailing list:

- <https://bitfieldconsulting.com/>